

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки Кафедра інформаційних
систем та технологій

Звіт з практичних робіт
з дисципліни «Інтернет-технології та проектування
WEB-застосувань»
на тему: «ОНЛАЙН-МАГАЗИН ТЕХНІКИ»

Виконала:

студентка 2 курсу, групи ІС-44 Баришева Анастасія Володимирівна

Прийняла:

Старший викладач кафедри ІСТ Проскура Світлана Леонідівна

Київ – 2025 року

ЗМІСТ

1.	АНАЛІЗ ТА МОДЕЛЮВАННЯ СИСТЕМИ (ПРАКТИЧНА РОБОТА 1)	7
	Тема:.....	7
	Актуальність теми:	7
	Мета роботи:	7
	Задачі роботи:.....	7
	Об'єкт роботи:.....	7
	Предмет роботи:	8
	Бізнес-логіка:.....	8
	Основний функціонал:	8
	Головна сторінка:.....	8
	Сторінка «Категорії»:	10
	Сторінка «Техніка»:.....	12
	Сторінка «Інформація»:	13
	Кабінет користувача:	15
	Функціональні вимоги:.....	15
	Нефункціональні вимоги:.....	16
	Use-case.....	16
	ER-діаграма	16
	Висновки:	17
	Контрольні питання	18
	ДОДАТОК А	20
	ДОДАТОК Б.....	21
2	РОЗРОБЛЕННЯ АДАПТИВНОГО ІНТЕРФЕЙСУ WEB-ЗАСТОСУНКУ (ПРАКТИЧНА РОБОТА 2)	22
	2.1 Мета роботи	22
	2.2 Реалізація структурних компонентів інтерфейсу	22
	2.2.1 Реалізація компоненту header	22

2.2.2 Реалізація компоненту main.....	27
2.2.3 Реалізація сторінок типів товарів.....	35
2.2.4 Реалізація сторінки товару.....	44
2.2.5 Реалізація компоненту кабінет користувача	54
2.2.6 Реалізація сторінки реєстрації.....	67
2.2.7 Реалізація сторінки авторизації.....	72
2.2.8 Реалізація сторінки інформації.....	76
2.2.8 Реалізація компоненту footer	94
2.3 Функціонально стилістичні рішення.....	96
2.3.1 Адаптивна навігація	96
2.3.2 Адаптивна сітка контенту	96
2.3.3 Реалізація медіа-запитів	97
2.3.4 Застосування відносних одиниць вимірювання	97
2.3.5 Реалізація візуальних ефектів та анімацій	97
2.4 Файлова структура веб-застосунку	100
2.5 Репозиторій на GitHub	101
2.6 Висновки до розділу	101

3 АДМІНІСТРАТОРСЬКА ЧАСТИНА WEB-ЗАСТОСУНКУ: ІНТЕРФЕЙС ТА CRUD-ОПЕРАЦІЇ (ПРАКТИЧНА РОБОТА 3)

3.1 Мета роботи	103
3.2 Візуальне відображення в різних станах сайту в залежності від стану користувача	103
3.2.1 Візуальне відображення для гостя	103
3.2.2 Візуальне відображення для зареєстрованого користувача	104
3.3 Реалізація адмін панелі	105
3.3.1 Візуальне відображення для Адміністратора	105
3.3.2 Реалізація головної сторінки для Адміністратора.....	107
3.3.3 Реалізація сторінки редагування категорії виробів для Адміністратора	112
3.4 Файлова структура адмінської частини проекту	120

3.5 Репозиторій на GitHub	121
3.6 Висновки до розділу	121
3.7 Контрольні запитання	122
4 СТВОРЕННЯ РЕЛЯЦІЙНОЇ БАЗИ ДАНИХ ВІДПОВІДНО (ПРАКТИЧНА РОБОТА 4)	124
Мета роботи:	124
Опис робочого середовища	124
4.1 Створення базових таблиць бази даних, відповідно до ER-моделі, покривши «бізнес-об'єкти» сайту	125
4.1.1 Створення таблиць	125
4.1.2 Заповнення таблиць	127
4.1.3 Створення зв'язків між таблицями	129
4.2 Створення таблиць для майбутньої рольової моделі	133
4.2.1 Створення таблиць для рольової моделі	133
4.2.2 Заповнення таблиць для рольової моделі	134
4.2.3 Створення зв'язків між таблицями для рольової моделі	134
4.3 Визначені ключі та обмеження бази даних	136
4.3.1 Primary key	136
4.3.2 Foreign key	136
4.3.3 Unique	137
4.3.4 Check	137
4.3.5 каскадні правила (ON DELETE/UPDATE)	137
4.3.6 Індeksi	138
4.4 Схема зв'язків та сутностей бази даних	138
4.5 Тестові SELECT-запити та оператори	138
4.5.1 Оператор №1: Показує кількість інвойсів та загальну суму по кожному клієнту.	138
4.5.3 Оператор №3: Показує всі продукти, їх кількість на складі та адресу складу.	139
4.6 Репозиторій на GitHub	140
4.7 Висновки до розділу	140

4.8 Контрольні запитання	141
--------------------------------	-----

5 РОЗГОРТАННЯ БЕКЕНД-СЕРВЕРА (ПРАКТИЧНА РОБОТА 5)..... 144

Мета роботи:	144
--------------------	-----

5.1 Опис структури серверної частини	144
--	-----

5.1.2 Принцип взаємодії компонентів серверної частини:	146
--	-----

5.2 Підключення до реляційної бази даних	146
--	-----

5.2.1 Код, де реалізоване підключення до реляційної бази:	146
---	-----

5.2.2 Приклад роботи з даними:	147
--------------------------------------	-----

5.3 Інтеграція фронкенду з серверною частиною	149
---	-----

5.3.1 Пошук виробів на головній сторінці	149
--	-----

5.3.2 Завантаження виробів за типом	151
---	-----

5.3.3. Завантаження даних окремого товару	152
---	-----

5.4 CRUD-операції у серверній частині	153
---	-----

5.4.1 Операція GET/READ	154
-------------------------------	-----

5.4.1 Операція POST/CREATE	155
----------------------------------	-----

5.4.3 Операція PUT/UPDATE	156
---------------------------------	-----

5.4.4 Операція DELETE/DELETE	158
------------------------------------	-----

5.5 Репозиторій на GitHub	160
---------------------------------	-----

5.6 Висновки до розділу	160
-------------------------------	-----

5.7 Контрольні запитання	161
--------------------------------	-----

6 АВТОРИЗАЦІЯ ТА РОЗМЕЖУВАННЯ ПРАВ ДОСТУПУ (ПРАКТИЧНА РОБОТА 6)..... 164

Мета роботи:	164
--------------------	-----

6.1 Реалізація моделі користувачі і ролей.....	164
--	-----

6.2 Розробка механізму автентифікації.....	166
--	-----

6.3 Розмежування прав доступу.....	168
------------------------------------	-----

6.4 Безпека API та обробка помилок	169
--	-----

6.5 Інтеграція з фронкендом.....	170
----------------------------------	-----

6.6 Тестування авторизації та прав доступу	172
6.7 Репозиторій на GitHub	172
6.8 Висновки до розділу	172
6.9 Контрольні питання	174

1. АНАЛІЗ ТА МОДЕЛЮВАННЯ СИСТЕМИ (ПРАКТИЧНА РОБОТА 1)

Онлайн-магазин техніки

«ByteShop»

Тема:

Теги та атрибути HTML-документа. Структурна розмітка. GIT. GitHub. Робота з репозиторіями. Опис логіки власного Web-застосунок.

Актуальність теми:

У сучасних умовах стрімкого розвитку цифрових технологій дедалі більше людей обирають онлайн-покупки, зокрема в сфері електроніки та побутової техніки. Актуальність теми пояснюється тим, що техніка відіграє важливу роль у повсякденному житті, забезпечуючи комфорт, ефективність роботи та відпочинку. Онлайн-магазин техніки дозволяє значно розширити аудиторію покупців, забезпечити швидкий доступ до широкого асортименту товарів, спростити процес придбання та підвищити якість сервісу як у межах України, так і за її межами.

Мета роботи:

Забезпечення зручного та швидкого вибору техніки онлайн, можливість замовлення та предзамовлення товарів, надання різних способів оплати й доставки, а також створення комфортних умов для покупців під час взаємодії з магазином.

Задачі роботи:

Проаналізувати потреби цільової аудиторії у сфері купівлі техніки; розробити сучасний дизайн та зручний інтерфейс онлайн-магазину; реалізувати функціонал каталогу товарів із фотографіями, характеристиками, описами та цінами; створити систему кошика та оформлення замовлень; інтегрувати платіжні системи і служби доставки; впровадити адміністраторську панель для управління асортиментом, цінами та відстеження замовлень; додати систему відгуків, рейтингів та рекомендацій для підвищення довіри покупців; реалізувати механізми акцій, знижок і гарантійного обслуговування.

Об'єкт роботи:

Інформаційна система електронної комерції у сфері продажу техніки та

електроніки.

Предмет роботи:

Програмне забезпечення (онлайн-магазин), яке забезпечує управління асортиментом, продажами, замовленнями, платежами та комунікацією з клієнтами.

Бізнес-логіка:

Для оформлення замовлення користувач проходить авторизацію, а оплата має бути здійснена протягом 24 годин після формування замовлення. Покупець може обрати зручні дату та спосіб доставки, додати коментар до замовлення за потреби. Для постійних клієнтів передбачена система лояльності зі знижками, а також спеціальні промокоди під час акцій та свят, інформація про які доступна у персональному акаунті.

Основний функціонал:

Головна сторінка:

При завантаженні веб-застосунку користувач може авторизуватись або зареєструватись, після завантаження користувач бачить головну сторінку (рис. 1), на якій він може, обрати певну категорію техніки та подивитись її на окремій сторінці, по-свайпати різні фото та акції на початку, перейти на сторінку інформації, відкрити каталог у навігаційному меню, перейти у кошик, а також побачити інформацію про магазин: контактні дані, розташування та послуги:

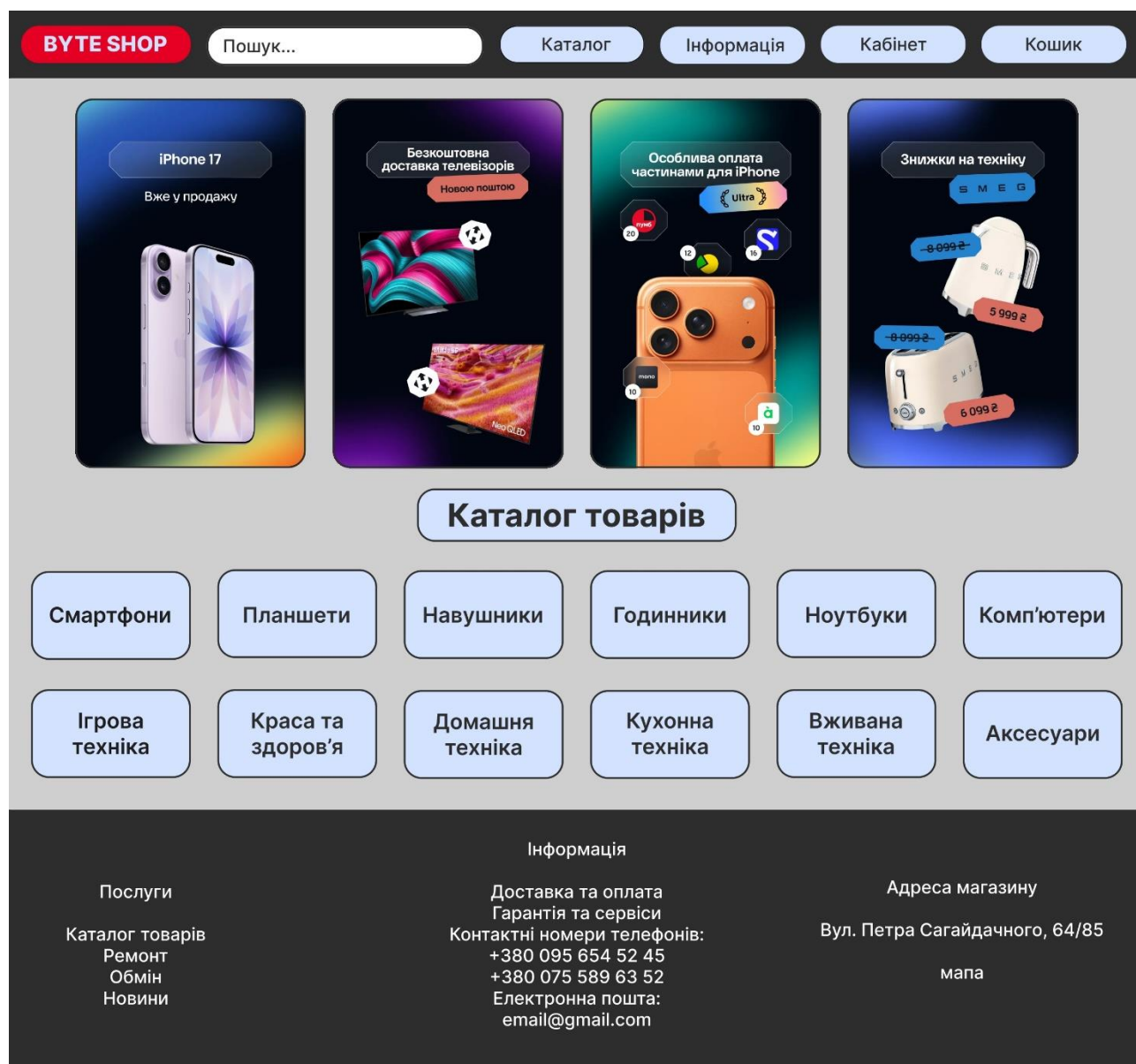


Рисунок 1 – Головна сторінка «Byte Shop»

Головна сторінка складається з трьох семантичних розділів:

1. «Header» – містить поле для пошуку, навігаційне меню категорій, пошуку, а також поле кабінет користувача/авторизація/реєстрація.
2. «Main» - містить умовно дві секції: перша секція це промо-блок (обто різні фото знижки або акції)щоб користувач бачив, що є у магазині; друга секція – фотографії виробів для захоплення користувача прикрасами та бажанням придбати якусь для себе; і третя секція це історія, ця секція розкаже про магазин та майстриню, та розповідає як виготовляються
3. «Footer» - останній розділ на головній сторінці, який має контактні дані, інформацію доставки та іншу інформацію.

Сторінка «Категорії»:

При натисканні користувача на будь-яку з категорій з каталога, йому відкривається інша сторінка «Категорії» (рис. 2). На ній користувач може переглядати техніку конкретної категорії, яку він обрав, а також виконувати пошук по певним критеріям, які встановить у лівому блоці: бренд, функціонал та різні характеристики.



Рис. 2 – Сторінка категорій «Byte Shop»

На цій сторінці є назва категорії зверху сторінки, під херedom, а також два умовних блоки:

1. «Пошук» - це лівий блок, де користувач буде виконувати пошук по тим критеріям, які йому захочуться або потрібні.
2. «Техніка» - це правий блок, тобто просто техніка, де не натискаючи на них користувач бачить їх назву та ціну.

Сторінка «Техніка»:

При натисканні користувача на будь-якої техніки, йому відкривається інша сторінка «Техніка» (рис. 3). На ній користувач може переглядати сам виріб конкретної категорії, який він обрав, а також його характеристик.



Рис. 3 – Сторінка виробу «Byte Shop»

На цій сторінці є назва виробу зверху сторінки, під херedom, його фото, ціна, спосіб доставки, а також опис виробу (його характеристики).

Сторінка «Інформація»:

При натисканні у хедері кнопки «Інформація»(рис. 4), йому відкривається сторінка з інформацією про доставку, оплату, гарантії та сервіси магазину.

Загальна інформація

Вул. Володимира Великого, 6/8

Умови оплати

Інформація про істотні характеристики продукту та попередження розміщені на сайті продукту: chast.monobank.ua

Гарантія та сервіси

Гарантія на спеціальні категорії товарів надається згідно умов виробника та може бути розширена магазином.

mapa

Рис. 4 – Сторінка реєстрації «Byte Shop»

Кабінет користувача:

Після реєстрації чи авторизації, у користувача створюється свій кабінет, де є його особиста інформація, улюблені позиції, кошик та архів його замовлення.

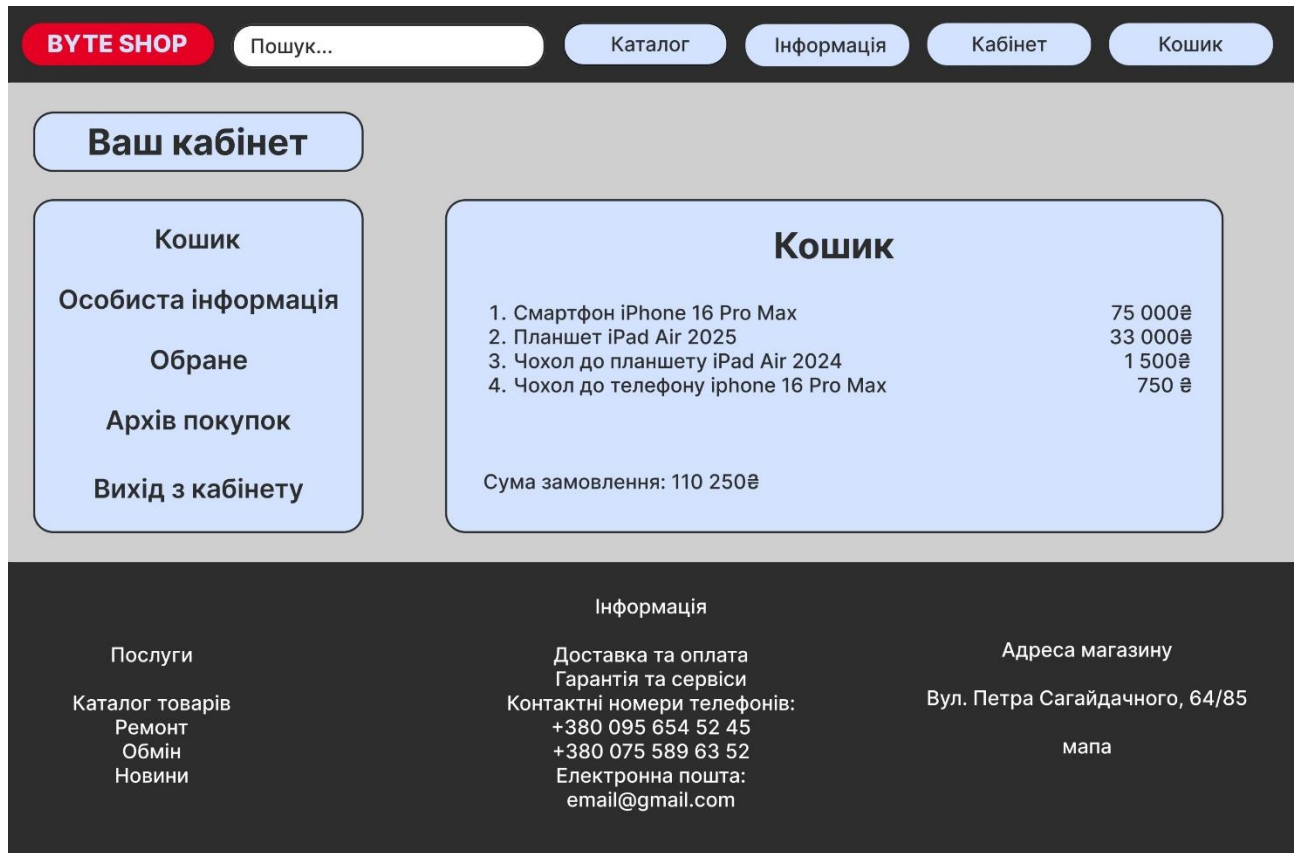


Рис. 6 – Кабінет користувача «Byte Shop»

На цій сторінці є назва сторінки «Ваш кабінет», а також умовні два блоки:

1. «Особисті дані» - це лівий блок, це прописані особисті дані користувача (ПІБ, номер телефону та електронна адреса) та його фото за бажанням.
2. «Замовлення» - це правий блок з назвою «Замовлення» та вся інформація про замовлення, які вже були зроблені.

Функціональні вимоги:

1. Гість може вільно відвідувати сайт
2. Гість може переглядати всю техніку певних категорій
3. Гість може переглядати детальну інформацію про техніку
4. Користувач може створити свій кабінет або увійти в нього
5. Наявний промо-блок з фотографіями техніки, щоб користувач міг їх переглядати

6. Наявна окрема сторінка інформацією про магазин (контактна інформація, місце розташування та соціальні мережі)
7. Змога авторизованих користувачів робити замовлення
8. Редагування кошику перед фінальним етапом замовлення
9. Вибір платіжної системи та доставки користувачем
10. Підтвердження замовлення користувачем
11. Змога адміністратора редагувати асортимент та кількість продуктів

Нефункціональні вимоги:

1. Не менше 1000 користувачів можуть користуватись сайтом одночасно
2. Інтерфейс простий у використанні та інтуїтивно зрозумілий користувачу
3. Система забезпечує захищеність даних, дані клієнтів конфіденційні і перебувають у безпеці, всі дані і паролі захешовані
4. Система працює як на комп'ютерах, так і на телефоні

Use-case

Use-case діаграма:

1. Адміністратор: має змогу редагувати товари, їх асортимент, кількість та наявність, додавати івенти/акції, а також керувати профілями різних користувачі
2. Користувач: має змогу переглядати сайт та техніку, виконати пошук їх за різними критеріями (ціна, колір, розмір), керувати кошиком.
3. Зареєстрований користувач: має змогу створити особистий кабінет та робити замовлення, залишати відгуки.

Сама діаграма проілюстрована на рисунку 1 Додатку А

ER-діаграма

ER-діаграма: на моїй моделі детально показано зв'язок сутностей між собою.

Розроблена ER-діаграми зображена на рисунку 2 Додатку Б.

Висновки:

У ході виконання роботи було розроблено концепцію онлайн-магазину цифрової та побутової техніки, який відповідає сучасним стандартам електронної комерції та враховує потреби як кінцевих користувачів, так і адміністратора системи. Основною метою стало створення веб-застосунку, що забезпечує високу зручність взаємодії покупців з продуктом, а також автоматизацію процесів управління асортиментом, замовленнями та клієнтськими даними.

Реалізовані функціональні можливості дозволяють користувачам переглядати каталог техніки, ознайомлюватися з детальними характеристиками товарів, створювати особистий кабінет, формувати замовлення та редагувати їх у процесі оформлення. Система підтримує авторизацію, роботу кошика, порівняння характеристик товарів, фільтри за параметрами, використання промокодів і програму лояльності для постійних покупців. Адміністратор отримує можливість гнучко керувати асортиментом, контролювати наявність товару на складі, обробляти замовлення та відстежувати їх статуси.

У процесі розробки було визначено як функціональні, так і нефункціональні вимоги: підтримка великої кількості одночасних користувачів, висока швидкість завантаження сторінок, зручний та інтуїтивний інтерфейс, адаптивність під різні пристрої, а також захист персональних даних. Побудовані Use-case та ER-діаграми відображають логіку взаємодії користувачів із системою та структуру бази даних, що забезпечує чітку архітектуру проєкту.

Таким чином, розроблений проєкт онлайн-магазину техніки є актуальним, технологічно прогресивним і має високу практичну цінність. Він може слугувати основою для подальшого масштабування та впровадження нових інструментів електронної комерції. Використання такої системи сприятиме підвищенню ефективності продажів, оптимізації бізнес-процесів та покращенню якості обслуговування клієнтів у сфері роздрібної торгівлі технікою як на внутрішньому, так і на зовнішньому ринках.

Контрольні питання

1. Що означає актуальність теми та чому важливо її обґрунтовувати при розробці інформаційної системи?

Актуальність теми показує, чому саме ця система потрібна зараз: які проблеми вона вирішує, чому важлива для користувачів чи бізнесу. Обґрунтування актуальності допомагає довести, що розробка має практичне значення і відповідає сучасним потребам.

2. Як правильно сформулювати мету роботи та чим вона відрізняється від завдань?

Мета роботи – це загальний кінцевий результат, якого прагне розробник (наприклад, створення зручної та ефективної системи).

Завдання – це конкретні кроки, які потрібно виконати для досягнення мети (аналіз предметної області, проєктування бази даних, реалізація інтерфейсу тощо).

3. У чому полягає різниця між об'єктом та предметом роботи?

- Об'єкт – ширше поняття, сфера або процес, у межах якого проводиться дослідження (наприклад, діяльність інтернет-магазину).
- Предмет – конкретна частина об'єкта, на якій зосереджується увага (наприклад, процес управління замовленнями у цьому магазині).

4. Що таке бізнес-логіка інформаційної системи та яку роль вона відіграє?

Бізнес-логіка – це набір правил і алгоритмів, що визначають, як система виконує основні функції відповідно до потреб бізнесу. Вона забезпечує правильність обробки даних, послідовність дій і автоматизацію процесів.

5. Які приклади функціональних вимог можна навести для типової інформаційної системи?

- Реєстрація та авторизація користувачів.
- Формування та обробка замовлень.
- Пошук та фільтрація товарів.
- Формування звітів.
- Керування каталогом даних.

6. Що таке нефункціональні вимоги і які їхні основні категорії?

Нефункціональні вимоги описують не «що робить система», а «як вона працює».

Основні категорії:

- Продуктивність (швидкість роботи).
- Масштабованість.
- Надійність і відмовостійкість.
- Безпека.
- Зручність інтерфейсу (юзабіліті).

7. Для чого використовується Use-case діаграма і що вона відображає?

Use-case діаграма використовується для опису функціональних вимог до системи.

Вона відображає взаємодію користувачів (акторів) із системою та сценарії використання (випадки застосування).

8. Хто такі актори у Use-case діаграмі та які функції вони виконують?

Актори – це користувачі або зовнішні системи, що взаємодіють із програмою. Вони ініціюють сценарії використання, тобто виконують дії (наприклад, «Клієнт» оформлює замовлення, «Адміністратор» додає новий товар).

9. Що таке ER-діаграма та яку інформацію вона дозволяє відобразити?

ER-діаграма (Entity-Relationship) – це схема, яка показує сутності предметної області та зв'язки між ними. Вона відображає логічну структуру бази даних.

10. Які основні елементи містить ER-діаграма та як вони між собою пов'язані?

- Сутності (наприклад, «Клієнт», «Замовлення», «Товар»).
- Атрибути (наприклад, ім'я клієнта, дата замовлення, ціна товару).
- Зв'язки (наприклад, «Клієнт оформлює Замовлення»).

Зв'язки визначають, як сутності взаємодіють (один-до-одного, один-до-багатьох, багато-до-багатьох).

ДОДАТОК А

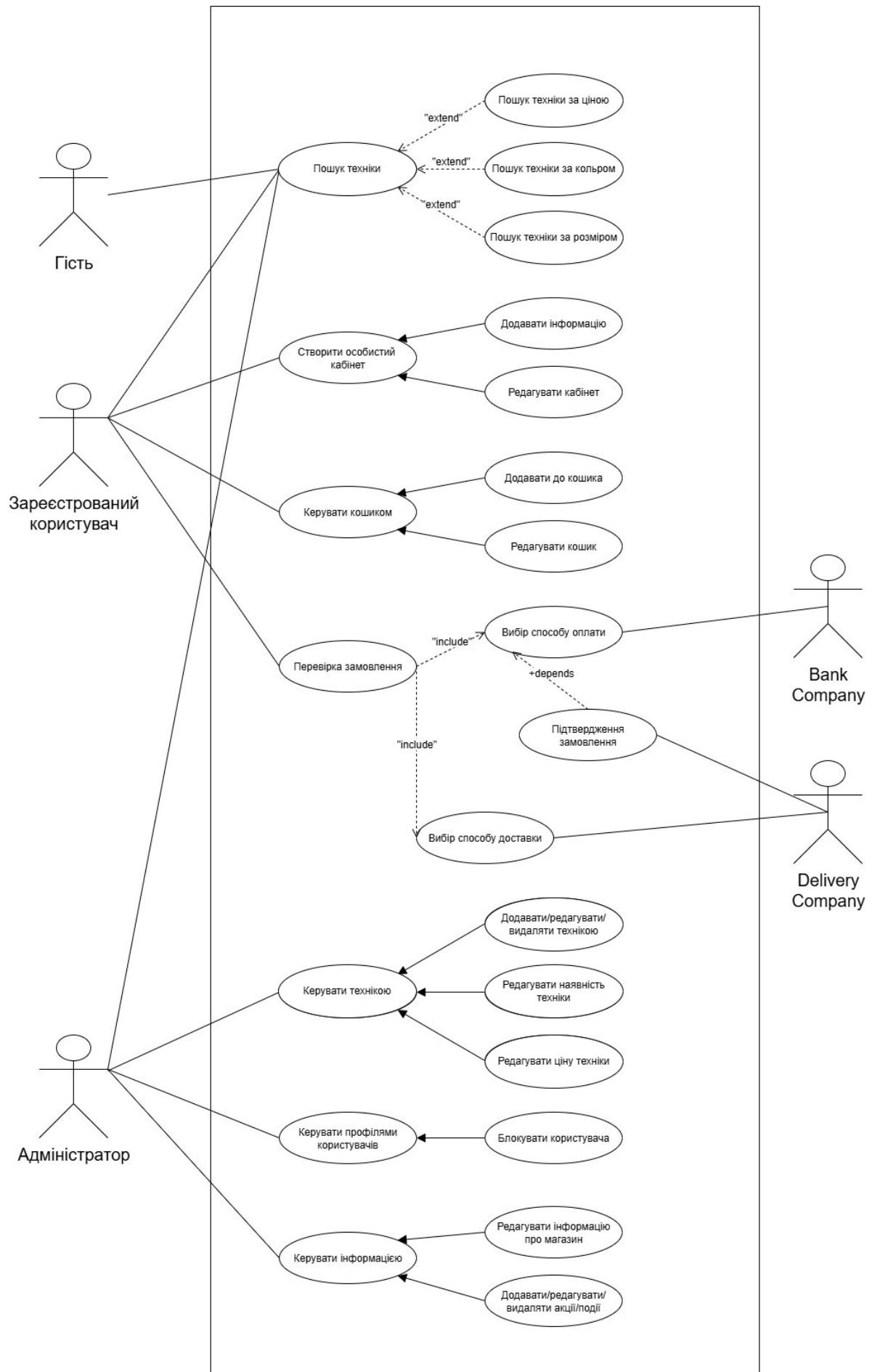


Рисунок 1. Use-case діаграма

ДОДАТОК Б

ER-діаграма

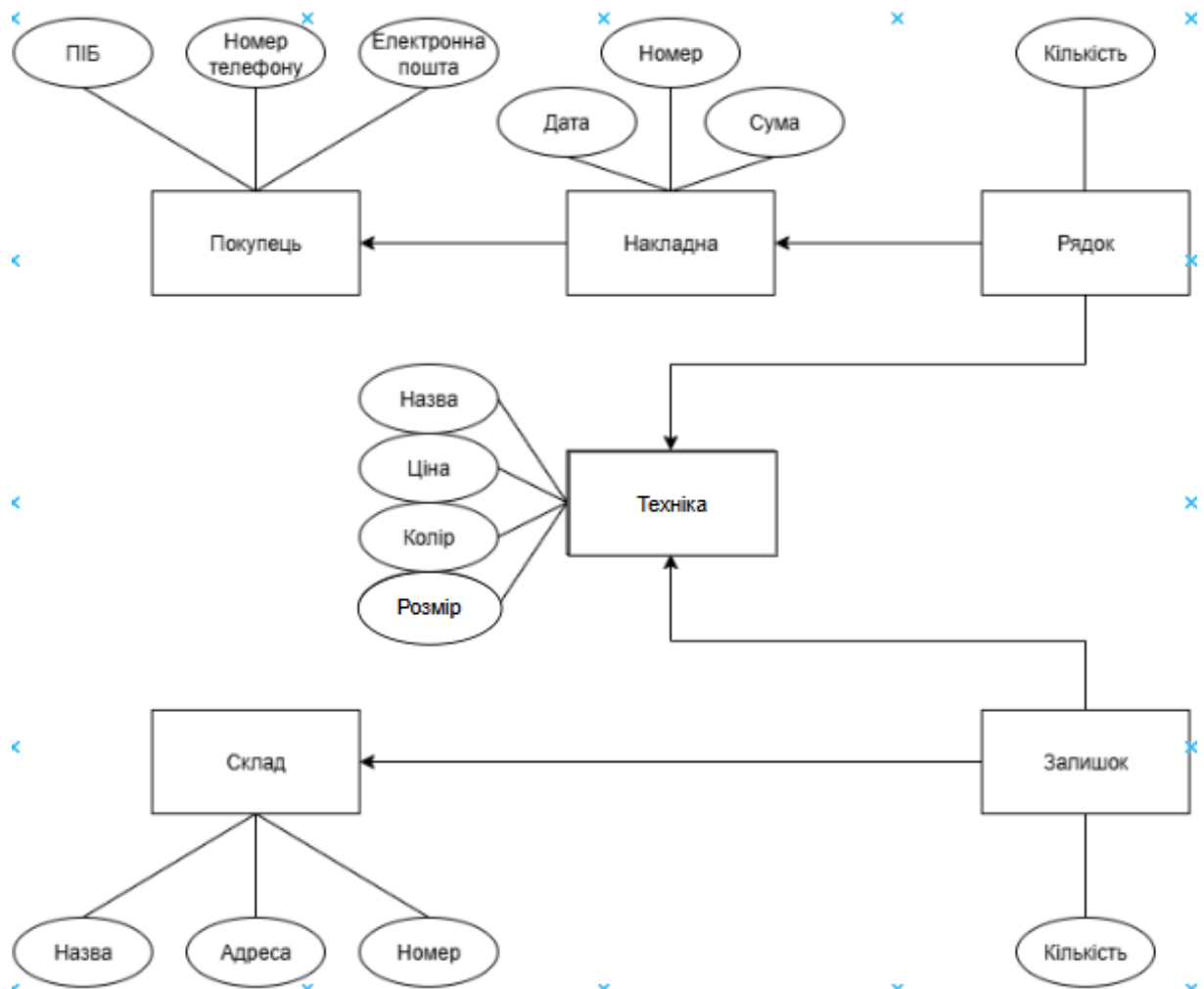


Рисунок 2 Діаграма зв'язку сутностей

2 РОЗРОБЛЕННЯ АДАПТИВНОГО ІНТЕРФЕЙСУ WEB-ЗАСТОСУНКУ (ПРАКТИЧНА РОБОТА 2)

2.1 Мета роботи

Мета роботи полягає у набутті навичок розроблення адаптивного інтерфейсу веб-застосунку з використанням сучасних засобів верстки, для забезпечення коректного відображення на різних пристроях.

2.2 Реалізація структурних компонентів інтерфейсу

2.2.1 Реалізація компоненту header

У застосунку “BYTE SHOP” було реалізовано компонент header. В цьому розділі розміщено логотип та адаптивне бургер-меню. Бургер меню має вигляд стрічки з розділів веб-застосунку у правій частині екрану для пристроїв з розширенням екрану більше ніж 900px. Вигляд зображено на рисунку 2.1. Для решти вигляд представлено на рисунку 2.2.



Рисунок 2.1 - Вигляд компоненту header для пристроїв з розширенням екрану більше ніж 900px



Рисунок 2.2 - Вигляд компоненту header для пристроїв з розширенням екрану менше ніж 900px

HTML-код, що відповідає за компонент header:

```
<!-- Header -->
<header>
  <div class="title">
    <a href="index.html" style="color:#D3E3FF; font-size: x-large; text-decoration:
none;">BYTE SHOP</a>
  </div>
  <div style="display: flex; gap: 10px; float:right;">
    <div class="search-container">
      <input type="text" id="searchInput" placeholder="Пошук...">
```

```

</div>

<button class="searchButton">Search</button>
<button class="catalog-btn">Каталог</button>
<button class="info-btn">Інформація</button>
<button class="RL-item" id="RL"><a href="pages/login.html">Рєєстрація</a></button>

</div>
</header>

```

CSS-код, що відповідає за компонент header:

```

header {
  width: 99%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: #FFFFFFF;
  background: #2E2E2E;
  padding: 10px;
}

.title {
  background: #E60023;
  border-radius: 20px;
  height: 100%;
  padding-left: 10px;
  padding-right: 10px;
  padding-bottom: 5px;
  padding-top: 5px;
}

.search-container {
  width: 100%;
}

.search-container input {
  width: 80%;
  height: 25px;
  border-radius: 20px;
}

```

```
background: #D3E3FF;
color: #000000;
display: flex;
align-items: center;
font-size: 16px;
padding: 10px;
gap: 10px;
}

.search-container input::placeholder {
  font-size: 110%;
  font-weight: bold;
  color: #2E2E2E;
  font-style: italic;
  opacity: 1;
}

.searchButton {
  Width: auto;
  height: 50px;
  border-radius: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: #D3E3FF;
  transition: background 0.3s;
  font-size: 110%;
  font-weight: bold;
  color: #2E2E2E;
  text-decoration: none;
}

.searchButton:hover {
  background-color: #E60023;
  color: #D3E3FF;
}
```

```
.search-results {  
  margin-top: 10px;  
  display: flex;  
  flex-wrap: wrap;  
  gap: 10px;  
}  
  
.search-result-item {  
  width: 270px;  
  background-color: #D3E3FF;  
  border-radius: 8px;  
  padding: 15px;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
  text-align: center;  
}  
  
.catalog-btn {  
  Width: 60%;  
  height: 50px;  
  border-radius: 20px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: #D3E3FF;  
  transition: background 0.3s;  
  font-size: 110%;  
  font-weight: bold;  
  color: #2E2E2E;  
  text-decoration: none;  
}  
  
.catalog-btn :hover {  
  background-color: #E60023;  
  color: #D3E3FF;  
}  
  
.info-btn {
```

```
Width: 60%;
height: 50px;
border-radius: 20px;
display: flex;
align-items: center;
justify-content: center;
background: #D3E3FF;
transition: background 0.3s;
font-size: 110%;
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
}

.info-btn :hover {
  background-color: #E60023;
  color: #D3E3FF;
}

@media (max-width: 900px) {
  .catalog-btn,
  .info-btn {
    display: none;
  }
}

.RL-item {
  Width: 60%;
  height: 50px;
  border-radius: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: #D3E3FF;
  transition: background 0.3s;
  font-size: 110%;
```

```
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
```

```
}
```

```
.RL-item:hover {
```

```
background-color: #E60023;
color: #D3E3FF;
```

```
}
```

2.2.2 Реалізація компоненту main

У компоненті main розташовані всі основні розділи веб-застосунку, описані у розділі 1. Присутні такі елементи як заголовки різних рівнів, звичайний текст, посилання на інші сторінки, зображення. Для зображень було застосовано адаптивну сітку засобами flexbox. Вигляд компоненту main на різних розмірах екрану представлено на рисунках 2.3 та 2.4 (анімація роботи фільтрів описана у підрозділі 2.3.5).

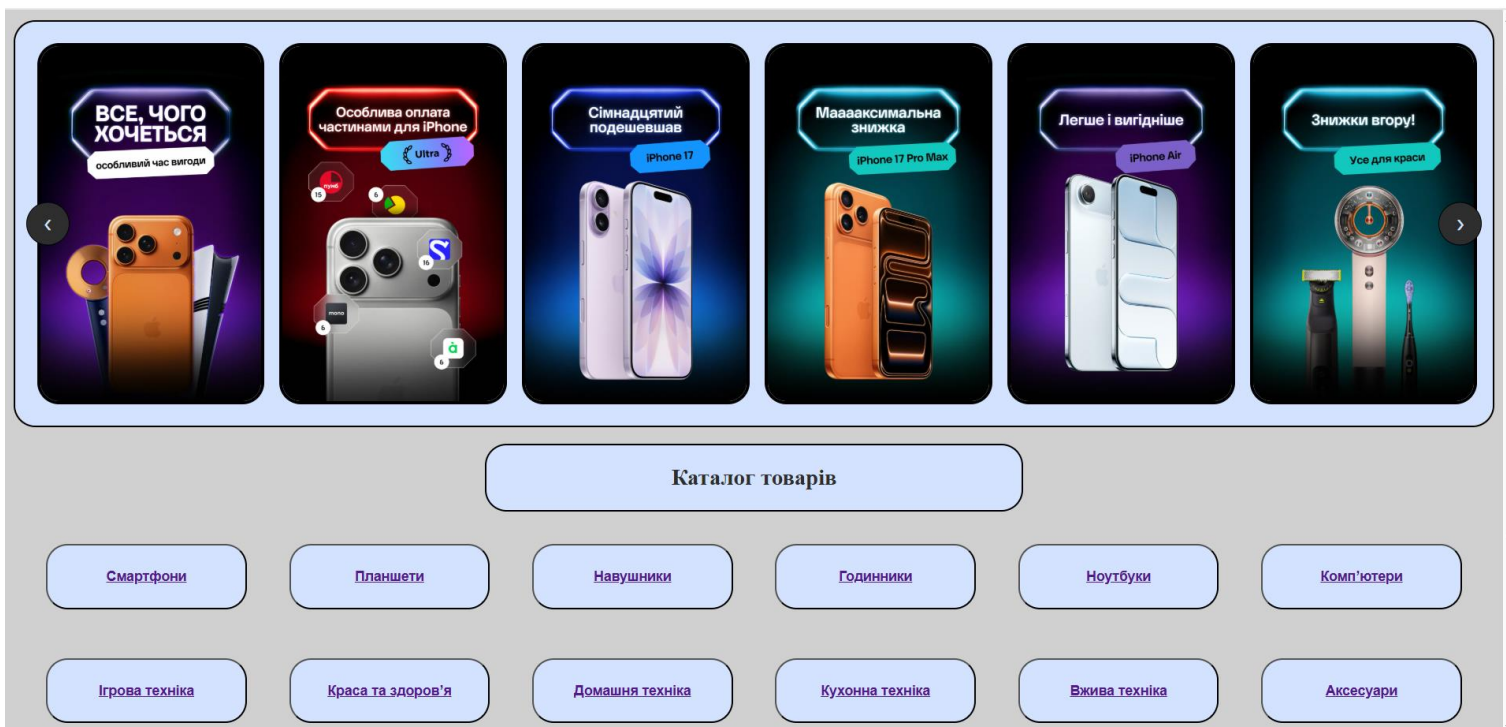


Рисунок 2.3- вигляд компоненту main на великих екранах



Рисунок 2.4 - вигляд компоненту main на малих екранах

HTML-код для компоненту main:

```
<!-- Photos -->
```

```
<div class="photos">
```

```
<div class="carousel-container">
```

```
<div class="carousel-track-container">
```

```
<div class="carousel-track">
```

```
<div class="carousel-item"></div>
```

```
<div class="carousel-item"></div>
```

```
<div class="carousel-item"></div>
```

```
<div class="carousel-item"></div>
```

```
<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>

<div class="carousel-item"></div>
```

```

    <div class="carousel-item"></div>

    <div class="carousel-item"></div>

    <div class="carousel-item"></div>

    <div class="carousel-item"></div>

    </div>
</div>

<button class="prev-btn"><</button>

<button class="next-btn">>>/button>

</div>
</div>

<!-- Types -->
<div class="types-container">
    <div class="title-TC">
        <h2 class="ttc" id="List">Каталог товарів</h2>
    </div>
    <div class="types-block">
        <button class="types-item"><a href="pages/phone.html">Смартфони</a></button>
        <button class="types-item"><a href="pages/pad.html">Планшети</a></button>
        <button class="types-item"><a href="pages/pods.html">Навушники</a></button>
        <button class="types-item"><a href="pages/watch.html">Годинники</a></button>
        <button class="types-item"><a href="pages/laptop.html">Ноутбуки</a></button>
        <button class="types-item"><a href="pages/computer.html">Комп'ютери</a></button>
    </div>
    <div class="types-block">
        <button class="types-item"><a href="pages/game.html">Ігрова техніка</a></button>
        <button class="types-item"><a href="pages/prettyHealth.html">Краса та
здоров'я</a></button>
        <button class="types-item"><a href="pages/home.html">Домашня
техніка</a></button>

```

```

<button class="types-item"><a href="pages/kitchen.html">Кухонна
техніка</a></button>
<button class="types-item"><a href="pages/BU.html">Вжива техніка</a></button>
<button class="types-item"><a href="pages/add.html">Аксесуари</a></button>
</div>
</div>

```

CSS -код, що відповідає за компонент main:

```

body {
  background-color: #D0D0D0;
}
.search-results {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  margin-top: 20px;
}
.product-item {
  background-color: #D3E3FF;
  padding: 15px;
  border-radius: 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  width: 250px;
  box-sizing: border-box;
}
.product-item h3 {
  margin-top: 0;
  font-size: 18px;
}
.product-item p {
  margin: 5px 0;
  font-size: 14px;
}
.types-container {

```

```
display: flex;
justify-content: center;
flex-direction: column;
gap: 15px;
padding: 15px;
}

.title-TC {
  Width: 37%;
  height: 60px;
  border-radius: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: #D3E3FF;
  border: 2px solid black;
  margin: 0 auto;
}

.ttc {
  font-weight: bolder;
  font-size: 130%;
  color: #2E2E2E;
  text-decoration: none;
  align-items: center;
  justify-content: center;
}

.types-block {
  display: flex;
  flex-direction: row;
  justify-content: center;
  gap: 3%;
  padding: 15px;
}

.types-item {
```

```
Width: 17%;
height: 60px;
border-radius: 20px;
display: flex;
align-items: center;
justify-content: center;
background: #D3E3FF;
transition: background 0.3s;
font-size: 80%;
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
}

.types-item:hover {
  background-color: #E60023;
  color: #D3E3FF;
}

.carousel-container {
  position: relative;
  margin: auto;
  border-radius: 20px;
  border: 2px solid black;
}

.carousel-track-container {
  overflow: hidden;
  border-radius: 20px;
  background: #D3E3FF;
  padding: 10px;
}

.carousel-track {
  display: flex;
  transition: transform 0.5s ease-in-out;
}
```

```
.carousel-item {  
  flex: 0 0 calc(100% / 6);  
  box-sizing: border-box;  
  padding: 10px;  
}  
  
.carousel-item img {  
  width: 100%;  
  height: auto;  
  display: block;  
  border-radius: 20px;  
  border: 3px solid black;  
}  
  
.prev-btn, .next-btn {  
  position: absolute;  
  top: 50%;  
  transform: translateY(-50%);  
  width: 40px;  
  height: 40px;  
  border-radius: 50%;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: #2E2E2E;  
  transition: background 0.3s;  
  cursor: pointer;  
  border: 1px solid black;  
  color: #D3E3FF;  
}  
  
.prev-btn:hover, .next-btn:hover {  
  background: #E60023;  
  color: #2E2E2E;  
}  
  
.prev-btn { left: 10px; }
```

```
.next-btn { right: 10px; }
```

2.2.3 Реалізація сторінок типів товарів

На сторінці категорії смартфонів представлено блок із назвою розділу, а також інтерактивні елементи, що дозволяють користувачу здійснювати фільтрацію товарів. У верхній частині сторінки розміщено заголовок другого рівня, який відображає назву поточної категорії — «Смартфони». Цей заголовок візуально відокремлений у власному контейнері та слугує орієнтиром для відвідувача.

Нижче представлено розгортуваний список фільтрів, оформлений як випадаюча панель. Кнопка «Фільтри» відкриває доступ до набору параметрів, за якими користувач може впорядкувати або обмежити виведення товарів. Панель містить кілька незалежних блоків фільтрації, кожен із яких оформлено за допомогою елементів `<fieldset>` та `<legend>`.

Перший блок дає можливість відсортувати смартфони за ціною — від дешевших до дорожчих або навпаки. Наступні поля включають фільтрацію за брендом (Apple, Google, Samsung, Asus), кольором корпусу, обсягом пам'яті та наявністю. Кожна група параметрів містить перелік чекбоксів, що дозволяють користувачу обрати один або кілька варіантів. Окрема кнопка «Показати» у кожній секції передбачена для застосування вибраного фільтра (анімація роботи фільтрів описана у підрозділі 2.3.5).

Основний контент сторінки складається з поля товарів, організованого у вигляді гнучкої сітки. Кожен товар розташований в окремому блоці з класом `product`. У цьому блоці міститься зображення моделі смартфона, назва товару та його актуальна ціна. Зображення обгорнуте посиланням, що веде на сторінку детального перегляду товару, де користувач може побачити повні характеристики моделі.

У поточному прикладі всі товари є ідентичними зразками iPhone 13, однак у реальному асортименті ці блоки наповнюються динамічними даними відповідно до вибраної категорії та застосованих фільтрів.

Структура контейнера `all-products` забезпечує адаптивне та впорядковане відображення елементів у декілька колонок, що дозволяє зручно переглядати каталог на різних розмірах екрана. Таким чином, ця частина коду відповідає за представлення

асортименту смартфонів із можливістю швидкого пошуку, сортування та переходу до сторінки окремого товару. Візуальний приклад відображення елементів наведено на рисунках 2.5 та 2.6.



Рисунок 2.5 - вигляд сторінки типу «Смартфони» на великих екранах



Рисунок 2.6 - вигляд сторінки типу «Смартфони» на малих екранах

HTML-код для компоненту сторінки типу «Смартфони»

```

<div class="titlePage">
  <h2>Смартфони</h2>
</div>

<!--Search field-->

<div class="dropdown">
  <button class="buttonF">Фільтри</button>
  <div class="search-field">
    <div class="price-field">
      <form>
        <fieldset>
          <legend>Ціна</legend>
          <button class="button1" type="button" name="search" value="increase-prise">Від
меншої до більшої</button>
          <button class="button1" type="button" name="search" value="reduced-price">Від
більшої до меншої</button>
        </fieldset>
      </form>
    </div>
  <div class="field">
    <form>
      <fieldset>
        <legend class="name-list">Бренд</legend>
        <div class="list">
          <input type="checkbox" name="brend" value="apple">Apple<br>
          <input type="checkbox" name="brend" value="google">Google<br>
          <input type="checkbox" name="brend" value="samsung">Samsung<br>
          <input type="checkbox" name="brend" value="asus">Asus<br>
        </div>
        <button class="button" type="button">Показати</button>
      </fieldset>
    </form>
  </div>
  <div class="field">

```

```
<form>
```

```
  <fieldset>
```

```
    <legend class="name-list">Колір</legend>
```

```
    <div class="list">
```

```
      <input type="checkbox" name="color" value="yellow">жовтий<br>
```

```
      <input type="checkbox" name="color" value="green">зелений<br>
```

```
      <input type="checkbox" name="color" value="blue">синій<br>
```

```
      <input type="checkbox" name="color" value="red">Червоний<br>
```

```
    </div>
```

```
    <button class="button" type="button">Показати</button>
```

```
  </fieldset>
```

```
</form>
```

```
</div>
```

```
<div class="field">
```

```
  <form>
```

```
    <fieldset>
```

```
      <legend class="name-list">Пам'ять</legend>
```

```
      <div class="list">
```

```
        <input type="checkbox" name="memory" value="128GB">128GB<br>
```

```
        <input type="checkbox" name="memory" value="256GB">256GB<br>
```

```
        <input type="checkbox" name="memory" value="512GB">512GB<br>
```

```
        <input type="checkbox" name="memory" value="1TB">1TB<br>
```

```
      </div>
```

```
      <button class="button" type="button">Показати</button>
```

```
    </fieldset>
```

```
  </form>
```

```
</div>
```

```
<div class="field">
```

```
  <form>
```

```
    <fieldset>
```

```
      <legend class="name-list">Наявність</legend>
```

```
      <div class="list">
```

```
        <input type="checkbox" name="exist" value="positive">В наявності<br>
```

```

        <input type="checkbox" name="exist" value="negative">На замовлення<br>
    </div>
    <button class="button" type="button">Показати</button>
</fieldset>
</form>
</div>
</div>
</div>
<!--Field of product-->
<div class="all-products">
    <div class="product">
        <a href="phoneP.html"></a>
        <h2 class="pname">iPhone 13</h2>
        <h3 class="pprice">21 350 ₪</h3>
    </div>
    <div class="product">
        <a href="phoneP.html"></a>
        <h2 class="pname">iPhone 13</h2>
        <h3 class="pprice">21 350 ₪</h3>
    </div>
    <div class="product">
        <a href="phoneP.html"></a>
        <h2 class="pname">iPhone 13</h2>
        <h3 class="pprice">21 350 ₪</h3>
    </div>
    <div class="product">
        <a href="phoneP.html"></a>
        <h2 class="pname">iPhone 13</h2>
        <h3 class="pprice">21 350 ₪</h3>
    </div>
</div>

```

CSS-код для компоненту сторінки типу «Смартфони»

```
body {background-color: #D0D0D0;}
```

```
.search-results {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
  margin-top: 20px;  
}  
  
.product-item {  
  background-color: #D3E3FF;  
  padding: 15px;  
  border-radius: 20px;  
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);  
  width: 250px;  
  box-sizing: border-box;  
}  
  
.product-item h3 {  
  margin-top: 0;  
  font-size: 18px;  
}  
  
.product-item p {  
  margin: 5px 0;  
  font-size: 14px;  
}  
  
.titlePage {  
  Width: 37%;  
  height: 60px;  
  border-radius: 20px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: #D3E3FF;  
  border: 2px solid black;  
  margin: 0 auto;  
}
```

```
.dropdown {  
  position: relative;  
  display: inline-block;  
}  
  
.buttonF {  
  width: 100%;  
  height: 40px;  
  font-size: 90%;  
  font-weight: bold;  
  color: #2E2E2E;  
  background: #D3E3FF;  
  border-radius: 20px;  
  margin: 5px;  
}  
  
.buttonF:hover {  
  background: #E60023;  
  color: #2E2E2E;  
}  
  
.search-field {  
  display: none;  
  position: absolute;  
  background: #D3E3FF;  
  border: 2px solid black;  
  min-width: 160px;  
  box-shadow: 0px 8px 16px 0px rgba(0, 0, 0, 0.2);  
  padding: 12px 16px;  
  z-index: 1;  
}  
  
.dropdown:hover .search-field {  
  display: block;  
}  
  
.title-search-field {  
  height: 100%;
```

```
display: flex;
align-items: center;
justify-content: center;
font-size: 110%;
margin: 0.1%;
color: #2E2E2E;
font-weight: bolder;
}

.button1 {
width: 90%;
height: 100%;
font-size: 60%;
font-weight: bold;
color: #D3E3FF;
background: #2E2E2E;
border-radius: 20px;
margin: 0.05%;
}

.button1:hover {
background: #E60023;
color: #2E2E2E;
}

.button {
width: 90%;
height: 100%;
font-size: 90%;
font-weight: bold;
color: #D3E3FF;
background: #2E2E2E;
border-radius: 20px;
margin: 5px;
}

.button:hover {
```

```
background: #E60023;
color: #2E2E2E;
}
.price-field {
margin: 0.02%;
padding: 0.05%;
font-size: 100%;
color: #2E2E2E;
}
.name-list {
margin: 0.02%;
padding: 0.05%;
font-size: 100%;
color: #2E2E2E;
}
.list {
font-size: 90%;
padding: 0.05%;
width: 90%;
}
.all-products {
display: flex;
flex-direction: row;
justify-content: center;
align-items: center;
}
.product {
width: 23%;
border-radius: 20px;
background-color: #D3E3FF;
padding: 5px;
margin: 10px;
display: flex;
```

```

flex-direction: column;
justify-content: center;
align-items: center;
}

.product img {
width: 90%;
border-radius: 20px;
margin: 10px;
}

```

2.2.4 Реалізація сторінки товару

На сторінці категорії представлено повноцінний розділ із товарами типу «Смартфони». У верхній частині документа розташовано заголовок із логотипом магазину BYTE SHOP та інтерактивною панеллю навігації. До неї входять поле пошуку, кнопка каталогу, інформаційний розділ та кнопка реєстрації. Весь цей блок оформлено у вигляді шапки сайту (header) і забезпечує швидкий доступ до основних функцій ресурсу.

Нижче розміщено заголовок сторінки категорії — елемент h2, що відображає назву обраного типу продукції («Смартфони»). Він винесений у спеціальний контейнер titlePage, що допомагає відокремити його від інших елементів та підкреслити структуру сторінки.

Далі подано інтерактивну панель фільтрів, оформлену у вигляді випадаючого меню. Вона відкривається натиском кнопки «Фільтри» та містить кілька окремих блоків для сортування та відбору товарів за критеріями. Кожен набір параметрів оформлений у вигляді fieldset, що надає фільтрам логічної структури:

- У першому полі наведено сортування за ціною — від меншої до більшої та навпаки.
- У наступних блоках передбачено вибір бренду, кольору, обсягу пам'яті та наявності відповідних моделей у магазині.
- Для кожного параметра використовуються чекбокси, що дають змогу комбінувати кілька умов одночасно.
- Кнопки «Показати» призначені для застосування вибраних фільтрів.

Таке компонування забезпечує зручність пошуку та дозволяє користувачеві швидко налаштовувати відображення товарів відповідно до власних потреб.

Основний зміст сторінки представлений у вигляді поля товарів — контейнера all-products. Тут товари відображені у вигляді окремих карток, організованих у сітку. Кожен продукт містить:

- зображення смартфона, яке також є посиланням на сторінку детального перегляду товару;
- назву моделі, оформлену тегом h2;
- ціну, подану елементом h3.

Повторювані елементи структуруються за допомогою класу product, що забезпечує рівне розташування та однаковий стиль для всіх карток.

У нижній частині сторінки розміщено футер із трьома інформаційними секціями. Тут наведено перелік послуг магазину, корисні посилання, контактні номери телефонів, електронна пошта та адреса торгової точки з активним посиланням на карту Google. Така структура дозволяє користувачеві швидко знайти інформацію щодо доставки, оплати, гарантії чи контактів.

Таким чином, наведений код формує повноцінну сторінку категорії товарів, яка поєднує в собі заголовок, панель фільтрів, інтерактивне поле товарів та інформативний футер. Сторінка забезпечує зручну навігацію, можливість швидкого сортування та перегляду смартфонів, а також слугує основою для відображення каталогу. Вигляд цієї сторінки подано на рисунках 2.7 та 2.8.



Рисунок 2.7 - вигляд сторінки виробу типу «Смартфони» на великих екранах

iPhone 13



Ціна: 21 350 €

Чорний

Білий

Сірий

Нюдовий

128GB

256GB

512GB

1TB

basic

basic+

business

premium

Купити

Опис товару

Здається, що може бути кращим за флагманські дванадцятки, які оснащені передовими технологіями у сфері мобільної розробки. Та все ж творцям з Купертіно кожного разу вдається здивувати користувачів новинками з особливо стильним дизайном та інноваційною начинкою.

У гарячий Apple iPhone 13 захоуєшся з першого погляду. Елегантні риси корпусу, нові кольори, дивовижний дисплей, камери, що можуть позмагатись з кращою фототехнікою, потужні процесори та швидкісна операційна система. Це вже давно не просто смартфони, а витвори високотехнологічного мистецтва.

Пропонуємо познайомитись з новинкою ближче та розглянути основні переваги та параметри девайсу:

1. Зменшені рамки.
2. Нове розміщення блоку основних камер.
3. Підвищена автономність акумулятора.
4. Нове покоління процесора A15 Bionic.
5. 5 нових кольорів корпусу.
6. Унікальна система стабілізації.
7. Портретний режим зйомки.
8. Обсяг пам'яті від 128GB.

Рисунок 2.8 - вигляд сторінки виробу типу «Смартфони» на малих екранах

HTML-код для компоненту сторінки виріб типу «Смартфони»

```

<div class="titlePage">
  <h2>Marshall Major V</h2>
</div>
<div class="infoProduct">
  <div class="img">
    
  </div>
  <div class="infoPP">
    <div class="price">
      <button>Ціна: 4 500 ₪</button>
    </div>
    <div class="color">
      <button>Чорний</button>
      <button>Білий</button>
      <button>Сірий</button>
      <button>Нюдовий</button>
    </div>
    <div class="color">
      <button>128GB</button>
      <button>256GB</button>
      <button>512GB</button>
      <button>1TB</button>
    </div>
    <div class="color">
      <button>basic</button>
      <button>basic+</button>
      <button>business</button>
      <button>premium</button>
    </div>
    <div class="price">
      <button>Купити</button>
    </div>
  </div>
</div>

```

</div>

</div>

<div class="titleDescription">

<h2>Опис товару</h2>

</div>

<div class="description">

<p>

Легендарна компанія Marshall є відомим виробником аудіообладнання, від гітарних підсилювачів до барабанних установок і

сценічних колонок — продукцію з культовим “М” можна зустріти на сценах по всій Земній кулі. Навіть їхні навушники та

Bluetooth-динаміки мають особливий попит серед шанувальників якісного звуку та музикантів.

</p>

<p>

Major V стали останнім на сьогодні оновлення лінійки накладних бездротових навушників бренду, які поєднують в собі всі

сильні характеристики Marshall та привертають увагу потужним звуком, фірмовим дизайном і безпрецедентним часом автономної

роботи: до 100 годин на одному заряді.

</p>

<p>

Чітка та динамічна звукова сцена Marshall, доповнена потужними 40-мм драйверами, ідеально підходить для прискіпливих

слухачів, яким потрібна деталізація та глибина кожної ноти. З Major V будь-який музичний жанр розкриється для вас

по-новому, створюючи крутий ефект занурення, немов ви перебуваєте в першому ряду на концерті улюбленого виконавця.

</p>

</div>

CSS-код для компоненту сторінки виріб типу «Смартфони»

```
body {background-color: #D0D0D0;}
```

```
.search-results {
```

```
display: flex;
flex-wrap: wrap;
gap: 20px;
margin-top: 20px;
}

.product-item {
background-color: #D3E3FF;
padding: 15px;
border-radius: 20px;
box-shadow: 0 2px 5px rgba(0,0,0,0.1);
width: 250px;
box-sizing: border-box;
}

.product-item h3 {
margin-top: 0;
font-size: 18px;
}

.product-item p {
margin: 5px 0;
font-size: 14px;
}

.titlePage {
Width: 37%;
height: 60px;
border-radius: 20px;
display: flex;
align-items: center;
justify-content: center;
background: #D3E3FF;
color: #2E2E2E;
border: 2px solid black;
margin: 0 auto;
}
```

```
.infoProduct {  
  display: flex;  
  align-items: flex-start;  
  justify-content: center;  
  gap: 20px;  
  flex-wrap: wrap;  
  width: 100%;  
  margin-top: 2%;  
}  
  
.item {  
  background: #D3E3FF;  
  padding: 20px;  
  border-radius: 10px;  
  width: 50%;  
}  
  
@media (max-width: 700px) {  
  .item {  
    width: 100%;  
  }  
}  
  
.img {  
  width: 45%;  
  background-color: #D3E3FF;  
  border: 2px solid black;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 20px;  
  padding: 10px;  
}  
  
.img img {  
  width: 95%;  
  border-radius: 20px;
```

```
margin: 10px;
display: flex;
align-items: center;
justify-content: center;
}
.infoPP {width: 50%;}
.price button {
width: 90%;
height: 50px;
font-size: 60%;
font-weight: bold;
font-size: larger;
color: #2E2E2E;
background: #D3E3FF;
border-radius: 20px;
margin: 3%;
display: flex;
align-items: center;
justify-content: center;
}
.price button:hover {
background: #E60023;
color: #2E2E2E;
}
.color {
display: flex;
flex-direction: row;
width: 100%;
display: flex;
align-items: center;
justify-content: center;
}
.color button {
```

```
width: 250px;
height: 40px;
font-size: 60%;
font-weight: bold;
color: #2E2E2E;
background: #D3E3FF;
font-size: medium;
border-radius: 20px;
padding: 5px;
margin: 2%;
display: flex;
align-items: center;
justify-content: center;
}

.color button:hover {
  background: #E60023;
  color: #2E2E2E;
}

.titleDescription {
  Width: 37%;
  height: 60px;
  border-radius: 20px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: #D3E3FF;
  color: #2E2E2E;
  border: 2px solid black;
  margin: 0 auto;
  margin-top: 2%;
}

.description {
  width: 95%;
```

```

background-color: #D3E3FF;
color: #2E2E2E;
padding: 10px;
margin: 2%;
border-radius: 20px;
border: 2px solid black;
font-size: 110%;
margin-top: 2%;
}
.description p {
text-indent: 20px;
margin-bottom: 5px;
line-height: 1.4;
}

```

2.2.5 Реалізація компоненту кабінет користувача

Розділ «Кабінет користувача» містить кілька взаємопов'язаних сторінок, кожна з яких призначена для роботи з персональними даними, замовленнями та кошиком користувача. Усі сторінки мають спільну структуру: фірмовий заголовок BYTE SHOP у верхній частині, панель навігаційних кнопок та бічне меню з основними пунктами кабінету. Це забезпечує впізнаваність інтерфейсу та однакову логіку переходів.

У лівій частині кожної сторінки розташовано навігаційний блок, який містить три ключові посилання: «Кошик», «Особиста інформація» та «Архів замовлень». Нижче розміщено кнопку «Вийти», що дозволяє завершити сеанс роботи. Така структура бокової панелі забезпечує швидкий доступ до основних функцій кабінету незалежно від того, на якій сторінці перебуває користувач.

1. Головна сторінка кабінету

Головна сторінка кабінету містить заголовок першого рівня із привітальним повідомленням «*Good to see you, dear User!*». Ця сторінка виконує роль вступної та не містить додаткових даних, але сприяє орієнтації користувача у структурі розділу. Основний контент розміщено в центральному блоці *main*, який візуально відокремлений від бокового меню.

2. Сторінка «Кошик»

Сторінка «Кошик» призначена для відображення товарів, доданих користувачем

для покупки. У центральній частині міститься заголовок «Кошик», під яким розташована таблиця з трьома стовпцями: ID товару, його назва та ціна. Кожен товар оформлений у вигляді окремого рядка таблиці. Така структура дозволяє легко переглянути перелік вибраних позицій, їхню кількість та вартість. Надалі ця таблиця може бути доповнена кнопками редагування або видалення товарів, проте базова версія вже підтримує основну логіку перегляду.

3. Сторінка «Архів замовлень»

Сторінка архіву містить повну історію оформлених замовлень. У таблиці наведено чотири стовпці: ID запису, номер замовлення, перелік придбаних товарів та загальна сума. Кожне замовлення подається в окремому рядку, а перелік товарів структурований у вигляді декількох параграфів усередині клітинки. Такий формат дозволяє швидко оцінити склад замовлення, не переходячи на додаткові сторінки. Архів дає змогу користувачу відстежувати свої минулі покупки й підтримує зручну навігацію по історії замовлень.

4. Сторінка «Особиста інформація»

На сторінці «Особиста інформація» відображаються основні персональні дані користувача. Таблиця містить комірки з полями: прізвище, ім'я, дата народження, номер телефону та електронна адреса. Дані структуровані у форматі «назва поля — значення», що полегшує перегляд та сприйняття інформації. Така форма подання дозволяє користувачеві швидко перевірити коректність своїх даних і, за потреби, змінити їх у майбутніх версіях сайту.

Підсумок

Усі сторінки кабінету мають єдиний стиль і структуру, що формує зрозумілу систему взаємодії користувача з персональними даними та замовленнями. Єдина навігаційна панель, повторювана бокова таблиця посилань та логічно оформлені заголовки створюють цілісний інтерфейс, у якому кожен розділ виконує свою чітку функцію: перегляд основної інформації, контроль кошика, управління замовленнями та ознайомлення з персональними даними.



Рисунок 2.9.1 - вигляд компоненту кабінет користувача на великих екранах

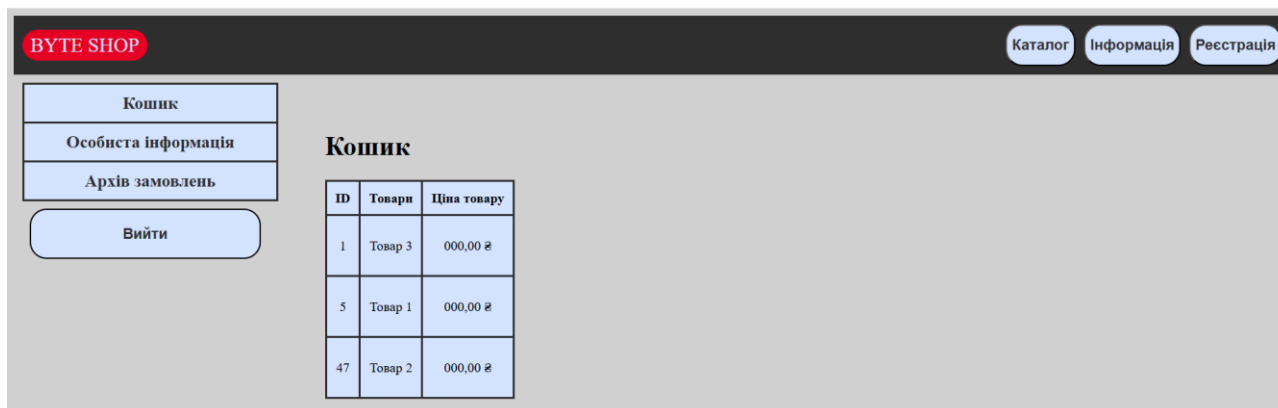


Рисунок 2.9.2 - вигляд компоненту кошику користувача на великих екранах

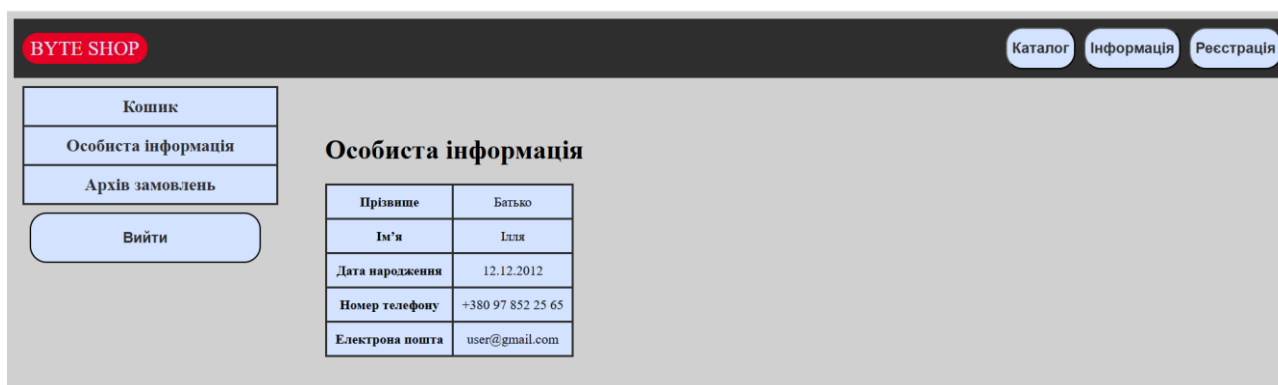


Рисунок 2.9.1 - вигляд компоненту особистої інформації користувача на великих екранах

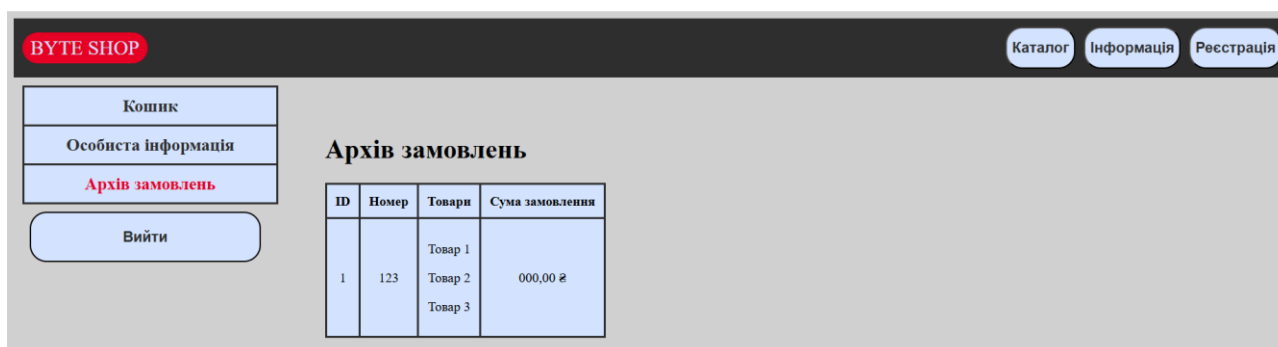


Рисунок 2.9.1 - вигляд компоненту архіву замовлень користувача на великих екранах



Рисунок 2.10.1 - вигляд компоненту кабінет користувача на малих екранах



Рисунок 2.10.2 - вигляд компоненту кошику користувача на малих екранах



Рисунок 2.10.3 - вигляд компоненту особистої інформації користувача на малих екранах



Рисунок 2.10.4 - вигляд компоненту архіву замовлень користувача на малих екранах

HTML-код для компоненту кабінет користувача:

```
<div class="general">
  <div class="table">
    <table>
      <tr>
        <th class="types-item"><a href="cabinetBuy.html">Кошик</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="cabinetPI.html">Особиста інформація</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="cabinetList.html">Архів замовлень</a></th>
      </tr>
    </table>
    <button class="button" type="button">Вийти</button>
  </div>
<div class="main">
  <h1 class="titl">
    Good to see you, dear User! :)
  </h1>
</div>
</div>
```

CSS-код для компоненту кабінет користувача:

```
body {background-color: #D0D0D0;}
.general {
  display: flex;
  flex-direction: row;
  width: 100%;
}
.table {
  width: 25%;
  display: flex;
  flex-direction: column;
```

```
padding: 10px;
}
table, th, td {
background-color: #D3E3FF;
border: 3px solid #2E2E2E;
border-collapse: collapse;
padding: 10px;
text-align: center;
vertical-align: middle;
}
.button {
background: #D3E3FF;
margin: 3%;
border-radius: 20px;
font-size: 110%;
font-weight: bold;
width: 90%;
height: 60px;
padding-left: 20px;
padding-top: 1px;
padding-bottom: 1px;
padding-right: 20px;
display: flex;
align-items: center;
justify-content: center;
border: 2px solid black;
color: #2E2E2E;
}
.button:hover {
background-color: #E60023;
color: #D3E3FF;
}
.types-item a {
```

```

font-size: 130%;
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
}
.types-item a:hover {
    color: #E60023;
    text-decoration: none;
}
.main {
    margin: 3%;
}

```

HTML-код для компоненту кошику користувача:

```

<div class="general">
    <div class="table">
        <table>
            <tr>
                <th class="types-item"><a href="cabinetBuy.html">Кошик</a></th>
            </tr>
            <tr>
                <th class="types-item"><a href="cabinetPI.html">Особиста інформація</a></th>
            </tr>
            <tr>
                <th class="types-item"><a href="cabinetList.html">Архів замовлень</a></th>
            </tr>
        </table>
        <button class="button" type="button">Вийти</button>
    </div>
    <div class="main">
        <h1 class="titleTopic">Кошик</h1>
        <table class="product" id="productTable">
            <tr>
                <th>ID</th>

```

```

    <th>Товари</th>
    <th>Ціна товару</th>

```

```

</tr>

```

```

<tr>
    <td>1</td>
    <td>
        <p>Товар 3</p>
    </td>
    <td>000,00 ₴</td>

```

```

</tr>

```

```

    <tr>
    <td>5</td>
    <td>
        <p>Товар 1</p>
    </td>
    <td>000,00 ₴</td>

```

```

</tr>

```

```

    <tr>
    <td>47</td>
    <td>
        <p>Товар 2</p>
    </td>
    <td>000,00 ₴</td>

```

```

</tr>

```

```

</table>

```

```

</div>

```

HTML-код для компоненту особистої інформації користувача:

```

<div class="general">
    <div class="table">
        <table>
            <tr>
                <th class="types-item"><a href="cabinetBuy.html">Кошик</a></th>
            </tr>

```

```

<tr>
  <th class="types-item"><a href="cabinetPI.html">Особиста інформація</a></th>
</tr>
<tr>
  <th class="types-item"><a href="cabinetList.html">Архів замовлень</a></th>
</tr>
</table>
<button class="button" type="button">Вийти</button>
</div>
<div class="main">
  <h1 class="titleTopic">Архів замовлень</h1>
  <table class="product" id="productTable">
    <tr>
      <th>ID</th>
      <th>Номер</th>
      <th>Товари</th>
      <th>Сума замовлення</th>
    </tr>
    <tr>
      <td>1</td>
      <td>123</td>
      <td>
        <p>Товар 1</p>
        <p>Товар 2</p>
        <p>Товар 3</p>
      </td>
      <td>000,00 ₴</td>
    </tr>
  </table>
</div>

```

HTML-код для компоненту архіву замовлень користувача:

```

<div class="general">
  <div class="table">

```

```

<table>
  <tr>
    <th class="types-item"><a href="cabinetBuy.html">Кошик</a></th>
  </tr>
  <tr>
    <th class="types-item"><a href="cabinetPI.html">Особиста інформація</a></th>
  </tr>
  <tr>
    <th class="types-item"><a href="cabinetList.html">Архів замовлень</a></th>
  </tr>
</table>
<button class="button" type="button">Вийти</button>
</div>
<div class="main">
  <h1 class="titleTopic">Особиста інформація</h1>
  <table class="product" id="productTable">
    <tr>
      <th>Прізвище</th>
      <td>Батько</td>
    </tr>
    <tr>
      <th>Ім'я</th>
      <td>Ілля</td>
    </tr>
    <tr>
      <th>Дата народження</th>
      <td>12.12.2012</td>
    </tr>
    <tr>
      <th>Номер телефону</th>
      <td>+380 97 852 25 65</td>
    </tr>
    <tr>

```

```

        <th>Електрона пошта</th>
        <td>user@gmail.com</td>
    </tr>
</table>
</div>

```

CSS-код для компоненту для сторінок користувача:

```
body {background-color: #D0D0D0;}
```

```
.general {
```

```
    display: flex;
```

```
    flex-direction: row;
```

```
    width: 100%;
```

```
}
```

```
.table {
```

```
    width: 25%;
```

```
    display: flex;
```

```
    flex-direction: column;
```

```
    padding: 10px;
```

```
}
```

```
table, th, td {
```

```
    background-color: #D3E3FF;
```

```
    border: 3px solid #2E2E2E;
```

```
    border-collapse: collapse;
```

```
    padding: 10px;
```

```
    text-align: center;
```

```
    vertical-align: middle;
```

```
}
```

```
.button {
```

```
    background: #D3E3FF;
```

```
    margin: 3%;
```

```
    border-radius: 20px;
```

```
    font-size: 110%;
```

```
    font-weight: bold;
```

```
    width: 90%;
```

```
height: 60px;
padding-left: 20px;
padding-top: 1px;
padding-bottom: 1px;
padding-right: 20px;
display: flex;
align-items: center;
justify-content: center;
border: 2px solid black;
color: #2E2E2E;
}
.button:hover {
  background-color: #E60023;
  color: #D3E3FF;
}
.types-item a {
  font-size: 130%;
  font-weight: bold;
  color: #2E2E2E;
  text-decoration: none;
}
.types-item a:hover {
  color: #E60023;
  text-decoration: none;
}
.main {
  margin: 3%;
  width: 90%;
}
```

2.2.6 Реалізація сторінки реєстрації

Сторінка «Реєстрації» призначена для створення нового облікового запису користувача у веб-застосунку. Весь вміст розміщено в основному контейнері `register`, який містить форму введення особистих даних.

У середині форми (`form`) послідовно розташовані текстові поля для заповнення основної інформації: прізвище, ім'я, номер телефону, електронна адреса, а також пароль. Кожне поле супроводжується зрозумілою текстовою підказкою для користувача, що полегшує процес заповнення форми.

Під основними полями розміщено текстове посилання «Вже маєте кабінет?», яке веде на сторінку авторизації (`login.html`) і дає можливість перейти до входу без повторного заповнення даних.

Завершує форму кнопка «Зареєструватись», що ініціює процес створення нового облікового запису.

Загалом дана частина коду формує інтуїтивно зрозумілий інтерфейс для введення персональної інформації з метою реєстрації користувача на сайті, забезпечуючи логічну послідовність елементів і зручність у використанні.

Прізвище:

Ім'я:

Номер телефону:

Електронна адреса:

Пароль:

[Вже маєте кабінет?](#)

Зареєструватись

Рисунок 2.11 - вигляд сторінки реєстрації на великих екранах

BYTE SHOP

Реєстрація

Прізвище:

Ім'я:

Номер телефону:

Електронна адреса:

Пароль:

[Вже маєте кабінет?](#)

Зареєструватись

Рисунок 2.12 - вигляд сторінки реєстрації на малих екранах

HTML-код для сторінки реєстрації:

```
<div class="register">
  <form class="form" action="">
    Прізвище:<br>
    <input type="text" name="firstname" value=""><br>
    Ім'я:<br>
    <input type="text" name="lastname" value=""><br>
    Номер телефону:<br>
    <input type="text" name="phone" value=""><br>
    Електронна адреса:<br>
```

```
<input type="text" name="email" value=""><br>
```

```
Пароль:<br>
```

```
<input type="text" name="password" value=""><br>
```

```
<a href="login.html">Вже маєте кабінет?</a>
```

```
<button class="button" type="button">Зареєструватись</button>
```

```
</form>
```

```
</div>
```

CSS-код для сторінки реєстрації:

```
.register {
```

```
padding: 15px;
```

```
width: 50%;
```

```
display: flex;
```

```
margin: auto;
```

```
height: 100%;
```

```
font-size: larger;
```

```
border-radius: 20%;
```

```
font-size: 130%;
```

```
font-weight: bold;
```

```
color: #2E2E2E;
```

```
justify-content: center;
```

```
}
```

```
.form {
```

```
background-color: #D3E3FF;
```

```
color: #2E2E2E;
```

```
border: 3px solid #2E2E2E;
```

```
border-radius: 20px;
```

```
width: 500px;
```

```
max-width: 90%;
```

```
padding: 30px;
```

```
z-index: 1000;
```

```
display: flex;
```

```
flex-direction: column;
```

```
gap: 15px;
```

```
}  
.form input[type="text"],  
.form textarea {  
    width: 90%;  
    padding: 8px 10px;  
    border-radius: 10px;  
    border: 1px solid #2E2E2E;  
    font-size: 16px;  
    margin-top: 5px;  
}  
.form label {  
    display: flex;  
    flex-direction: column;  
    font-weight: bold;  
    margin-bottom: 10px;  
}  
.form button {  
    width: 100%;  
    padding: 10px;  
    font-weight: bold;  
    color: #D3E3FF;  
    background-color: #2E2E2E;  
    border: none;  
    border-radius: 10px;  
    cursor: pointer;  
    margin-top: 10px;  
    border: 2px solid black;  
}  
.form button:hover {  
    background-color: #E60023;  
    color: #D3E3FF;  
}  
.button {
```

```
width: 100%;  
height: 50px;  
font-size: 100%;  
font-weight: bold;  
color: #2E2E2E;  
background: #707077;  
border-radius: 20%;  
margin: auto;  
}
```

2.2.7 Реалізація сторінки авторизації

Сторінка «Авторизації» призначена для входу користувача до особистого кабінету на сайті. Уся структура сторінки зосереджена в контейнері `register`, який містить форму входу (`form`). Такий підхід дозволяє компактно розмістити всі елементи інтерфейсу та забезпечити зручність взаємодії.

У формі передбачено два основних поля введення: номер телефону або електронна пошта та пароль. Така реалізація дає можливість користувачу самостійно обрати спосіб входу, залежно від того, який контактний метод він використовував під час реєстрації. Кожне поле супроводжується коротким описом, що робить інтерфейс зрозумілим навіть для користувачів без технічного досвіду та полегшує навігацію.

Під полями введення розташовано посилання «Зареєструватися», яке перенаправляє на сторінку створення нового облікового запису (`pageRegister.html`). Воно призначене для нових користувачів, які ще не мають доступу до системи.

Нижче знаходиться кнопка «Увійти», яка виконує функцію підтвердження введених даних і запускає процес авторизації. Після натискання система перевіряє коректність введеної інформації та надає доступ до персонального кабінету у разі успішної автентифікації.

Таким чином, дана сторінка забезпечує просту, інтуїтивну та логічно структуровану форму входу в систему, що дозволяє користувачу швидко отримати доступ до персональних розділів веб-застосунку.

Рисунок 2.13 – вигляд сторінки авторизації на великих екранах

Рисунок 2.14 – вигляд сторінки авторизації на малих екранах

HTML-код для сторінки авторизації:

```
<div class="register">
  <form class="form" action="">
    Електронна пошта:<br>
    <input type="text" name="lastname" value=""><br>
    Пароль:<br>
```

```

<input type="text" name="lastname" value=""><br>
<a href="register.html">Зареєструватися</a>
<button class="button" type="button">Увійти</button>
</form>
</div>

```

CSS-код для сторінки авторизації:

```

.register {
    padding: 15px;
    width: 50%;
    display: flex;
    margin: auto;
    height: 100%;
    font-size: larger;
    border-radius: 20%;
    font-size: 130%;
    font-weight: bold;
    color: #2E2E2E;
    justify-content: center;
}

.form {
    background-color: #D3E3FF;
    color: #2E2E2E;
    border: 3px solid #2E2E2E;
    border-radius: 20px;
    width: 500px;
    max-width: 90%;
    padding: 30px;
    z-index: 1000;
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.form input[type="text"],

```

```
.form textarea {  
  width: 90%;  
  padding: 8px 10px;  
  border-radius: 10px;  
  border: 1px solid #2E2E2E;  
  font-size: 16px;  
  margin-top: 5px;  
}  
  
.form label {  
  display: flex;  
  flex-direction: column;  
  font-weight: bold;  
  margin-bottom: 10px;  
}  
  
.form button {  
  width: 100%;  
  padding: 10px;  
  font-weight: bold;  
  color: #D3E3FF;  
  background-color: #2E2E2E;  
  border: none;  
  border-radius: 10px;  
  cursor: pointer;  
  margin-top: 10px;  
  border: 2px solid black;  
}  
  
.form button:hover {  
  background-color: #E60023;  
  color: #D3E3FF;  
}  
  
.button {  
  width: 100%;  
  height: 50px;
```

```

font-size: 100%;
font-weight: bold;
color: #2E2E2E;
background: #707077;
border-radius: 20%;
margin: auto;
}

```

2.2.8 Реалізація сторінки інформації

Сторінка «Інформація» виконує роль довідкового розділу сайту BYTE SHOP, де користувач може швидко знайти основні контакти магазину, умови доставки, оплати, гарантії та послуги ремонту. Уся структура сторінки вибудована так, щоб забезпечити послідовну навігацію між різними тематичними блоками та не перевантажувати користувача надлишковим контентом.

У верхній частині розташовано знайому для всіх сторінок сайту шапку (header) з логотипом, пошуковим рядком та кнопками переходів у каталог, інформаційний розділ і форму реєстрації. Це забезпечує миттєвий доступ до ключових функцій незалежно від того, на якій саме сторінці знаходиться користувач.

Основна частина сторінки складається з кількох послідовно розміщених секцій. Перший блок — «Загальна інформація». У ньому представлені основні контактні дані магазину: номери телефонів, електронна пошта та адреси торгових точок з активними посиланнями на Google Maps. Такий підхід дозволяє одразу прокласти маршрут до магазину та уточнити графік роботи або доступність товарів.

Наступним великим розділом є блоки «Умови доставки» та «Умови оплати». Вони розташовані поруч у вигляді двох окремих підсекцій, що робить інформацію структурованою та легко порівнюваною. У частині про доставку детально описані варіанти отримання товару — від самовивозу до доставки службою «Нова Пошта». Користувач одразу бачить строки доставки, умови оплати за пересилання та наявність повної або часткової передоплати для окремих категорій товарів. У секції «Умови оплати» зосереджено пояснення програми «Покупка частинами» від monobank: перелік вимог, переваги та посилання на офіційну сторінку продукту.

Завдяки цьому користувач отримує повне уявлення про доступні фінансові опції без необхідності переходити в сторонні ресурси.

Далі подано розділ «Гарантія та сервіси». У ньому зібрана вся ключова інформація про гарантійні зобов'язання BYTE SHOP: строк дії стандартної гарантії, можливість розширення, умови для нової та вживаної техніки, а також перелік додаткових сервісних послуг. Опис подано у вигляді цілісного тексту, який легко читати навіть тим, хто вперше купує техніку Apple. Завдяки цьому користувач може зрозуміти, які саме види послуг пропонує магазин і які переваги може отримати після покупки.

Завершує сторінку великий блок «Ремонт». У ньому детально описано роботу сервісного центру, типові несправності та особливості ремонту різних категорій техніки — смартфонів, планшетів, ноутбуків, MacBook, iMac та інших пристроїв. Така структура дозволяє користувачеві швидко знайти потрібну інформацію щодо свого пристрою, а також зрозуміти процес діагностики, строки ремонту та рівень відповідальності сервісу.

Усі розділи сторінки розміщені в логічній послідовності: від загальної інформації — до деталей гарантії та післяпродажного обслуговування. Така організація контенту робить сторінку зручною, інформативною та доступною для будь-якого користувача, незалежно від його досвіду.

Сторінка «Інформація» виконує функцію централізованого довідкового розділу, де зібрано повний пакет інформації про магазин. Вона створена таким чином, щоб відвідувач міг знайти відповіді на більшість запитань без звернення в службу підтримки — від графіка роботи до умов оплати, від гарантії до ремонту. Завдяки чіткій структурі та продуманому оформленню сторінка забезпечує зручний доступ до важливої інформації і підвищує загальний рівень користувацького досвіду.

Інформація

Загальна інформація

Контактні номери телефонів:
+380 095 654 52 45
+380 075 589 63 52

Електронна пошта:
email@gmail.com

Адреси магазинів:

Вул. Петра Сагайдачного, 64/85

Вул. Ярослава Мудрого, 4/5

Вул. Володимира Великого, 6/8

Умови доставки

Видача з магазину:

- Доставка безкоштовна.
- Термін доставки становить від 1 до 3 днів.

Доставка Новою Поштою:

- Термін доставки становить від 1 до 3 днів.
- Доставка можлива після повної оплати так і накладенням платежем для нової техніки.
- Доставка у відділення Нової Пошти, після повної оплати товару — безкоштовна.
- Вартість доставки оплачується одержувачем згідно з умовами Нової Пошти.

* Певні категорії товарів доставляються після внесення авансового платежу.

Умови оплати

Покупка частинами:
monobank

Чому вигідно купити особливу техніку через Покупку Частинами від моно?

- Вигідні умови — без комісії під 0,000001%.
- Необхідна лише картка monobank.
- Миттєве оформлення в фізичному чи інтернет-магазині Ябло.

Вимоги для оформлення Покупки Частинами від monobank:

- Потрібно бути власником картки monobank.
- Мати достатній для покупки ліміт на Покупку частинами.

Інформація про істотні характеристики продукту та попередження розміщені на сайті продукту: chast.monobank.ua

Гарантія та сервіс

Загальна гарантія від ByteShop

Магазин Ябло надає гарантію на нову та вживану техніку Apple, а також додаткові розширені програми обслуговування.

Офіційна гарантія Apple

На всю нову техніку поширюється офіційна гарантія виробника.

Гарантія магазину

Після покупки діє гарантія магазину терміном 31 день, з можливістю розширення до 1 або 2 років.

Пакети розширеної гарантії можуть включати:

1. гарантійну заміну у разі заводського браку протягом 3 днів діагностики;
2. сервісне та гарантійне обслуговування;
3. повну технічну підтримку;
4. індивідуальне налаштування пристрою;
5. встановлення необхідного ПЗ (для Mac);
6. знижку 10% на ремонт у разі негарантійних випадків;
7. знижки до 20% на захисні аксесуари;
8. Cashback на обмін техніки.

Додаткові програми захисту можуть включати:

1. одноразову безкоштовну заміну дисплея при пошкодженні;
2. одноразову заміну або ремонт пристрою при будь-якому негарантійному випадку (крім втрати);
3. 100% компенсацію вартості пристрою у разі неможливості ремонту;
4. чистку від пилу та заміну термопаст (для Mac);
5. обмін пристрою протягом 3 днів у випадку заводського браку.

Вживана техніка

На б/у моделі також діє гарантія магазину 31 день з можливістю розширення до 1–2 років з аналогічними умовами.

Особлива техніка

Гарантія на спеціальні категорії товарів надається згідно умов виробника та може бути розширена магазином.

Ремонт

Ремонт особливої техніки у Ябло

Сервісний центр у мережі Ябло — це місце, де вирішуються будь-які негаразди з пристроями у найкоротші терміни та за вигідною вартістю. Розбили екран новенького смартфона? З'явилися несправності в улюбленого ноутбук або ж потрібно почистити ігрову консоль? Хочете налаштувати або ж оновити програмне забезпечення в планшета? Трапилось гарантійна поломка в Apple Watch? Поспішайте у Ябло і наша команда допоможе з ремонтом особливої техніки. У своїй роботі спеціалісти використовують лише перевірені інструменти, сучасні технології та якісні запчастини. На виконані ремонтні роботи мережа Ябло надає гарантію терміном від 1 місяця. Сучасна техніка від популярних брендів — це зразок досконалості та ідеальної комбінації дизайну та технологій. Та, на жаль, навіть довершений технічний пристрій може пошкодитися після тривалого використання. Пил, волога, високі або ж низькі температури — все це може негативно вплинути на функціональний стан. Також, варто пам'ятати, що гаджети потребують діагностики, яка допоможе завчасно виявити несправності та усунути їх. Якщо ви помітили зміни у роботі смартфона, ноутбук нагрівається або ж видає сторонні шуми, розумний годинник погано працює після відвідин басейну — рекомендуємо якнайшвидше звернутися за консультацією до представників мережі магазинів Ябло у вашому регіоні України.

Ремонт iPhone

Найчастіші несправності зі смартфонами від Apple пов'язані з дисплеєм, акумуляторною батареєю або ж динаміками. Здебільшого причини їх появи криються у тривалому користуванні, відсутності захисних аксесуарів або ж у використанні несертифікованих зарядних пристроїв.

Ябло — це сервісний центр в Україні, що пропонує послуги з ремонту усіх лінійок флагманських смартфонів всесвітньо відомого бренду з долини Купертіно. Фахівці мережі Jabko займаються сервісним обслуговуванням всіх моделей, починаючи від iPhone 8-го покоління до флагманських iPhone 17 Pro Max.

Ремонт смартфонів Samsung, Xiaomi та Google

Окрім пристроїв Apple, мережа Ябло надає професійний ремонт смартфонів Samsung, Xiaomi, Google та інших популярних виробників. Ми працюємо з усіма типовими та складними видами ремонту — від заміни дисплейних модулів та акумуляторів до відновлення материнських плат.

Для відновлення використовуються перевірені комплектуючі, що відповідають вимогам бренду. Наші фахівці також допоможуть із міграцією даних, оновленням та налаштуванням програмного забезпечення. Після ремонту смартфон працюватиме швидко, стабільно та без зайвих збоїв.

Ремонт планшетів iPad

Для значної частини користувачів, планшети вже давно замінили комп'ютер, адже основна їх перевага у портативності та високому рівню зручності. Та часто подорожі, транспортування у сумках чи рюкзаках, перепад між високими та низькими температурами може негативно позначитися на роботі планшета. Якщо у вашого iPad виникли несправності — варто звернутися у сервісний центр в Україні. Мережа магазинів та сервісів Ябло здійснює ремонт усіх актуальних моделей планшетів від Apple: iPad Mini, iPad Pro, iPad Air, iPad.

Майстри проводять детальну діагностику пристрою, після чого усувають виявлені проблеми у найкоротші терміни. Ми здійснюємо ремонт роз'ємів, динаміків, кнопок, мікрофонів, а також заміну дисплейного модуля та скла. За необхідності надаємо консультації щодо налаштувань, оновлення програмного забезпечення та будь-яких запитань, що можуть з'явитися у процесі користування iPad.

Рисунок 2.13 – вигляд сторінки інформації на великих екранах

Інформація

Загальна інформація

Контактні номери телефонів:
+380 095 654 52 45
+380 075 589 63 52

Електронна пошта:
email@gmail.com

Адреси магазинів:

Вул. Петра
Сагайдачного, 64/85

Вул. Ярослава
Мудрого, 4/5

Вул. Володимира
Великого, 6/8

Умови доставки

Видача з магазину:

— Доставка безкоштовна.
— Термін доставки становить від 1 до 3 днів.

Доставка Новою Поштою:

— Термін доставки становить від 1 до 3 днів.
— Доставка можлива після повної оплати так і накладеним платежем для нової техніки.
— Доставка у відділення Нової Пошти, після повної оплати товару — безкоштовна.
— Вартість доставки оплачується одержувачем згідно з умовами Нової Пошти.

* Певні категорії товарів доставляються після внесення авансового платежу.

Умови оплати

Покупка частинами:
monobank

Чому вигідно купити особливу техніку через Покупку Частинами від моно?

- Вигідні умови — без комісії під 0,000001%.
- **Необхідна лише картка monobank.**
- Миттєве оформлення в фізичному чи інтернет-магазині Ябло.

Вимоги для оформлення Покупки Частинами від monobank:

- Потрібно бути власником картки monobank.
- Мати достатній для покупки ліміт на Покупку частинами.

Інформація про істотні характеристики продукту та попередження розміщені на сайті
продукту: chast.monobank.ua

Гарантія та сервіс

Загальна гарантія від ByteShop

Магазин Ябло надає гарантію на нову та вживану техніку Apple, а також додаткові розширені програми обслуговування.

Офіційна гарантія Apple

На всю нову техніку поширюється офіційна гарантія виробника.

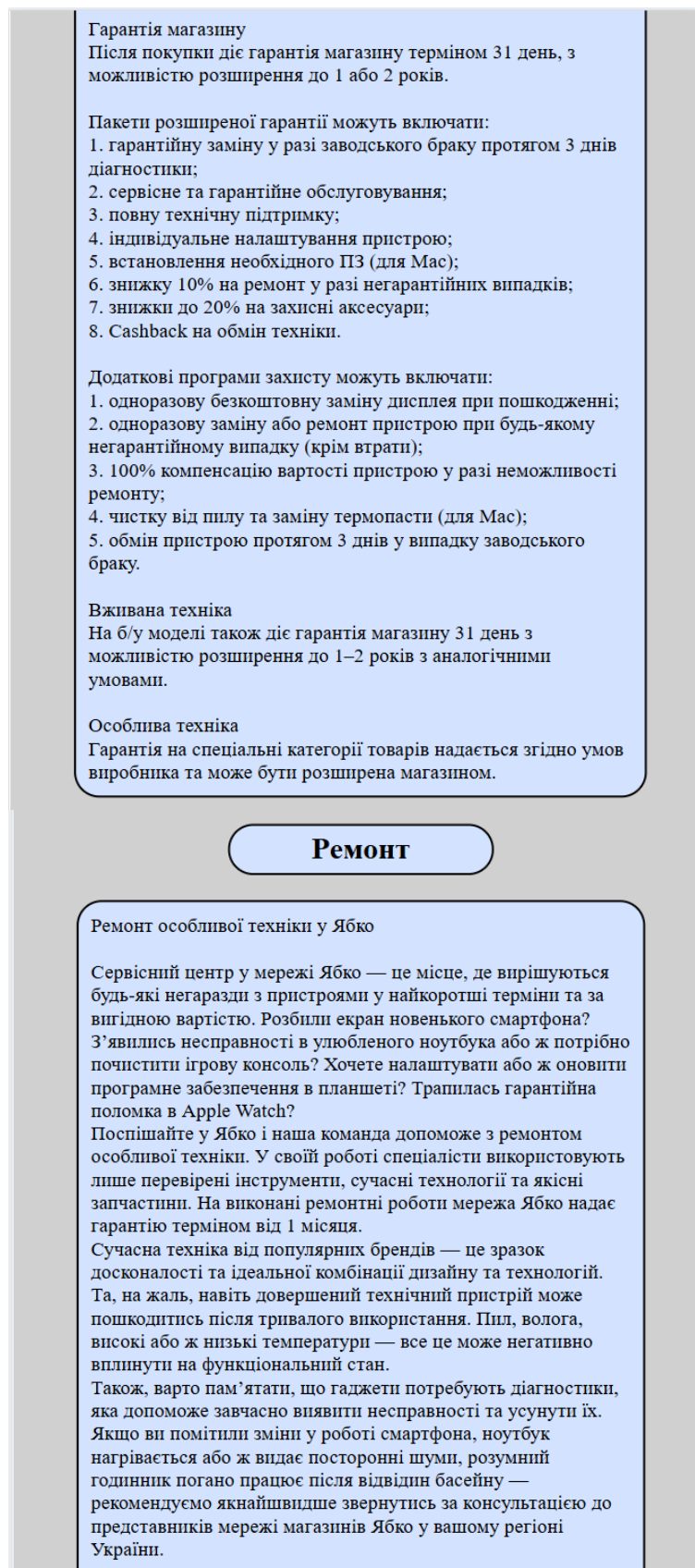


Рисунок 2.14 – вигляд сторінки інформації на малих екранах

HTML-код для сторінки авторизації:

```
<div class="general">
```

```
<div class="Title">
```

```
    Інформація
```

```
</div>
```

```
<div class="GI-title">
```

Загальна інформація

</div>

<div class="text">

<div>

Контактні номери телефонів:

+380 095 654 52 45

+380 075 589 63 52

Електронна пошта:

email@gmail.com

</div>

<div>

Адреси магазинів:

Вул. Петра Сагайдачного, 64/85

<a

href="https://www.google.com/maps/place/%D0%B2%D1%83%D0%BB%D0%B8%D1%86%D1%8F+%D0%AF%D1%80%D0%BE%D1%81%D0%BB%D0%B0%D0%B2%D0%B0+%D0%9C%D1%83%D0%B4%D1%80%D0%BE%D0%B3%D0%BE,+4,+%D0%91%D1%96%D0%BB%D0%B0+%D0%A6%D0%B5%D1%80%D0%BA%D0%B2%D0%B0,+%D0%9A%D0%B8%D1%97%D0%B2%D1%81%D1%8C%D0%BA%D0%B0+%D0%BE%D0%B1%D0%BB%D0%B0%D1%81%D1%82%D1%8C,+09100/@49.7933117,30.1138982,18.02z/data=!4m6!3m5!1s0x40d3421ef8141875:0x8c3d9bb91d52145c!8m2!3d49.7933946!4d30.1137475!16s%2F

g%2F11bw3z2w0v?entry=ttu&g_ep=EgoyMDI1MTIwMi4wIKXMDSOASAFQAw%3D%3D">

Вул. Ярослава Мудрого, 4/5

Вул. Володимира Великого,

6/8

</div>

</div>

<div class="DP" id="DP">

<div>

<div class="DP-title">

Умови доставки

</div>

<div class="text">

Видача з магазину:

— Доставка безкоштовна.

— Термін доставки становить від 1 до 3 днів.

Доставка Новою Поштою:

— Термін доставки становить від 1 до 3 днів.

— Доставка можлива після повної оплати так і накладеним платежем для нової техніки.

— Доставка у відділення Нової Пошти, після повної оплати товару — безкоштовна.

— Вартість доставки оплачується одержувачем згідно з умовами Нової Пошти.

* Певні категорії товарів доставляються після внесення авансового платежу.

</div>

</div>

<div>

<div class="DP-title">

Умови оплати

</div>

<div class="text">

Покупка частинами:

monobank

Чому вигідно купити особливу техніку через Покупку Частинами від mono?

- Вигідні умови — без комісії під 0,000001%.

- Необхідна лише картка monobank.

- Миттєве оформлення в фізичному чи інтернет-магазині Ябло.

Вимоги для оформлення Покупки Частинами від monobank:

- Потрібно бути власником картки monobank.

- Мати достатній для покупки ліміт на Покупку частинами.

Інформація про істотні характеристики продукту та попередження розміщені на сайті продукту: chast.monobank.ua

</div>

</div>

</div>

<div class="GI-title" id="GS">

Гарантія та сервіси

</div>

<div class="text">

Загальна гарантія від ByteShop

Магазин Ябло надає гарантію на нову та вживану техніку Apple, а також додаткові розширені програми обслуговування.

Офіційна гарантія Apple

На всю нову техніку поширюється офіційна гарантія виробника.

Гарантія магазину

Після покупки діє гарантія магазину терміном 31 день, з можливістю розширення до 1 або 2 років.

Пакети розширеної гарантії можуть включати:

1. гарантійну заміну у разі заводського браку протягом 3 днів діагностики;

2. сервісне та гарантійне обслуговування;

3. повну технічну підтримку;

4. індивідуальне налаштування пристрою;

5. встановлення необхідного ПЗ (для Mac);

6. знижку 10% на ремонт у разі негарантійних випадків;

7. знижки до 20% на захисні аксесуари;

8. Cashback на обмін техніки.

Додаткові програми захисту можуть включати:

1. одноразову безкоштовну заміну дисплея при пошкодженні;

2. одноразову заміну або ремонт пристрою при будь-якому негарантійному випадку (крім втрати);

3. 100% компенсацію вартості пристрою у разі неможливості ремонту;

4. чистку від пилу та заміну термопасти (для Mac);

5. обмін пристрою протягом 3 днів у випадку заводського браку.

Вживана техніка

На б/у моделі також діє гарантія магазину 31 день з можливістю розширення до 1–2 років з аналогічними умовами.

Особлива техніка

Гарантія на спеціальні категорії товарів надається згідно умов виробника та може бути розширена магазином.

</div>

<div class="GI-title" id="fix">

Ремонт

</div>

<div class="text">

Ремонт особливої техніки у Ябло

Сервісний центр у мережі Ябло — це місце, де вирішуються будь-які негаразди з пристроями у найкоротші терміни

та за вигідною вартістю. Розбили екран новенького смартфона? З'явилися несправності в улюбленого ноутбука або ж потрібно

почистити ігрову консоль? Хочете налаштувати або ж оновити програмне забезпечення в планшеті? Трапилася гарантійна поломка

в Apple Watch?

Поспішайте у Ябло і наша команда допоможе з ремонтом особливої техніки. У своїй роботі спеціалісти використовують лише

перевірені інструменти, сучасні технології та якісні запчастини. На виконанні ремонтні роботи мережа Ябло надає гарантію

терміном від 1 місяця.

Сучасна техніка від популярних брендів — це зразок досконалості та ідеальної комбінації дизайну та технологій. Та, на жаль,

навіть довершений технічний пристрій може пошкодитись після тривалого використання. Пил, волога, високі або ж низькі

температури — все це може негативно вплинути на функціональний стан.

Також, варто пам'ятати, що гаджети потребують діагностики, яка допоможе завчасно виявити несправності та усунути їх. Якщо

ви помітили зміни у роботі смартфона, ноутбук нагрівається або ж видає сторонні шуми, розумний годинник погано працює

після відвідин басейну — рекомендуємо якнайшвидше звернутись за консультацією до представників мережі магазинів Ябло у

вашому регіоні України.

Ремонт iPhone

Найчастіші несправності зі смартфонами від Apple пов'язані з дисплеєм, акумуляторною батареєю або ж динаміками. Здебільшого

причини їх появи криються у тривалому користуванні, відсутності захисних аксесуарів або ж у використанні несертифікованих

зарядних пристроїв.

Ябло — це сервісний центр в Україні, що пропонує послуги з ремонту усіх лінійок флагманських смартфонів всесвітньо відомого

бренду з долини Купертіно. Фахівці мережі Jabko займаються сервісним обслуговуванням всіх моделей, починаючи від iPhone

8-го покоління до флагманських iPhone 17 Pro Max.

Ремонт смартфонів Samsung, Xiaomi та Google

Окрім пристроїв Apple, мережа Ябло надає професійний ремонт смартфонів Samsung, Xiaomi, Google та інших популярних

виробників. Ми працюємо з усіма типовими та складними видами ремонту — від заміни дисплейних модулів та акумуляторів до

відновлення материнських плат.

Для відновлення використовуються перевірені комплектуючі, що відповідають вимогам бренду. Наші фахівці також допоможуть

із міграцією даних, оновленням та налаштуванням програмного забезпечення.

Після ремонту смартфон працюватиме швидко,

стабільно та без зайвих збоїв.

Ремонт планшетів iPad

Для значної частини користувачів, планшети вже давно замінили комп'ютер, адже основна їх перевага у портативності та

високому рівню зручності. Та часто подорожі, транспортування у сумках чи рюкзаках, перепад між високими та низькими

температурами може негативно позначитись на роботі планшета. Якщо у вашого iPad виникли несправності — варто звернутись

у сервісний центр в Україні. Мережа магазинів та сервісів Ябло здійснює ремонт усіх актуальних моделей планшетів від

Apple: iPad Mini, iPad Pro, iPad Air, iPad.

Майстри проводять детальну діагностику пристрою, після чого усувають виявлені проблеми у найкоротші терміни. Ми здійснюємо

ремонт роз'ємів, динаміків, кнопок, мікрофонів, а також заміну дисплейного модуля та скла. За необхідності надаємо

консультації щодо налаштувань, оновлення програмного забезпечення та будь-яких запитань, що можуть з'явитись у процесі

користування iPad.

Ремонт MacBook

MacBook — це чудовий інструмент для професіоналів, у якого достатньо потужності та можливостей для втілення екстравагантних

та складних задумів. Та для стабільної роботи не достатньо налаштувань та якісних аксесуарів, потрібно дещо більше.

Варто пам'ятати про сервісне обслуговування, періодичну заміну термопасти, чистку клавіатури, вентиляторів та портів.

Завдяки правильному підходу до користування та обслуговування можна значно продовжити термін служби пристрою. Ябло —

це сервісний центр в Україні, що надає послуги з ремонту та технічного обслуговування MacBook.

За необхідності, фахівці проведуть комплексну діагностику MacBook та вирішать навіть найскладніші проблеми. Варто знати,

що діагностика MacBook займає більше ніж 24 годин, адже необхідно перевірити всі елементи та програмне забезпечення

пристрою. Точний час діагностики та ремонту вам назвуть при прийомі пристрою в одному з магазинів Ябло. Уточнити деталі

та отримати консультацію ви зможете за номером безкоштовної гарячої лінії СЦ Ябло, який вказаний на квитанції.

Ремонт ноутбуків

Ноутбуки, як і будь-які інші пристрої, потребують періодичного обслуговування. У сервісному центрі Ябло доступні послуги

ремонту ноутбуків різних брендів та моделей: заміна клавіатури, акумулятора, екрану, портів, чистка системи охолодження,

заміна термоінтерфейсу та ремонт материнської плати.

Завдяки професійній діагностиці ми визначаємо першопричину несправності та пропонуємо оптимальні умови відновлення. Після

ремонту пристрій працюватиме тихіше, швидше та надійніше.

Ремонт iMac та iMac Pro

Один з найкрутіших інструментів для дизайнерів, звукорежисерів, фотографів, відеографів, архітекторів та професіоналів

у будь-якій сфері — це однозначно моноблок iMac та iMac Pro від Apple. Це гіганти, що розкривають безліч можливостей

завдяки розкішним дисплеям, що сягають розміром 21.5", 24" та 27" дюймів.

Ябло здійснює ремонт моноблоків від Apple за вигідною вартістю та з використанням лише оригінальних запчастин. Усі

ремонтні роботи проводяться лише після детальної діагностики пристрою та попередньо погоджуються з клієнтом. Вартість

ремонту коливається залежно від моделі та року випуску iMac.

Ремонт смарт-годинників Apple Watch

Мініатюрний та потужний, це слова, що найкраще описують розумні годинники від бренду з Купертіно — Apple Watch. Сім

поколінь годинників здобули любов та популярність серед мільйонів користувачів. Надійні механізми, гартоване скло,

анодований алюміній — якість матеріалів відмінна. Та при щоденному користуванні годинником вірогідність механічних

пошкоджень збільшується.

Якщо ж не використовувати захисне покриття для дисплея Apple Watch, то можна завдати пристрою вагомих проблем. Удари,

падіння, тривалий контакт з водою, вплив надмірно високих або ж низьких температур — все це може призвести до того, що

буде потрібен терміновий ремонт.

Найчастіше фахівці з ремонту в Ябло здійснюють заміну акумулятора, розбитого скла або ж модуля дисплея у розумних

годинниках. Вартість ремонту коливається залежно від покоління пошкодженого Apple Watch, а з прайсом ви можете

ознайомитись у таблиці на нашому сайті, що розташована у категорії “Ремонт”.

Ремонт техніки Dyson

Техніка Dyson — це поєднання інноваційних технологій та преміальних матеріалів. Проте навіть найякісніші пристрої

потребують регулярного сервісного обслуговування. У мережі Ябло ви можете відремонтувати фени та стайлери для волосся

Dyson, а також вертикальні та портативні пилососи виробника.

Фахівці нашого сервісного центру проводять діагностику та усунення несправностей двигуна, акумулятора, фільтраційної

системи та електронних модулів. У разі необхідності здійснюється заміна окремих компонентів на оригінальні або

сертифіковані аналоги. Ми повернемо ваш Dyson до стабільної та ефективної роботи у найкоротші терміни.

Ремонт ігрових консолей PlayStation та Xbox

PlayStation та Xbox — це центри домашніх розваг, але інтенсивне використання, накопичення пилу або механічні пошкодження

можуть призвести до збоїв у роботі. Сервісний центр Ябло здійснює ремонт та профілактичне обслуговування ігрових консолей

у всіх популярних конфігураціях.

Майстри проводять чистку та відновлення системи охолодження, заміну кулерів, термопасти, ремонт HDMI-портів, дискових

приводів та внутрішніх накопичувачів. За потреби можна модернізувати систему зберігання даних або відновити працездатність

геймпадів. Після ремонту консоль працюватиме стабільно навіть при тривалих ігрових сесіях.

Ремонт роботів-пилососів

Роботи-пилососи стали незамінними помічниками для дому, але їхня ефективність напряду залежить від стану сенсорів,

акумулятора і системи всмоктування. У Ябло ми здійснюємо діагностику роботів-пилососів різних брендів і моделей.

Виконуємо заміну батарей, сервіс очистки турбощіток і каналів, відновлення датчиків навігації, ремонтуємо блок живлення,

плату керування та систему обертання коліс. Після сервісу пилосос знову повертається до якісного прибирання.

Найпоширеніші запитання

У сервісний центр Ябло по всій Україні часто звертаються за консультацією щодо ремонту та профілактичного

обслуговування пристроїв. Команда професійних майстрів Ябло зібрала найпоширеніші запитання, та підготувала відповіді

на них.

1. Яка тривалість ремонту пристрою?

Точні терміни виконання роботи залежать від типу пристрою та складності ремонту. У середньому базовий ремонт

(заміна динаміків, батареї, екрану) займає орієнтовно 2-3 години.

2. Чи встановлюються оригінальні запчастини?

Для проведення максимально якісної роботи та безпеки пристрою використовуються лише перевірені та оригінальні компоненти.

3. Як і де залишити пристрій на ремонт?

У вкладці контакти ви можете знайти найближчий до вас магазин Ябло, а також контактні дані для зв'язку. Залишити заявку

на ремонт за вартістю сайту можливо заповнивши форму у розділі “Ремонт”. Для того, щоб залишити пристрій на ремонт —

достатньо звернутись в один із наших магазинів, або ж проконсультуватись за номером 0 800 30 777 5 (безкоштовно).

4. Яка вартість ремонту?

За умови оформлення заявки онлайн за вами закріплюється вартість вказана у прайс-листі на сайті. У офлайн точках вартість може відрізнятись та залежить від міста.

</div>

</div>

CSS-код для сторінки авторизації:

```
.general {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
}

.Title {
  background-color: #D3E3FF;
  border-radius: 20px;
  width: 40%;
  font-size: xx-large;
  font-weight: bold;
  display: flex;
  justify-content: center;
  align-items: center;
  border: 2px solid black;
  padding: 5px;
  margin: 10px;
}

.GI-title {
  background-color: #D3E3FF;
```

```
border-radius: 20px;
width: 37%;
font-size: x-large;
font-weight: bold;
display: flex;
justify-content: center;
align-items: center;
border: 2px solid black;
padding: 5px;
margin: 10px;
}

.text {
background-color: #D3E3FF;
border-radius: 20px;
width: 80%;
font-size: 100%;
display: flex;
flex-direction: row;
justify-content: center;
align-items: center;
border: 2px solid black;
padding: 10px;
margin: 10px;
}

.text div {
width: 33%;
float: left;
}

.text a {
text-decoration: none;
color: inherit;
}

.text a:hover {
```

```
    text-decoration: none;
}

.DP {
    width: 100%;
    display: flex;
    flex-direction: row;
    justify-content: center;
    align-items: center;
    padding: 10px;
    margin: 10px;
}

.DP div {
    width: 80%;
    margin: auto;
    margin: 10px;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}

.DP-title {
    background-color: #D3E3FF;
    border-radius: 20px;
    width: 100%;
    font-size: x-large;
    font-weight: bold;
    display: flex;
    justify-content: center;
    align-items: center;
    border: 2px solid black;
    padding: 5px;
    margin: 10px;
}
```

2.2.8 Реалізація компоненту footer

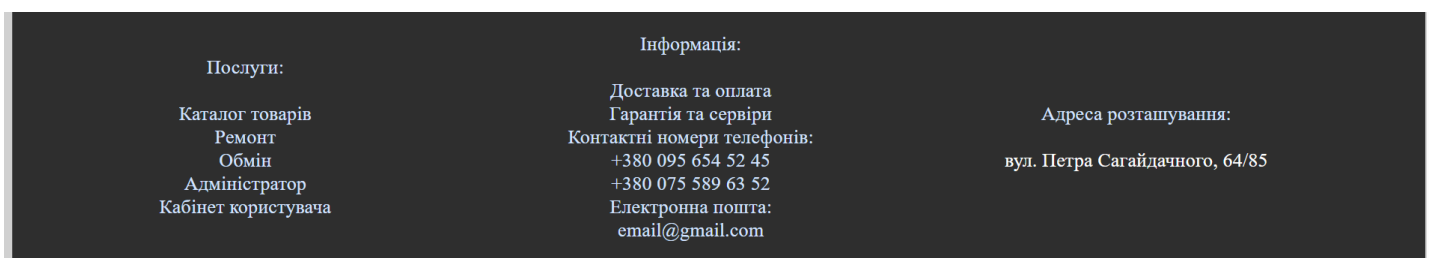


Рисунок 2.15 - вигляд компоненту footer на малих екранах



Рисунок 2.16 - вигляд компоненту footer на малих екранах

HTML-код для компоненту footer:

```
<div class="footer">
  <div class="Info">
    Послуги:<br>
    <br>
    <a href="index.html#List">Каталог товарів</a>
    <a href="pages/information.html#fix">Ремонт</a>
    Обмін<br>
    <a href="admin/index.html">Адміністратор</a>
    <a href="pages/cabinet.html">Кабінет користувача</a>
  </div>
  <div class="Info">
    <a href="pages/information.html">Інформація:</a><br>
    <a href="pages/information.html#DP">Доставка та оплата</a>
    <a href="pages/information.html#GS">Гарантія та сервіси</a>
```

Контактні номери телефонів:

+380 095 654 52 45

+380 075 589 63 52

Електронна пошта:

email@gmail.com

</div>

<div class="Info">

Адреса розташування:

вул. Петра Сагайдачного, 64/85

</div>

</div>

CSS-код для компоненту footer:

.footer {

width: 99%;

height: 1100%;

color: #D3E3FF;

background: #2E2E2E;

display: flex;

flex-direction: row;

display: grid;

grid-template-columns: 32% 32% 32%;

```

font-size: larger;
padding: 10px;
}
.Info {
margin: 5px;
font-size: 90%;
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
text-align: center;
padding: 5px 10px;
}
.Info a {
text-decoration: none;
color: inherit;
}
.Info a:hover {
text-decoration: none;
}

```

2.3 Функціонально стилістичні рішення

2.3.1 Адаптивна навігація

Зроблено адаптивну навігацію у формі бургер-меню на пристроях з малою роздільною здатністю та у формі стрічки в правій частині екрану на пристроях з великою роздільною здатністю. З виглядом та кодом для реалізації такої навігації можна ознайомитись у розділі 2.2.1 - реалізація компоненту header.

2.3.2 Адаптивна сітка контенту

Адаптивна сітка контенту реалізована засобами flexbox, із динамічною перебудовою залежно від ширини екрана. Властивість display: flex; застосована у класах types-block та photo-block для адаптивного розташування елементів меню та фотоблоків відповідно. Також flexbox використано у компоненті header для вирівнювання пошуку та кнопок входу/реєстрації, а у компоненті footer

використовується властивість `display: grid`; для розміщення інформаційних блоків.

2.3.3 Реалізація медіа-запитів

Медіа-запит реалізовано для адаптації стилю навігації під різні розміри екранів. З виглядом та кодом для реалізації такої навігації можна ознайомитися у розділі 2.2.1 — реалізація компоненту `header`. На сторінці товару використано додатковий медіа-запит, який відповідає за зміну поведінки елементів меню при зменшенні ширини екрана. Зокрема, при переході до мобільної версії автоматично приховуються кнопки «Інформація» та «Каталог», що дозволяє уникнути перевантаження інтерфейсу на малих екранах.

Для цього застосовано умову `@media (max-width: 768px)`, у межах якої відповідним елементам меню встановлено стиль `display: none`. Завдяки такому рішенню навігація лишається зручною й компактною, а користувач отримує адаптивний інтерфейс без зайвих відволікаючих елементів. Це забезпечує коректне відображення сторінки товару на планшетах та смартфонах.

2.3.4 Застосування відносних одиниць вимірювання

У проєкті переважно застосовано таку одиницю вимірювання, як відсотки (%). Де це доцільно залишено звичну одиницю виміру - `px`.

2.3.5 Реалізація візуальних ефектів та анімацій

У проєкті на основній сторінці не реалізовано візуальний ефект, проте він присутній на додаткових сторінках. Наприклад, інтерактивні фільтри на сторінках категорій (рисунки 2.17 та 2.18) супроводжуються анімаціями наведення, які покращують взаємодію користувача з інтерфейсом.

Для підсилення візуального ефекту використано низку `hover`-анімацій, що змінюють зовнішній вигляд елементів при наведенні курсора. Такі стилі застосовано до кнопок перемикавання (`.prev-btn`, `.next-btn`), елементів списків (`.types-item`, `.RL-item`), а також до кнопок каталогу (`.catalog-btn`), інформаційної кнопки (`.info-btn`) та кнопки пошуку (`.searchButton`). При наведенні фоновий колір змінюється на акцентний червоний відтінок `#E60023`, а текст — на контрастний колір, що забезпечує чіткість та зручність сприйняття.

Анімації також реалізовано для кнопки відкриття фільтрів (.buttonF), яка при наведенні змінює колір фону та тексту, підкреслюючи інтерактивність елемента. Додатково оформлено поведінку текстових посилань у блоках категорій: при наведенні змінюється колір тексту без додавання підкреслення, що підтримує загальну стилістичну цілісність сайту.

Таким чином, наведені hover-ефекти слугують не лише декоративною частиною дизайну, а й підвищують інтуїтивність інтерфейсу, підказуючи користувачу про можливість взаємодії з елементами. Завдяки цим анімаціям сторінки категорій стають більш динамічними та приємними у використанні.



Рисунок 2.17.1 - Реалізація візуальних ефектів та анімацій

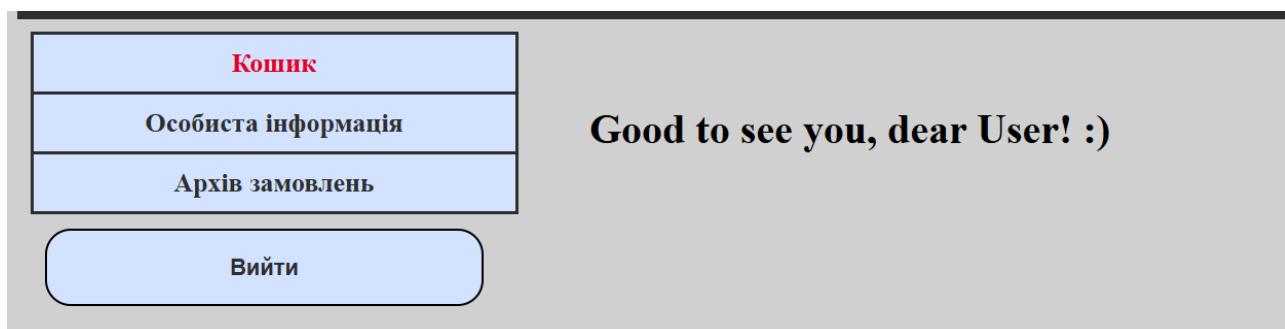


Рисунок 2.17.2 - Реалізація візуальних ефектів та анімацій

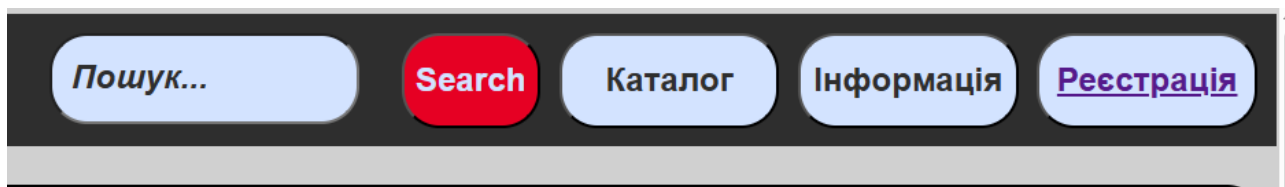


Рисунок 2.17.3 - Реалізація візуальних ефектів та анімацій

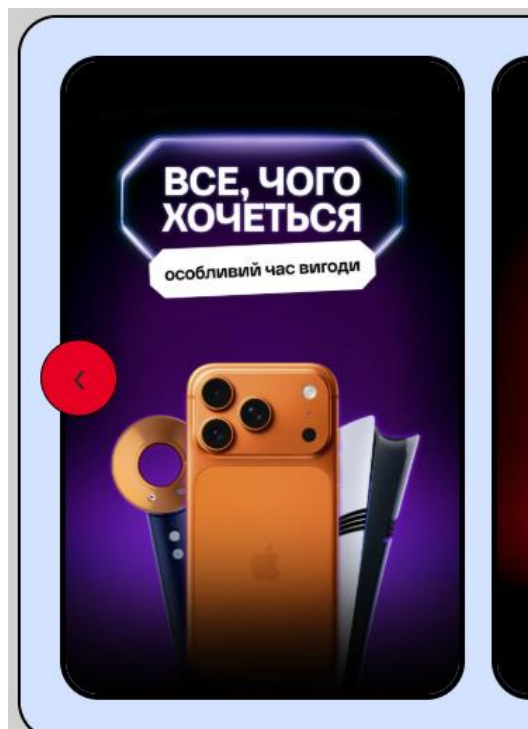


Рисунок 2.17.4 - Реалізація візуальних ефектів та анімацій

Ціна: 21 350 ₴

Чорний

Білий

Сірий

Нюдовий

128GB

256GB

512GB

1TB

basic

basic+

business

premium

Купити

Рисунок 2.17.5 - Реалізація візуальних ефектів та анімацій

Кошик

Особиста інформація

Архів замовлень

Вийти

Good to see you, dear User! :)

Рисунок 2.17.6 - Реалізація візуальних ефектів та анімацій

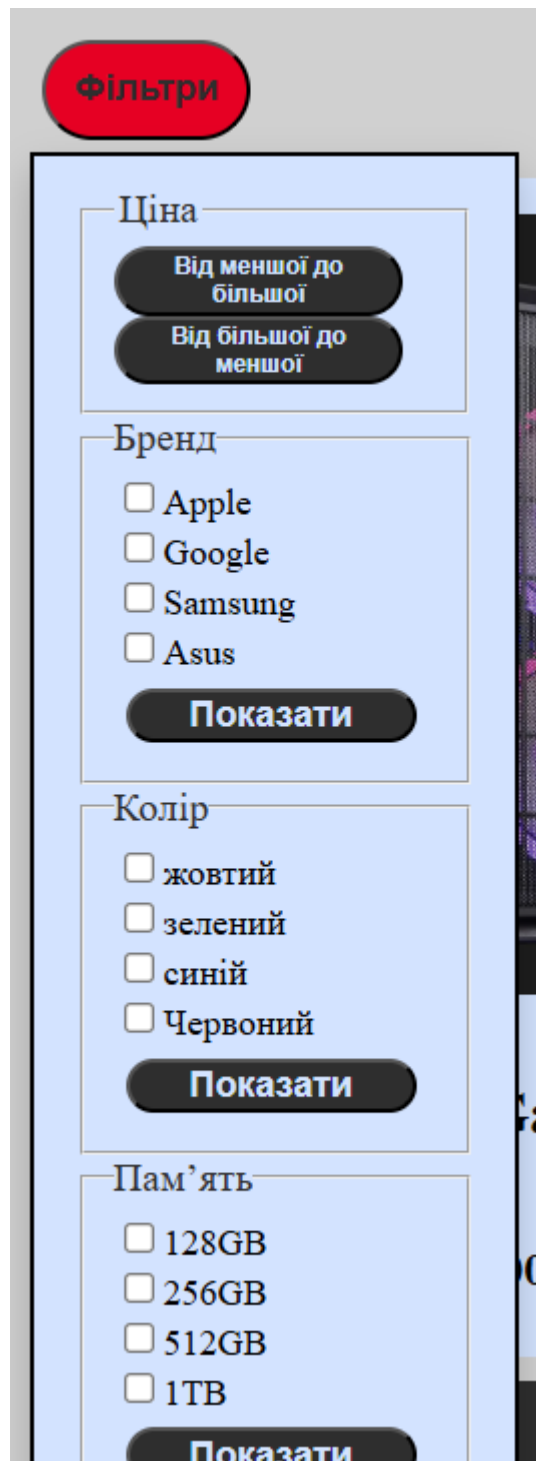


Рисунок 2.17.7 - Реалізація візуальних ефектів та анімацій

2.4 Файлова структура веб-застосунку

Файлова структура веб-застосунку повинна бути організована логічно:

HTML-документ, документ стилів, та документ для скрипту зберігаються окремо відповідно до їх призначення, що забезпечує зручність підтримки та масштабування проєкту. Структура організації зображена рисунку 2.19.

Коренева папка має такі папки та файли: папки images для зображень, pages для файлів типу html, styles для файлів типу css, файли index.html, README.md, script.js.

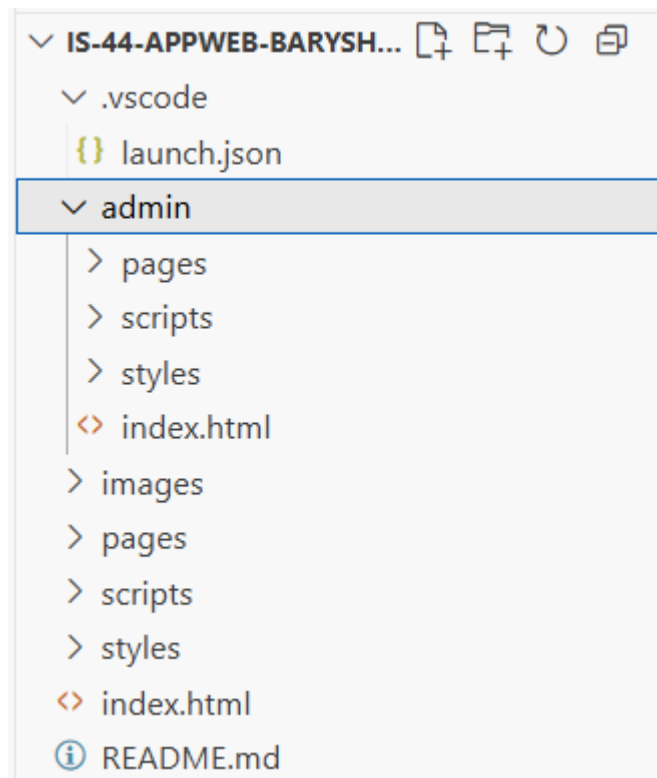


Рисунок 2.18 - Структура організації зображена рисунку

2.5 Репозиторій на GitHub

Посилання на репозиторій на GitHub:

<https://github.com/SandraBSV/IS-44-appWEB-BaryshevaAnastasiia-FIOT-2025>

Коміти, який було виконано:

fix: integrate a new main page

- 1. Add main style and index.html
- 2. Add folder with main photo

feat(template): register & login

- 1. create login page
- 2. create register page
- 3. add some styles for login and register pages

feat(template): type-page + styles

- 1. create 12 pages with types
- 2. create personal page of product (12 times)
- 3. add all styles for this pages
- 4. add particular images for products

fix(template): add info-page

- 1. create a new information page
- 2. add style for info-page
- 3. update links in footer in index.html

feat(template): personal cabiner

- 1. create user's personal cabinet
- 2. add some styles for personal cabinet
- 3. create additional pages: shopping bag, personal information and shopping list
- 4. add some styles for create additional pages

2.6 Висновки до розділу

У результаті виконання роботи було досягнуто мети — набуті практичні навички створення адаптивного інтерфейсу веб-застосунку з використанням HTML, CSS,

Flexbox та медіа-запитів. Реалізовано адаптивну навігацію, сітку контенту, анімації та візуальні ефекти. Організовано структуру проєкту та оформлено репозиторій з комітами на GitHub. Робота дозволила закріпити знання з верстки та підготувала базу для подальшого вивчення фронтенд-розробки.

3 АДМІНІСТРАТОРСЬКА ЧАСТИНА WEB-ЗАСТОСУНКУ: ІНТЕРФЕЙС ТА CRUD-ОПЕРАЦІЇ (ПРАКТИЧНА РОБОТА 3)

3.1 Мета роботи

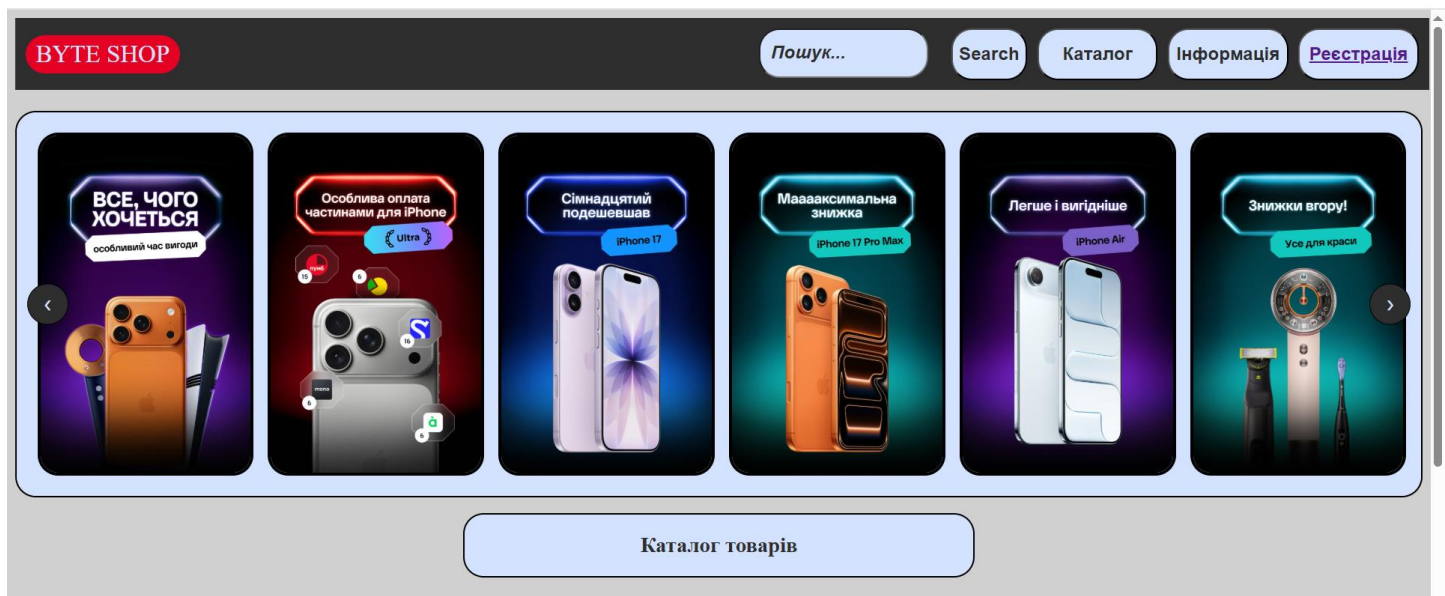
Мета роботи набути практичних навичок проектування та реалізації адміністраторської частини веб-застосунку з повним циклом CRUD-операцій над сутностями.

3.2 Візуальне відображення в різних станах сайту в залежності від стану користувача

3.2.1 Візуальне відображення для гостя

На рисунку 3.1 можна побачити вигляд сайту для гостя. Він може вільно переглядати сайт, дивитись вироби, а також замовляти їх, але йому не доступні різні акції та відслідковувати свої замовлення. Він може зареєструватись та відслідковувати їх в персональному кабінеті, а також автоматично зберігати інформацію для замовлень.

Тобто, коли гість/незареєстрований користувач відкриває сайт перед ним відкривається головна сторінка сайту. Він може бачити фото, каталог, та футер. Якщо він хоче побачити будь-який тип товару окремо, він натискає на нього та йому відкривається сторінка типу товару, де є його назва, фільтр пошуку певних товарів, а також самі товари. Якщо він хоче побачити окремо товар, то натискає на нього, йому відкривається сторінка товару, де є його назва, фото, ціна та опис. Він також може замовити товар/ товари.



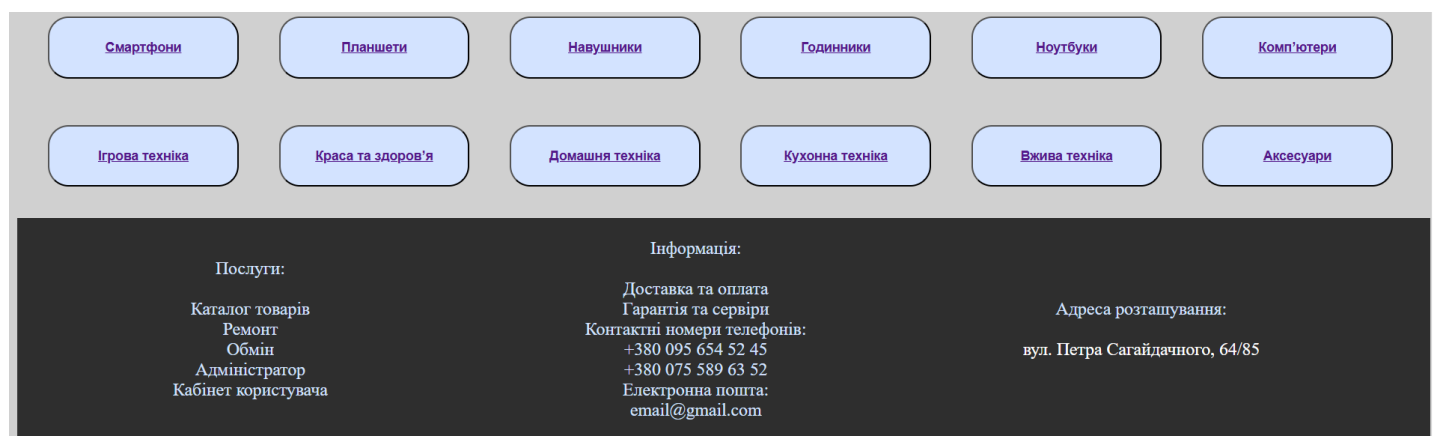


Рисунок 3.1 – вигляд сайту для гостя/незареєстрованого користувача

3.2.2 Візуальне відображення для зареєстрованого користувача

На рисунку 3.2 та 3.3 можна побачити вигляд сайту для зареєстрованого користувача, головну сторінку та персональний кабінет. Він може вільно переглядати сайт, дивитись товари, а також замовляти їх, також брати участь у різних акціях та подіях, відслідковувати свої замовлення в персональному кабінеті, а також автоматично зберігати інформацію для замовлень.

Тобто, якщо гість/незареєстрований користувач хочуть увійти/зареєструватись вони натискають Реєструватись, після цього заповнюють відповідну форму і відкривається персональний кабінет. Якщо це новий користувач, то відображається тільки персональна інформація, якщо вже мав кабінет, то історію замовлень та персональну інформацію. Головна сторінка та інші не відрізняються від гостя.

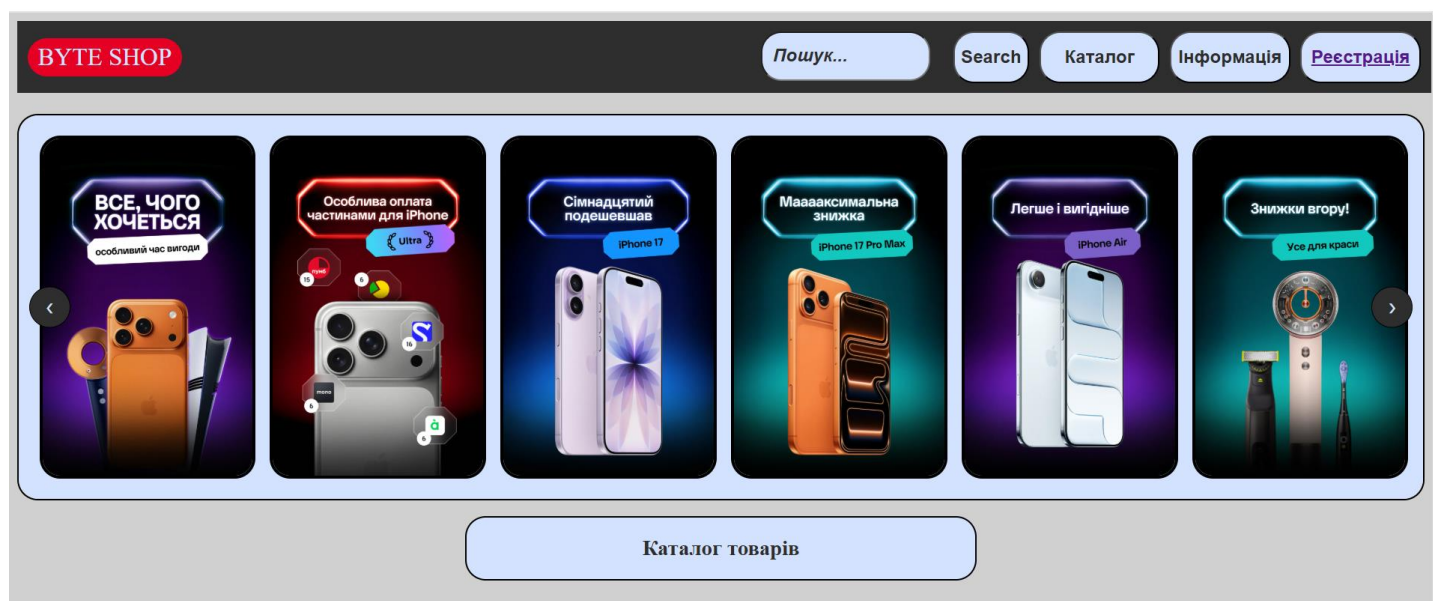




Рисунок 3.2 – вигляд сайту для зареєстрованого користувача, головна сторінка

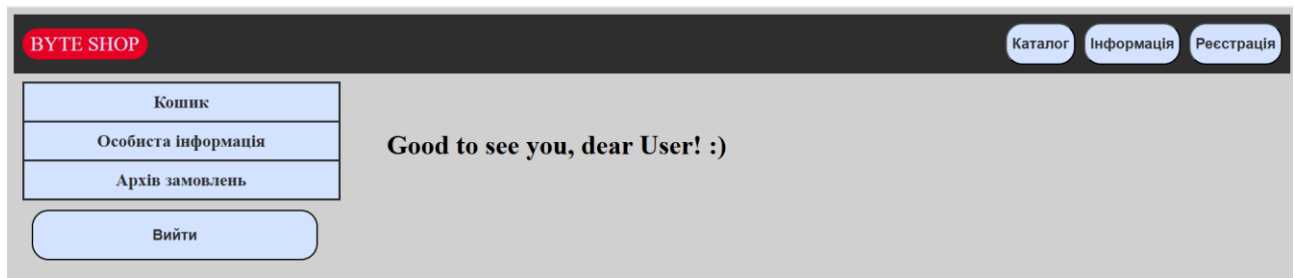


Рисунок 3.3 – вигляд сайту для зареєстрованого користувача, персональний кабінет

Оскільки частини коду цих сторінок були показані у розділі 2, а конкретно у пункт 2.2 Реалізація структурних компонентів інтерфейсу, то дублювати їх не будемо.

3.3 Реалізація адмін панелі

3.3.1 Візуальне відображення для Адміністратора

На рисунку 3.4-3. можна побачити вигляд сайту для адміністратора, головну сторінку та адмін панель. Йому доступно все теж саме, що й зареєстрованому користувачу, але додатково він може виконувати CRUD-операції із сутностями сайту.

Тобто, коли адміністратор авторизувався йому відкривається відразу адмін панель (рисунок 3.4), з якої можна повністю керувати сайтом. На ній з лівого боку є таблиця з посиланнями на редагування різних категорій, а також привітання до адміністратора. Коли адміністратор натискає на посилання будь-якої категорії, його перекидає на сторінку категорії (рисунок 3.5), яку він натиснув для перегляду. (тобто операція READ), там з'являється таблиця з редагуванням. В ній повна інформація про вироби, він може редагувати інформацію відразу в таблиці (тобто операція UPDATE) та для збереження натискання кнопки «Оновити», коли адміністратор редагує виріб йому підсвічується вікно, яке він редагує (рисунок 3.6). Може видалити виріб (тобто операція DELETE). А також під таблицею виробів є кнопка «Створити», щоб створити виріб відповідно (тобто операція CREATE). Так створено для кожної

категорії виробів.

Якщо натиснути на назву сайту у хедері зі сторінки категорії, то відкриється головна сторінка адмін панелі, і якщо натиснути з головної сторінки адмін панелі на назву, то відкриється головна сторінка сайту. Адміністратор може входити до своєї панелі за допомогою посилання на адмін панель у хедері після авторизації.

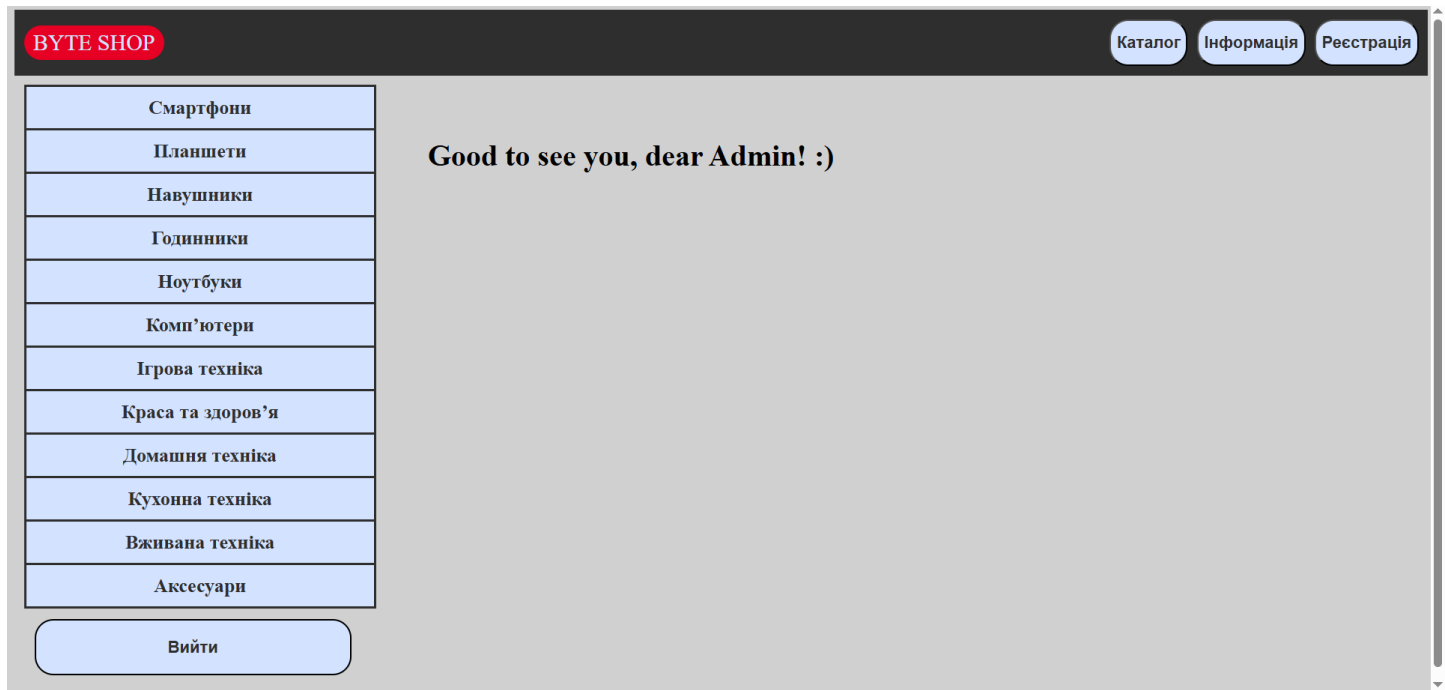


Рисунок 3.4 – Вигляд адмін панелі на сайті після авторизації

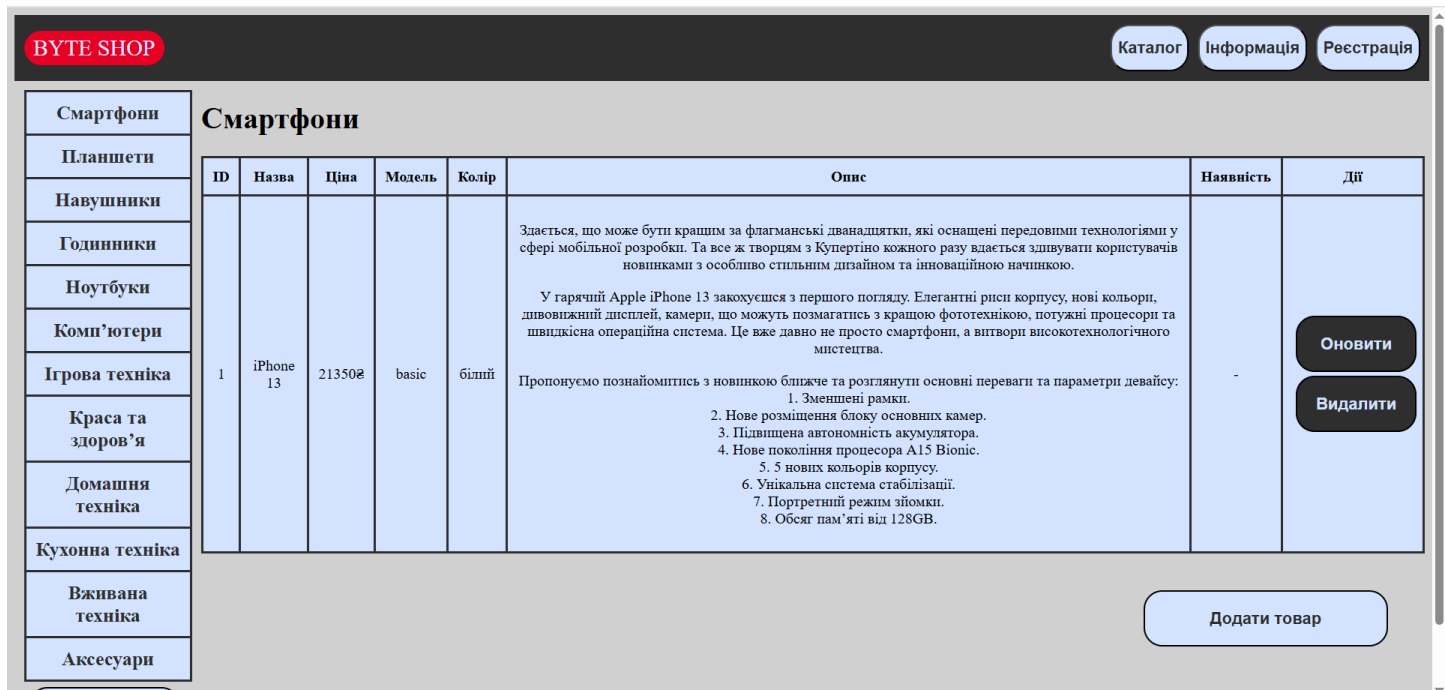


Рисунок 3.5 – Вигляд редагування категорії в адмін панелі

Смартфони

ID	Назва	Ціна	Модель	Колір	Опис	Наявність	Дії
1	iPhone 13	21350€	basic	білий	Здається, що може бути кращим за флагманські дванадцятки, які оснащені передовими технологіями у сфері мобільної розробки. Та все ж творцям з Купертіно кожного разу вдається здивувати користувачів новинками з особливо стильним дизайном та інноваційною начинкою. У гарячий Apple iPhone 13 закохуєшся з першого погляду. Елегантні риси корпусу, нові кольори, дивовижний дисплей, камери, що можуть позмагатись з кращою фототехнікою, потужні процесори та швидкісна операційна система. Це вже давно не просто смартфони, а витвори високотехнологічного мистецтва. Пропонуємо познайомитись з новинкою ближче та розглянути основні переваги та параметри девайсу: 1. Зменшені рамки. 2. Нове розміщення блоку основних камер. 3. Підвищена автономність акумулятора. 4. Нове покоління процесора A15 Bionic. 5. 5 нових кольорів корпусу. 6. Унікальна система стабілізації. 7. Портретний режим зйомки. 8. Обсяг пам'яті від 128GB.	-	Оновити Видалити

Рисунок 3.6 – Вигляд редагування таблиці виробів в категорії в адмін панелі

3.3.2 Реалізація головної сторінки для Адміністратора

Рисунок головної сторінки наведено на рисунку 3.4, ось код її реалізації:

HTML-код реалізації index.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>BYTE SHOP</title>
  <link rel="stylesheet" href="styles/main.css">
</head>
<body>
  <!-- Header -->
  <header>
    <div class="title">
      <a href=" ../index.html" style="color:#D3E3FF; font-size: x-large; text-decoration:
none;">BYTE SHOP</a>
    </div>
    <div style="display: flex; gap: 10px; float:right;">
      <button class="RL-item" id="">Каталог</button>
      <button class="RL-item" id="">Інформація</button>
      <button class="RL-item" id="RL">Реєстрація</button>
    </div>
  </header>

```

```
</header>
```

```
<div class="general">
```

```
  <div class="table">
```

```
    <table>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/phone.html">Смартфони</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/pad.html">Планшети</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/pods.html">Навушники</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/watch.html">Годинники</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/laptop.html">Ноутбуки</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/computer.html">Комп'ютери</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/game.html">Ігрова техніка</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/prettyHealth.html">Краса та здоров'я</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/home.html">Домашня техніка</a></th>
```

```
      </tr>
```

```
      <tr>
```

```
        <th class="types-item"><a href="pages/kitchen.html">Кухонна техніка</a></th>
```

```

</tr>
<tr>
  <th class="types-item"><a href="pages/BU.html">Вживана техніка</a></th>
</tr>
<tr>
  <th class="types-item"><a href="pages/add.html">Акcesуари</a></th>
</tr>
</table>
<button class="button" type="button">Вийти</button>
</div>
<div class="main">
  <h1 class="titl">
    Good to see you, dear Admin! :)
  </h1>
</div>

</div>
<script src="../scripts/logout.js"></script>

```

```

</body>

```

CSS-код реалізації main.css:

```

header {
  width: 99%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: #FFFFFFF;
  background: #2E2E2E;
  padding: 10px;
}
.title {
  background: #E60023;
  border-radius: 20px;

```

```
height: 100%;  
padding-left: 10px;  
padding-right: 10px;  
padding-bottom: 5px;  
padding-top: 5px;
```

```
}  
.RL-item {  
  Width: 60%;  
  height: 50px;  
  border-radius: 20px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: #D3E3FF;  
  transition: background 0.3s;  
  font-size: 110%;  
  font-weight: bold;  
  color: #2E2E2E;  
  text-decoration: none;  
}
```

```
.RL-item:hover {  
  background-color: #E60023;  
  color: #D3E3FF;  
}
```

```
body {  
  background-color: #D0D0D0;  
}
```

```
.general {  
  display: flex;  
  flex-direction: row;  
  width: 100%;  
}
```

```
.table {
```

```
width: 25%;  
display: flex;  
flex-direction: column;  
padding: 10px;
```

```
}
```

```
table, th, td {
```

```
background-color: #D3E3FF;  
border: 3px solid #2E2E2E;  
border-collapse: collapse;  
padding: 10px;  
text-align: center;  
vertical-align: middle;
```

```
}
```

```
.button {
```

```
background: #D3E3FF;  
margin: 3%;  
border-radius: 20px;  
font-size: 110%;  
font-weight: bold;  
width: 90%;  
height: 60px;  
padding-left: 20px;  
padding-top: 1px;  
padding-bottom: 1px;  
padding-right: 20px;  
display: flex;  
align-items: center;  
justify-content: center;  
border: 2px solid black;  
color: #2E2E2E;
```

```
}
```

```
.button:hover {
```

```
background-color: #E60023;
```

```

color:#D3E3FF;
}

.types-item a {
font-size: 130%;
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
}

.types-item a:hover {
color: #E60023;
text-decoration: none;
}

.main {
margin: 3%;
}

```

3.3.3 Реалізація сторінки редагування категорії виробів для Адміністратора

Рисунок головної сторінки наведено на рисунку 3.5, ось код її реалізації:

HTML-код реалізації phone.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>BYTE SHOP</title>
  <link rel="stylesheet" href="../styles/type.css">
</head>
<body>
  <!-- Header -->
  <header>
    <div class="title">
      <a href="../index.html" style="color:#D3E3FF; font-size: x-large; text-decoration:
none;">BYTE SHOP</a>
    </div>

```

```

<div style="display: flex; gap: 10px; float:right;">
  <button class="RL-item" id="">Каталог</button>
  <button class="RL-item" id="">Інформація</button>
  <button class="RL-item" id="RL">Реєстрація</button>
</div>
</header>
<div class="general">
  <div class="table">
    <table>
      <tr>
        <th class="types-item"><a href="phone.html">Смартфони</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="pad.html">Планшети</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="pods.html">Навушники</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="watch.html">Годинники</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="laptop.html">Ноутбуки</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="computer.html">Комп'ютери</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="game.html">Ігрова техніка</a></th>
      </tr>
      <tr>
        <th class="types-item"><a href="prettyHealth.html">Краса та здоров'я</a></th>
      </tr>
    </table>
  </div>
</div>

```

```

<tr>
  <th class="types-item"><a href="home.html">Домашня техніка</a></th>
</tr>
<tr>
  <th class="types-item"><a href="kitchen.html">Кухонна техніка</a></th>
</tr>
<tr>
  <th class="types-item"><a href="BU.html">Вживана техніка</a></th>
</tr>
<tr>
  <th class="types-item"><a href="add.html">Акcesуари</a></th>
</tr>
</table>
<button class="buttonE" type="button">Вийти</button>
</div>
<div class="main">
  <h1 class="titleTopic">Смартфони</h1>
  <table class="product" id="productTable">
    <tr>
      <th>ID</th>
      <th>Назва</th>
      <th>Ціна</th>
      <th>Модель</th>
      <th>Колір</th>
      <th>Опис</th>
      <th>Наявність</th>
      <th>Дії</th>
    </tr>
    <tr>
      <td>1</td>
      <td contenteditable="true">iPhone 13</td>
      <td contenteditable="true">21350€</td>
      <td contenteditable="true">basic</td>

```

<td contenteditable="true">білий</td>

<td contenteditable="true">

<p>

Здається, що може бути кращим за флагманські дванадцятки, які оснащені передовими технологіями у сфері мобільної розробки.

Та все ж творцям з Купертіно кожного разу вдається здивувати користувачів новинками з особливо стильним дизайном та інноваційною начинкою.

</p>

<p>

У гарячий Apple iPhone 13 закохуєшся з першого погляду. Елегантні риси корпусу, нові кольори, дивовижний дисплей, камери,

що можуть позмагатись з кращою фототехнікою, потужні процесори та швидкісна операційна система. Це вже давно не просто смартфони, а витвори високотехнологічного мистецтва.

</p>

<p>

Пропонуємо познайомитись з новинкою ближче та розглянути основні переваги та параметри девайсу:

1. Зменшені рамки.

2. Нове розміщення блоку основних камер.

3. Підвищена автономність акумулятора.

4. Нове покоління процесора A15 Bionic.

5. 5 нових кольорів корпусу.

6. Унікальна система стабілізації.

7. Портретний режим зйомки.

8. Обсяг пам'яті від 128GB.

</p>

</td>

<td contenteditable="true">-</td>

<td>

<button class="button1" type="button">Оновити</button>

<button class="button1" type="button">Видалити</button>

```

        </td>
    </tr>
</table>
<div class="create">
    <button class="button" type="button">Додати товар</button>
</div>
</div>
</div>
<script>const PRODUCT_TYPE = "braslet"</script>
<script src="../scripts/type.js"></script>
</body>

```

CSS-код реалізації type.css:

```

header {
    width: 99%;
    display: flex;
    justify-content: space-between;
    align-items: center;
    color: #FFFFFF;
    background: #2E2E2E;
    padding: 10px;
}
.title {
    background: #E60023;
    border-radius: 20px;
    height: 100%;
    padding-left: 10px;
    padding-right: 10px;
    padding-bottom: 5px;
    padding-top: 5px;
}
.RL-item {
    Width: 60%;
    height: 50px;

```

```
border-radius: 20px;
display: flex;
align-items: center;
justify-content: center;
background: #D3E3FF;
transition: background 0.3s;
font-size: 110%;
font-weight: bold;
color: #2E2E2E;
text-decoration: none;
}
RL-item:hover {
background-color: #E60023;
color: #D3E3FF;
}
body {
background-color: #D0D0D0;
}
.general {
display: flex;
flex-direction: row;
width: 100%;
}
.table {
width: 25%;
display: flex;
flex-direction: column;
padding: 10px;
}
table, th, td {
background-color: #D3E3FF;
border: 3px solid #2E2E2E;
border-collapse: collapse;
```

```
padding: 10px;
text-align: center;
vertical-align: middle;
}

.buttonE {
    background: #D3E3FF;
    margin: 3%;
    border-radius: 20px;
    font-size: 110%;
    font-weight: bold;
    width: 90%;
    height: 60px;
    padding-left: 20px;
    padding-top: 1px;
    padding-bottom: 1px;
    padding-right: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 2px solid black;
    color: #2E2E2E;
}

.buttonE:hover {
    background-color: #E60023;
    color: #D3E3FF;
}

.button1 {
    background: #2E2E2E;
    margin: 3%;
    border-radius: 20px;
    font-size: 110%;
    font-weight: bold;
    width: 100%;
```

```
height: 60px;
padding-left: 20px;
padding-top: 1px;
padding-bottom: 1px;
padding-right: 20px;
display: flex;
align-items: center;
justify-content: center;
border: 2px solid black;
color: #D3E3FF;
}

.button1:hover {
    background-color: #E60023;
    color: #2E2E2E;
}

.button {
    background: #D3E3FF;
    margin: 3%;
    float: right;
    border-radius: 20px;
    font-size: 110%;
    font-weight: bold;
    width: 20%;
    height: 60px;
    padding-left: 20px;
    padding-top: 1px;
    padding-bottom: 1px;
    padding-right: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 2px solid black;
    color: #2E2E2E;
```

```

}

.button:hover {
    background-color: #E60023;
    color: #D3E3FF;
}

.types-item a {
    font-size: 130%;
    font-weight: bold;
    color: #2E2E2E;
    text-decoration: none;
}

.types-item a:hover {
    color: #E60023;
    text-decoration: none;
}

```

Таким чином було створено десять сторінок для редагування всіх категорій виробів. Тобто сторінки на рисунку 3.7.

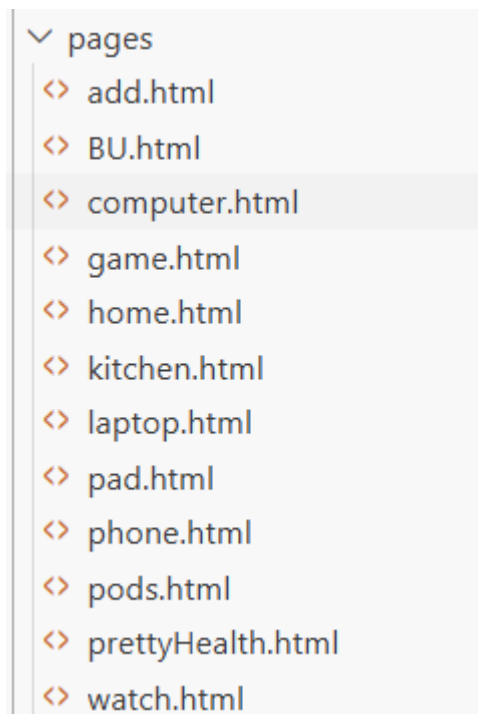


Рисунок 3.7 – Сторінки категорій

3.4 Файлова структура адмінської частини проекту

Папка `admin` має наступні компоненти: папки `images`, `pages`, `styles`, а також головний файл `index.html`.

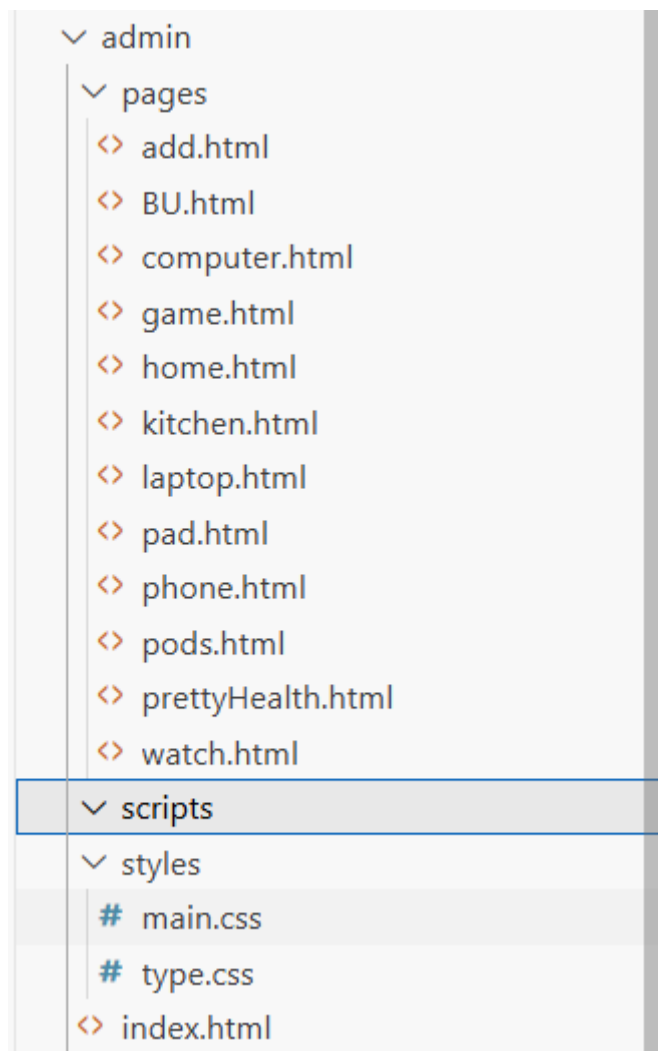


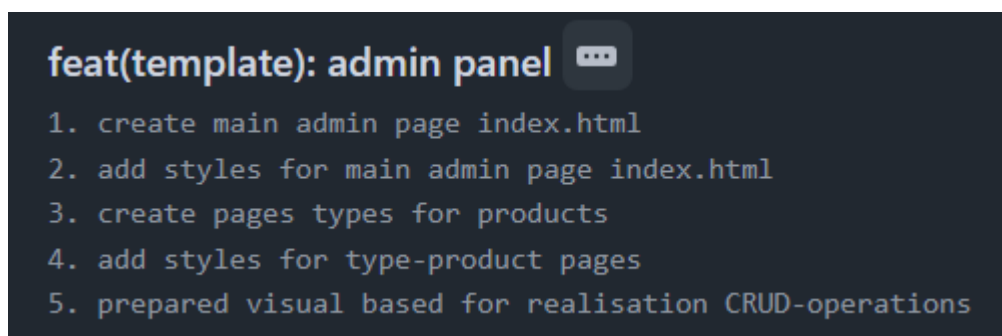
Рисунок 3.8 – Файлова структура адмінської частини проекту

3.5 Репозиторій на GitHub

Посилання на репозиторій на GitHub:

<https://github.com/SandraBSV/IS-44-appWEB-BaryshevaAnastasiia-FIOT-2025>

Коміти, який було виконано:



3.6 Висновки до розділу

У ході виконання практичної роботи було розроблено статичний макет інтерфейсу веб-застосунку з урахуванням різних ролей користувачів: гостя, звичайного користувача та адміністратора. Створено та стилізовано окремі HTML-

сторінки, що демонструють зовнішній вигляд і логіку переходів між станами системи.

Було реалізовано:

- головну сторінку для гостя з базовою навігацією;
- сторінку користувача з персоналізованим інтерфейсом;
- адміністраторську частину з можливістю виконання CRUD-операцій над сутностями (у вигляді макета);
- стилізовані форми входу з декоративними елементами та зрозумілою візуальною структурою.

Забезпечено єдиний дизайн для всіх ролей з чітким візуальним розмежуванням прав доступу. Розроблені сторінки дозволяють проілюструвати логіку майбутнього веб-застосунку та слугують основою для подальшої реалізації динамічного функціоналу.

Таким чином, у процесі роботи набуті практичні навички проєктування інтерфейсів адміністраторської частини веб-застосунків та закладено основу для реалізації повного циклу CRUD-операцій.

3.7 Контрольні запитання

1. Адміністраторська частина веб-застосунку призначена для керування вмістом сайту, користувачами та іншими сутностями. Вона дає змогу адміністратору виконувати операції редагування, видалення, додавання та перегляду даних.
2. CRUD-операції — це Create (створення), Read (читання), Update (оновлення) та Delete (видалення). В адмінській частині зазвичай реалізуються всі ці операції для керування контентом або користувачами.
3. У макеті передбачені ролі гість, користувач і адміністратор. Гість бачить лише загальну інформацію, користувач має персональний доступ, а адміністратор — інтерфейс з елементами керування та CRUD-операціями.
4. Сторінка входу повинна містити поля для введення логіна та пароля, кнопку входу, а також декоративні або інформаційні повідомлення про статус входу.

5. Візуальне розмежування інтерфейсів досягається за допомогою кольорової гами, розміщення елементів, відмінного меню та спеціальних панелей для адміністратора.

6. Під час оформлення інтерфейсу адміністратора важливо враховувати зручність, простоту навігації, логічну структуру та швидкий доступ до основних функцій.

7. Єдність стилю забезпечується використанням спільних CSS-класів, однакової кольорової схеми, шрифтів і сітки верстки на всіх сторінках.

8. Окремі HTML-сторінки або секції для кожного стану дозволяють чітко розділити доступні функції та забезпечити зручну навігацію між різними ролями користувачів.

9. Для побудови форм входу використовуються базові елементи HTML — `<form>`, `<input>`, `<label>`, `<button>` — та стилізація через CSS.

10. Декоративні повідомлення можна оформити за допомогою стилів CSS, використовуючи кольори, анімації, блоки повідомлень (`<div>`) або псевдокласи без JavaScript.

11. Логічна організація файлової структури допомагає підтримувати порядок у проєкті, спрощує навігацію для розробників та полегшує подальшу підтримку й розвиток застосунку.

4 СТВОРЕННЯ РЕЛЯЦІЙНОЇ БАЗИ ДАНИХ ВІДПОВІДНО (ПРАКТИЧНА РОБОТА 4)

Мета роботи:

Мета роботи набути навичок логічного та фізичного проєктування реляційної БД для веб-застосунку: визначення сутностей і зв'язків, нормалізація, розроблення, початкове наповнення, а також моделювання користувачів, ролей і прав доступу.

Опис робочого середовища

У даній лабораторній роботі я працюю в системі керування базами даних **phpMyAdmin**, яка функціонує на базі СУБД **MySQL**. Для створення локального середовища розробки було використано програмний комплекс **Open Server**, що забезпечує роботу веб-сервера, інтерпретатора PHP та сервера баз даних **MySQL** на персональному комп'ютері.

Для налаштування середовища роботи було виконано такі основні кроки:

1. З офіційного сайту <https://ospanel.io> завантажено інсталяційний пакет **Open Server**.
2. Архів програми було розпаковано у каталог `C:\OpenServer`, після чого запущено виконуваний файл **Open Server.exe**.
3. У меню «Налаштування → Сервер» було обрано необхідні компоненти:
 - **HTTP-сервер** — Apache;
 - **Інтерпретатор PHP** — актуальної версії (8.1 або вище);
 - **Система керування базами даних** — MySQL;
 - **Веб-інтерфейс для роботи з БД** — phpMyAdmin.
4. Після збереження параметрів сервер було запущено, і його коректну роботу підтверджено зеленою іконкою в системному треї.
5. Для входу до phpMyAdmin у браузері відкрито адресу `http://localhost/phpmyadmin`, де здійснено авторизацію під користувачем **root** (без пароля за замовчуванням).

Після виконання зазначених дій отримано повнофункціональне локальне середовище для створення, редагування та адміністрування баз даних у СУБД **MySQL** за допомогою **phpMyAdmin**. Таке середовище забезпечує зручну роботу з

таблицями, виконання SQL-запитів і візуальне керування даними без потреби у зовнішньому сервері.

4.1 Створення базових таблиць бази даних, відповідно до ER-моделі, покривши «бізнес-об’єкти» сайту

4.1.1 Створення таблиць

Відповідно до ER-моделі, рисунок 4.1, нам необхідно створити такі таблиці: Customer, Product, Line, SaleInvoice, Remainder, Storage. Скрипт створення цих таблиць з відповідними атрибутами наведений на рисунках 4.2-4.7 відповідно.

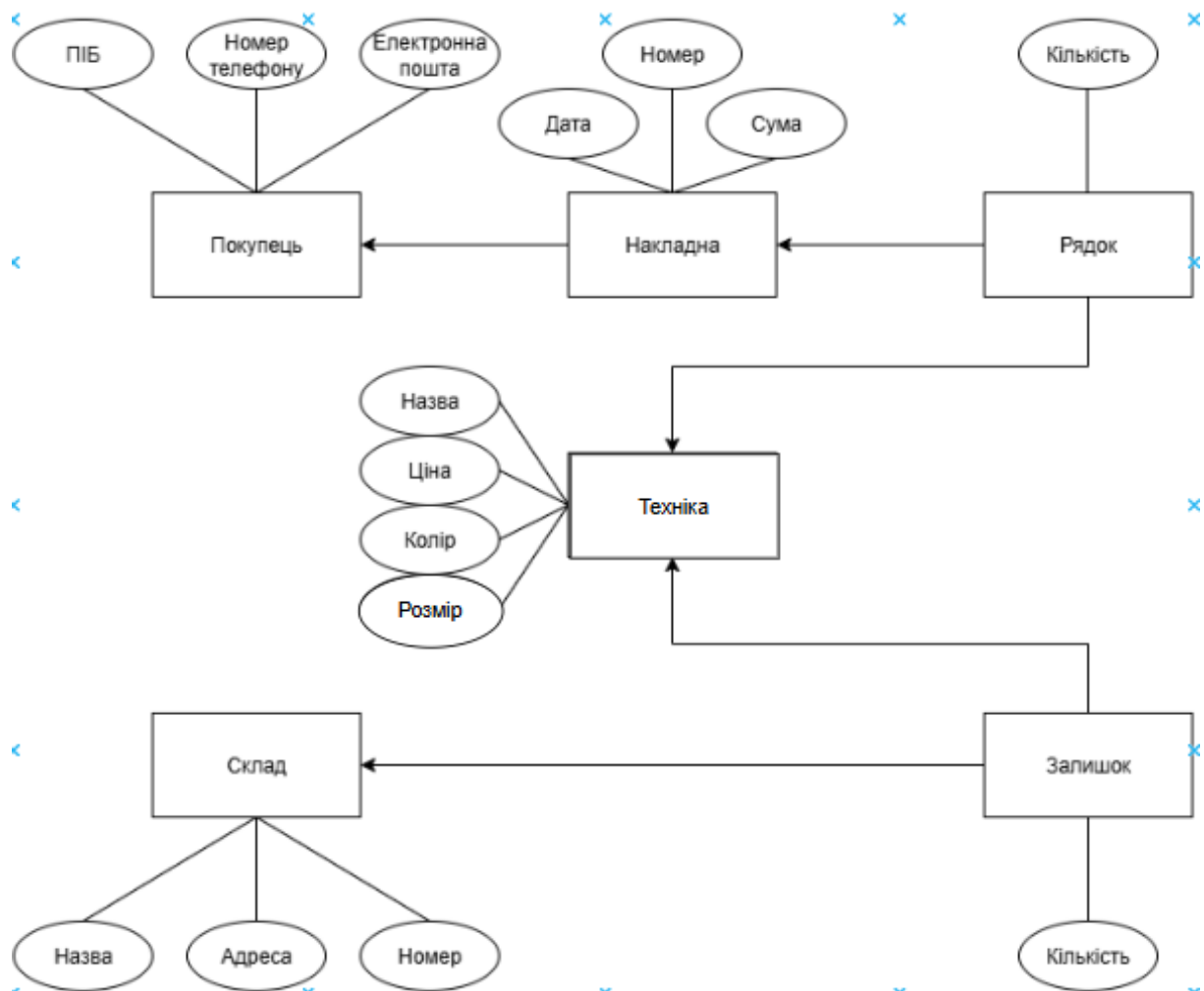


Рисунок 4.1 – ER-модель веб-застосунку «BYTE SHOP»

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 CREATE TABLE client (
2     clientId INT PRIMARY KEY AUTO_INCREMENT,
3     clientFisrtname VARCHAR(50) NOT NULL,
4     clientLastname VARCHAR(50) NOT NULL,
5     clientBirthday VARCHAR(50) NOT NULL,
6     clientPhone VARCHAR(50) NOT NULL UNIQUE,
7     clientEmail VARCHAR(50) NOT NULL UNIQUE
8 );
9

```

Рисунок 4.2 – скрипт по створенню таблиці «client»

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 CREATE TABLE product (
2     productId INT PRIMARY KEY AUTO_INCREMENT,
3     productName VARCHAR(50) NOT NULL UNIQUE,
4     productPrice DECIMAL NOT NULL,
5     productType VARCHAR(50) NOT NULL,
6     productModel VARCHAR(50) NOT NULL,
7     productColor VARCHAR(50) NOT NULL,
8     productAvaliability VARCHAR(50) NOT NULL
9 );
10

```

Рисунок 4.3 – скрипт по створенню таблиці «product»

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 CREATE TABLE linePD (
2     linePDId INT PRIMARY KEY AUTO_INCREMENT,
3     linePDAmount INT NOT NULL,
4     linePDSum INT NOT NULL,
5     productId INT NOT NULL,
6     invoiceId INT NOT NULL
7 );
8

```

Рисунок 4.4 – скрипт по створенню таблиці «linePD»

Виконати SQL запит/запити у базі даних byteShop: ?

```
1 CREATE TABLE invoice (  
2     invoiceId INT PRIMARY KEY AUTO_INCREMENT,  
3     invoiceNumber VARCHAR(50) NOT NULL UNIQUE,  
4     invoiceDate DATE NOT NULL,  
5     productId INT NOT NULL  
6 );  
7
```

Рисунок 4.5 – скрипт по створенню таблиці «invoice»

Виконати SQL запит/запити у базі даних byteShop: ?

```
1 CREATE TABLE remainder (  
2     remainderId INT PRIMARY KEY AUTO_INCREMENT,  
3     remainderAmount INT NOT NULL,  
4     productId INT NOT NULL,  
5     storageId INT NOT NULL  
6 );  
7
```

Рисунок 4.6 – скрипт по створенню таблиці «remainder»

Виконати SQL запит/запити у базі даних byteShop: ?

```
1 CREATE TABLE storage (  
2     storageId INT PRIMARY KEY AUTO_INCREMENT,  
3     storageNumber VARCHAR(50) NOT NULL,  
4     storageAdress VARCHAR(50) NOT NULL  
5 );  
6
```

Рисунок 4.7 – скрипт по створенню таблиці «storage»

4.1.2 Заповнення таблиць

Далі необхідно заповнити такі таблиці: client, product, linePD, invoice, remainder, storage. Скрипт заповнення таблиць на рисунках 4.8-4.13 відповідно.

Виконати SQL запит(и) у базі даних byteShop.client: ⓘ

```

1 INSERT INTO `client`(`clientFisrtname`, `clientLastname`, `clientBirthday`, `clientPhone`, `clientEmail`)
2 VALUES
3 ('John', 'Smith', '1990-05-12', '+380501234567', 'john.smith@example.com'),
4 ('Emily', 'Brown', '1988-11-02', '+380671112233', 'emily.brown@example.com'),
5 ('Michael', 'Johnson', '1995-07-29', '+380931234567', 'm.johnson@example.com'),
6 ('Olivia', 'Davis', '2000-03-15', '+380991112244', 'olivia.davis@example.com'),
7 ('William', 'Taylor', '1985-12-09', '+380661234890', 'william.taylor@example.com');
8 |

```

Рисунок 4.8 – скрипт по заповненню таблиці «client»

Виконати SQL запит(и) у базі даних byteShop.product: ⓘ

```

1 INSERT INTO `product`(`productName`, `productPrice`, `productType`, `productModel`, `productColor`, `productAvaliability`)
2 VALUES
3 ('iPhone 13', 21350, 'phone', 'basic', 'білий', '-'),
4 ('Samsung Galaxy Tab S11', 32800, 'pad', 'basic', 'чорний', '+'),
5 ('Marshall Major V', 4500, 'pods', 'basic', 'чорний', '+'),
6 ('Garmin Fenix 8 47mm AMOLED', 32200, 'watch', 'basic', 'чорний', '+'),
7 ('GIGABYTE G6 KF 2024 16"', 43000, 'laptop', 'basic', 'чорний', '-'),
8 ('ARTLINE Gaming X47', 34000, 'computer', 'basic', 'чорний', '+'),
9 ('Sony PlayStation 5', 37000, 'game', 'basic', 'чорний', '+'),
10 ('Dyson Airwrap Coanda2x', 35000, 'prettyHealth', 'basic', 'чорний', '-'),
11 ('Roborock Vacuum Cleaner Q8', 11000, 'home', 'basic', 'чорний', '-'),
12 ('Тостер SMEG', 6000, 'kitchen', 'basic', 'білий', '+'),
13 ('6/у Наушники Apple AirPods Max', 13000, 'BU', 'premium', 'рожевий', '-'),
14 ('Бездротова зарядка Belkin 3in1', 8000, 'add', 'premium', 'чорний', '+');
15 |

```

Рисунок 4.9 – скрипт по заповненню таблиці «product»

Виконати SQL запит(и) у базі даних byteShop.linePD: ⓘ

```

1 INSERT INTO `linePD` (`linePDId`, `linePDAmount`, `linePDSum`, `productId`, `invoiceId`) VALUES
2 (1, 2, 500, 1, 1),
3 (2, 1, 250, 3, 2),
4 (3, 3, 750, 6, 3),
5 (4, 5, 1250, 3, 4),
6 (5, 1, 150, 10, 5);

```

Рисунок 4.10 – скрипт по заповненню таблиці «linePD»

Виконати SQL запит(и) у базі даних byteShop.invoice: ⓘ

```

1 INSERT INTO `invoice`(`invoiceId`, `invoiceNumber`, `invoiceDate`, `productId`)
2 VALUES
3 (1, 'INV-001', '2025-01-10', 101),
4 (2, 'INV-002', '2025-01-12', 102),
5 (3, 'INV-003', '2025-01-15', 103),
6 (4, 'INV-004', '2025-01-18', 104),
7 (5, 'INV-005', '2025-01-20', 105);
8 |

```

Рисунок 4.11 – скрипт по заповненню таблиці «invoice»

Виконати SQL запит(и) у базі даних byteShop remainder: ?

```

1 INSERT INTO `remainder` (`remainderId`, `remainderAmount`, `productId`, `storageId`)
2 VALUES
3 (1, 15, 1, 1),
4 (2, 8, 2, 1),
5 (3, 20, 3, 2),
6 (4, 5, 4, 2),
7 (5, 12, 5, 3);
8

```

Рисунок 4.12 – скрипт по заповненню таблиці «remainder»

Виконати SQL запит(и) у базі даних byteShop storage: ?

```

1 INSERT INTO `storage` (`storageId`, `storageNumber`, `storageAddress`)
2 VALUES
3 (1, 'ST-001', 'вул. Хрещатик, 10, Київ'),
4 (2, 'ST-002', 'вул. Січових Стрільців, 5, Львів'),
5 (3, 'ST-003', 'вул. Дерибасівська, 12, Одеса'),
6 (4, 'ST-004', 'просп. Науки, 20, Харків'),
7 (5, 'ST-005', 'вул. Грушевського, 15, Дніпро');
8

```

Рисунок 4.13 – скрипт по заповненню таблиці «storage»

4.1.3 Створення зв'язків між таблицями

Далі необхідно створити такі зв'язки між таблицями: client -> invoice (один до багатьох), invoice -> linePD (один до багатьох), linePD <- product (багато до одного), product -> remainder (один до багатьох) та remainder <- storage (багато до одного). Скрипт заповнення таблиць на рисунках 4.14-4.16 відповідно.

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 ALTER TABLE linePD
2 ADD FOREIGN KEY (invoiceId) REFERENCES invoice (invoiceID);
3
4 ALTER TABLE linePD
5 ADD FOREIGN KEY (productId) REFERENCES product (productId);
6

```

Рисунок 4.14 – скрипт по створенню зв'язків у таблиці «linePD»

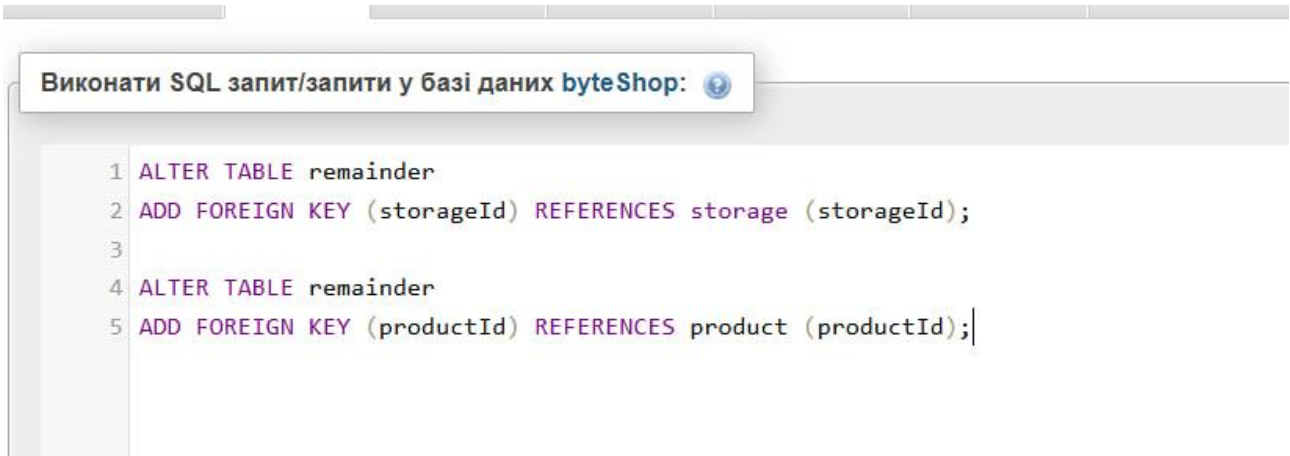


Рисунок 4.15 – скрипт по створенню зв'язків у таблиці «remainder»

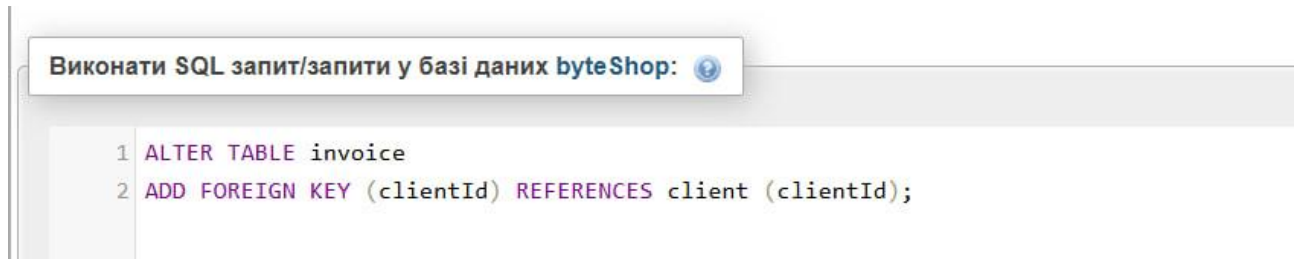


Рисунок 4.16 – скрипт по створенню зв'язків у таблиці «invoice»

Як висновок підрозділу «4.1 Створення базових таблиць бази даних, відповідно до ER-моделі, покривши «бізнес-об'єкти» сайту», додаю фото готових таблиць, тобто створених, заповнених, пов'язаних. Відповідно рисунки 4.17-4.22.

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0002 секунди.)

[SELECT * FROM `client`](#)

☐ Профілювання [\[Порядкове редагування\]](#) [\[Редагувати\]](#) [\[Тлумачити SQL\]](#) [\[Створити PHP код\]](#) [\[Оновити\]](#)

☐ Показати все | Число рядків: 25 | Фільтрувати рядки: | Сортувати за ключем: Жодного

Екстра параметри

	clientId	clientFirstname	clientLastname	clientBirthday	clientPhone	clientEmail
☐ Редагувати ☐ Копіювати ☐ Видалити	1	John	Smith	1990-05-12	+380501234567	john.smith@example.com
☐ Редагувати ☐ Копіювати ☐ Видалити	2	Emily	Brown	1988-11-02	+380671112233	emily.brown@example.com
☐ Редагувати ☐ Копіювати ☐ Видалити	3	Michael	Johnson	1995-07-29	+380931234567	m.johnson@example.com
☐ Редагувати ☐ Копіювати ☐ Видалити	4	Olivia	Davis	2000-03-15	+380991112244	olivia.davis@example.com
☐ Редагувати ☐ Копіювати ☐ Видалити	5	William	Taylor	1985-12-09	+380661234890	william.taylor@example.com

Рисунок 4.17 – готова таблиця «client»

✓ Показано рядки 0 - 11 (всього 12, Запит виконувався 0.0002 секунди.)

`SELECT * FROM `product``

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 | Фільтрувати рядки: | Сортувати за ключем: Жодного

Екстра параметри

	productId	productName	productPrice	productType	productModel	productColor	productAvailability
☐ Редагувати Копіювати Видалити	1	iPhone 13	21350	phone	basic	білий	-
☐ Редагувати Копіювати Видалити	2	Samsung Galaxy Tab S11"	32800	pad	basic	чорний	+
☐ Редагувати Копіювати Видалити	3	Marshall Major V	4500	Pods	basic	чорний	+
☐ Редагувати Копіювати Видалити	4	Garmin Fenix 8 47mm AMOLED	32200	watch	basic	чорний	+
☐ Редагувати Копіювати Видалити	5	GIGABYTE G6 KF 2024 16"	43000	laptop	basic	чорний	-
☐ Редагувати Копіювати Видалити	6	ARTLINE Gaming X47	34000	computer	basic	чорний	+
☐ Редагувати Копіювати Видалити	7	Sony PlayStation 5	37000	game	basic	чорний	+
☐ Редагувати Копіювати Видалити	8	Dyson Airwrap Coanda2x	35000	prettyHealth	basic	чорний	-
☐ Редагувати Копіювати Видалити	9	Roborock Vacuum Cleaner Q8	11000	home	basic	чорний	-
☐ Редагувати Копіювати Видалити	10	Тостер SMEG	6000	kitchen	basic	білий	+
☐ Редагувати Копіювати Видалити	11	б/у Навушники Apple AirPods Max	13000	BU	premium	рожевий	-
☐ Редагувати Копіювати Видалити	12	Бездротова зарядка Belkin 3in1	8000	add	premium	чорний	+

Рисунок 4.18 – готова таблиця «product»

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0002 секунди.)

`SELECT * FROM `linePD``

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 | Фільтрувати рядки: | Сортувати за ключем

Екстра параметри

	linePDId	linePDAmount	linePDSum	productId	invoiceId
☐ Редагувати Копіювати Видалити	1	2	500	1	1
☐ Редагувати Копіювати Видалити	2	1	250	3	2
☐ Редагувати Копіювати Видалити	3	3	750	6	3
☐ Редагувати Копіювати Видалити	4	5	1250	3	4
☐ Редагувати Копіювати Видалити	5	1	150	10	5

Рисунок 4.19 – готова таблиця «linePD»

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0001 секунди.)

```
SELECT * FROM `invoice`
```

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 ▼ Фільтрувати рядки: Сорт

Екстра параметри

			invoiceId	invoiceNumber	invoiceDate	clientId
☐	Редагувати	Копіювати	Видалити	1 INV-001	2025-01-10	1
☐	Редагувати	Копіювати	Видалити	2 INV-002	2025-01-12	2
☐	Редагувати	Копіювати	Видалити	3 INV-003	2025-01-15	3
☐	Редагувати	Копіювати	Видалити	4 INV-004	2025-01-18	4
☐	Редагувати	Копіювати	Видалити	5 INV-005	2025-01-20	5

Рисунок 4.20 – готова таблиця «invoice»

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0002 секунди.)

```
SELECT * FROM `remainder`
```

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 ▼ Фільтрувати рядки: Сортувати

Екстра параметри

				remainderId	remainderAmount	productId	storageld
☐	Редагувати	Копіювати	Видалити	1	15	1	1
☐	Редагувати	Копіювати	Видалити	2	8	2	1
☐	Редагувати	Копіювати	Видалити	3	20	3	2
☐	Редагувати	Копіювати	Видалити	4	5	4	2
☐	Редагувати	Копіювати	Видалити	5	12	5	3

Рисунок 4.21 – готова таблиця «remainder»

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0002 секунди.)

`SELECT * FROM `storage``

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 ▼ | Фільтрувати рядки: | Сортувати з

Екстра параметри

	storageId	storageNumber	storageAddress
☐ Редагувати ☐ Копіювати ☐ Видалити	1	ST-001	вул. Хрещатик, 10, Київ
☐ Редагувати ☐ Копіювати ☐ Видалити	2	ST-002	вул. Січових Стрільців, 5, Львів
☐ Редагувати ☐ Копіювати ☐ Видалити	3	ST-003	вул. Дерибасівська, 12, Одеса
☐ Редагувати ☐ Копіювати ☐ Видалити	4	ST-004	просп. Науки, 20, Харків
☐ Редагувати ☐ Копіювати ☐ Видалити	5	ST-005	вул. Грушевського, 15, Дніпро

Рисунок 4.22 – готова таблиця «storage»

4.2 Створення таблиць для майбутньої ролівої моделі

Для підготовки до ролівої моделі необхідно створити таблиці role (для ролів) та clientRole (для того, щоб пов'язати таблиці client та role), заповнити їх та пов'язати між собою.

4.2.1 Створення таблиць для ролівої моделі

Скрипт для створення таблиць role та clientRole поданий на рисунках 4.23-4.24 відповідно.

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 CREATE TABLE role (
2     roleId INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
3     roleName VARCHAR(50) NOT NULL,
4     roleDescription VARCHAR(100) NOT NULL
5 );

```

Рисунок 4.23 – скрипт по створенню таблиці «role»

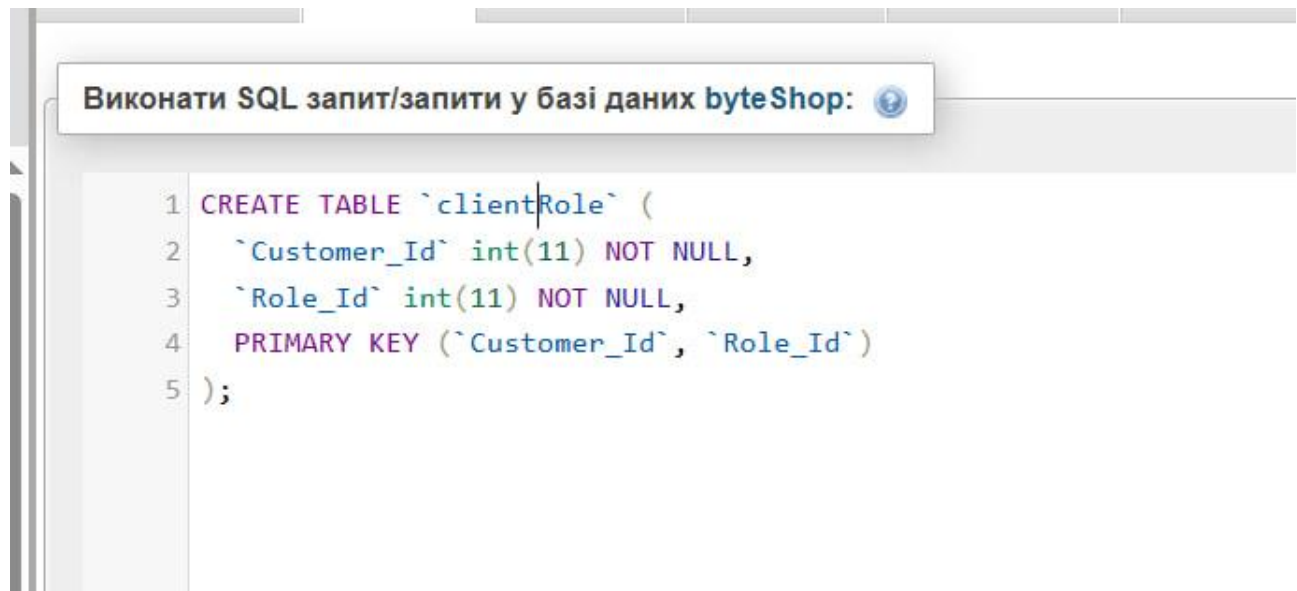


Рисунок 4.24 – скрипт по створенню таблиці «clientRole»

4.2.2 Заповнення таблиць для рольової моделі

Далі необхідно заповнити такі таблиці: role та clientRole. Скрипт заповнення таблиць на рисунках 4.25-4.26 відповідно.

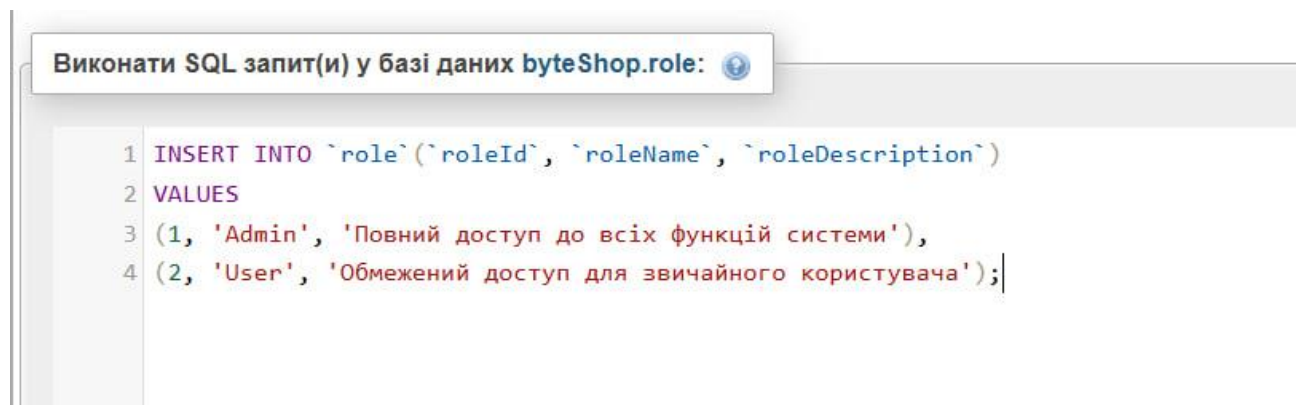


Рисунок 4.25 – скрипт по заповненню таблиці «role»

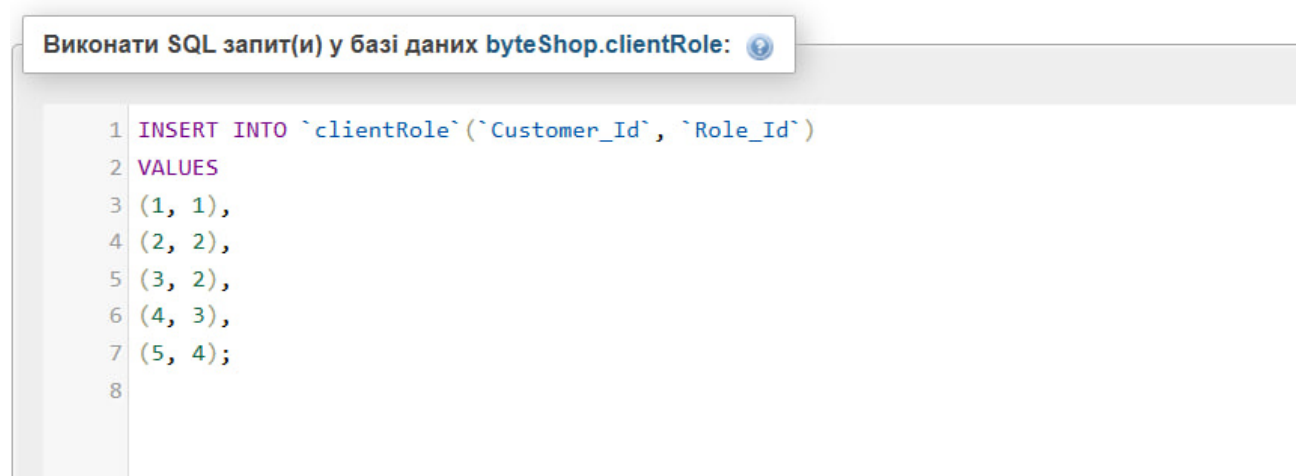


Рисунок 4.26 – скрипт по заповненню таблиці «clientRole»

4.2.3 Створення зв'язків між таблицями для рольової моделі

Далі необхідно створити такі зв'язки між таблицями: client -> clientRole (один до багатьох) та clientRole <- role (багато до одного). Скрипт заповнення таблиць на

рисунках 4.27 відповідно.



Рисунок 4.27 – скрипт по створенню зв'язків у таблиці «clientRole»

Як висновок підрозділу «4.2 Створення таблиць для майбутньої рольової моделі», додаю фото готових таблиць, тобто створених, заповнених, пов'язаних. Відповідно рисунки 4.28-4.29.

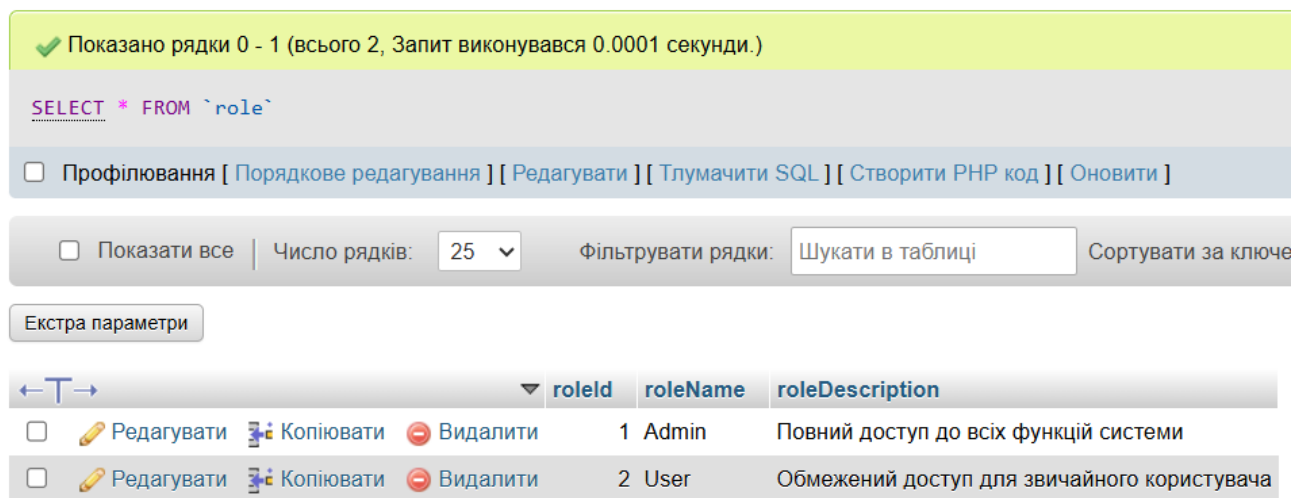


Рисунок 4.28 – готова таблиця «Role»

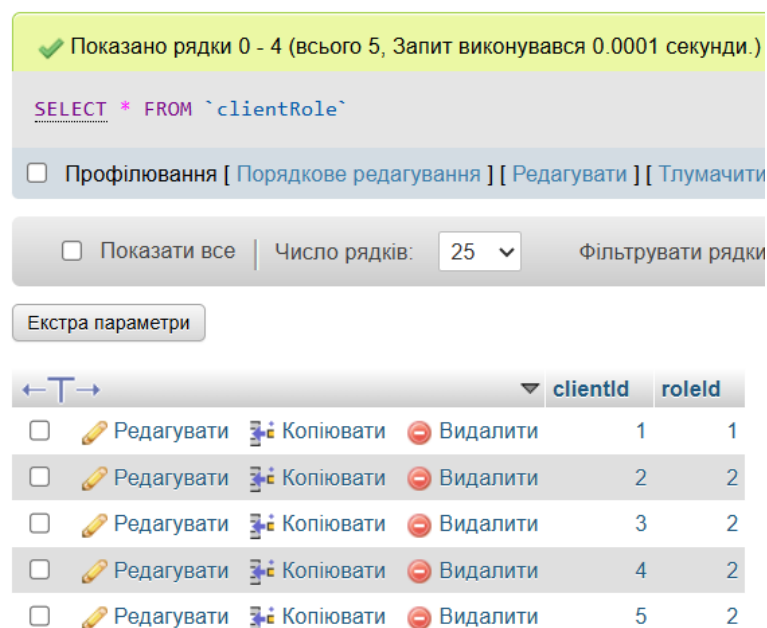


Рисунок 4.29 – готова таблиця «UserRole»

4.3 Визначені ключі та обмеження бази даних

4.3.1 Primary key

Первинні ключі встановлено на таких таблицях:

- client
- clientRole (складений ключ: clientId, roleId)
- invoice
- linePD
- product
- remainder
- role
- storage

Первинний ключ забезпечує унікальність рядків у таблиці та використовується для ідентифікації кожного запису.

4.3.2 Foreign key

Зовнішні ключі встановлюються для зв'язку таблиць між собою.

У моїй базі даних 7 зовнішніх ключів:

- Таблиця clientRole:
 - clientId пов'язаний з client.clientId
 - roleId пов'язаний з role.roleId
- Таблиця invoice:
 - clientId пов'язаний з client.clientId
- Таблиця linePD:
 - productId пов'язаний з product.productId
 - invoiceId пов'язаний з invoice.invoiceId
- Таблиця remainder:
 - productId пов'язаний з product.productId
 - storageId пов'язаний з storage.storageId

Це забезпечує коректне відображення зв'язків «один-до-багатьох» між таблицями.

4.3.3 Unique

Унікальні обмеження використовуються там, де значення не повинні повторюватися. У моїй базі даних це застосовано до таких полів:

- client.clientPhone
- client.clientEmail
- invoice.invoiceNumber
- product.productName

Це запобігає дублюванню важливих даних.

4.3.4 Check

CHECK використовується для контролю діапазонів значень. Для забезпечення коректності значень у базі даних можуть бути застосовані обмеження:

- linePD.linePDAmount > 0
- linePD.linePDSum >= 0
- product.productPrice > 0
- remainder.remainderAmount >= 0

Такі обмеження запобігають введенню негативних або некоректних значень.

4.3.5 каскадні правила (ON DELETE/UPDATE)

Для підтримки узгодженості даних застосовуються такі правила:

- ON DELETE CASCADE — при видаленні запису автоматично видаляються залежні рядки (наприклад, записи у linePD при видаленні invoice).
- ON DELETE RESTRICT — заборона видалення запису, якщо на нього є посилання (наприклад, клієнта не можна видалити, якщо він має накладні).
- ON UPDATE CASCADE — при зміні ключа автоматично оновлюються всі зв'язані записи.

Це забезпечує підтримку цілісності бази даних без ручного втручання.

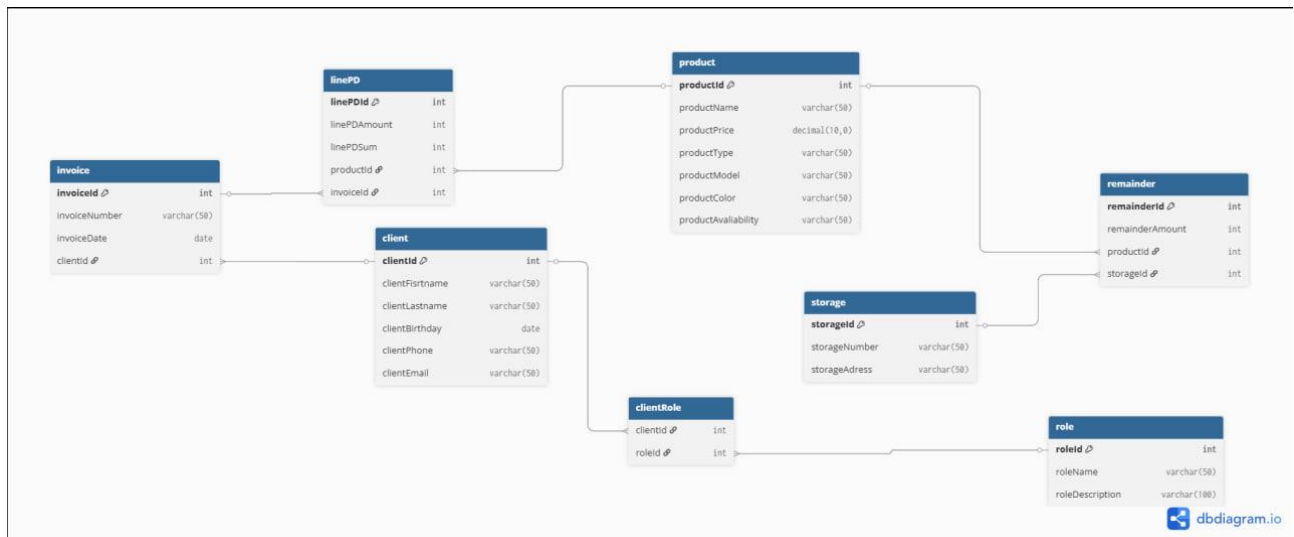
4.3.6 Індокси

Індоксация використовується для прискорення пошуку та фільтрації:

- Автоматично створюються індокси на всіх первинних і зовнішніх ключах.
- Додаткові індокси для полів, які часто використовуються у запитах:
 - client.clientEmail
 - product.productType
 - invoice.invoiceDate

Це покращує продуктивність при виконанні SELECT-запитів, сортуванні та фільтруванні.

4.4 Схема зв'язків та сутностей бази даних



4.5 Тестові SELECT-запити та оператори

4.5.1 Оператор №1: Показує кількість інвойсів та загальну суму по кожному клієнту.

Виконати SQL запит/запити у базі даних byteShop: ?

```

1 SELECT c.clientId, c.clientFirstname, c.clientLastname,
2       COUNT(i.invoiceId) AS totalInvoices,
3       IFNULL(SUM(lp.linePDSum), 0) AS totalSpent
4 FROM client c
5 LEFT JOIN invoice i ON c.clientId = i.clientId
6 LEFT JOIN linePD lp ON i.invoiceId = lp.invoiceId
7 GROUP BY c.clientId, c.clientFirstname, c.clientLastname
8 ORDER BY totalSpent DESC;
  
```

Рисунок 30 – створення оператора №1

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0003 секунди.)

```
SELECT c.clientId, c.clientFirstname, c.clientLastname, COUNT(i.invoiceId) AS totalInvoices, IFNULL(SUM(lp.linePDSum), 0) AS totalSpent FROM client c LEFT JOIN invoice i ON c.clientId = i.clientId LEFT JOIN linePD lp ON i.invoiceId = lp.invoiceId GROUP BY c.clientId, c.clientFirstname, c.clientLastname ORDER BY totalSpent DESC;
```

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

☐ Показати все | Число рядків: 25 | Фільтрувати рядки:

Екстра параметри

clientId	clientFirstname	clientLastname	totalInvoices	totalSpent
4	Olivia	Davis	1	1250
3	Michael	Johnson	1	750
1	John	Smith	1	500
2	Emily	Brown	1	250
5	William	Taylor	1	150

Рисунок 31 – відтворення оператора №1

4.5.2 Оператор №2: Визначає 5 найпопулярніших продуктів за кількістю продажів.

Виконати SQL запит/запити у базі даних byteShop: [?](#)

```
1 SELECT p.productId, p.productName, SUM(lp.linePDAmount) AS totalSold
2 FROM product p
3 JOIN linePD lp ON p.productId = lp.productId
4 GROUP BY p.productId, p.productName
5 ORDER BY totalSold DESC
6 LIMIT 5;
7 |
```

Рисунок 32 – Створення оператора №2

✓ Показано рядки 0 - 3 (всього 4, Запит виконувався 0.0003 секунди.)

```
SELECT p.productId, p.productName, SUM(lp.linePDAmount) AS totalSold FROM product p JOIN linePD lp ON p.productId = lp.productId GROUP BY p.productId, p.productName ORDER BY totalSold DESC LIMIT 5;
```

☐ Профілювання [[Порядкове редагування](#)] [[Редагувати](#)] [[Тлумачити SQL](#)] [[Створити PHP код](#)] [[Оновити](#)]

Екстра параметри

productId	productName	totalSold
3	Marshall Major V	6
6	ARTLINE Gaming X47	3
1	iPhone 13	2
10	Тостер SMEG	1

Рисунок 33 – Виконання оператора №2

4.5.3 Оператор №3: Показує всі продукти, їх кількість на складі та адресу складу.

Виконати SQL запит/запити у базі даних byteShop: [?](#)

```
1 SELECT p.productName, s.storageNumber, s.storageAddress, r.remainderAmount
2 FROM remainder r
3 JOIN product p ON r.productId = p.productId
4 JOIN storage s ON r.storageId = s.storageId
5 WHERE r.remainderAmount > 0
6 ORDER BY s.storageNumber, p.productName;
7
```

Рисунок 34 – створення оператора №3

✓ Показано рядки 0 - 4 (всього 5, Запит виконувався 0.0004 секунди.) [storageNumber: ST-001... - ST-003...] [productName: IPHONE 13... - GIGABYTE G6 KF 2024 16"...]

```
SELECT p.productName, s.storageNumber, s.storageAddress, r.remainderAmount FROM remainder r JOIN product p ON r.productId = p.productId JOIN storage s ON r.storageId = s.storageId WHERE r.remainderAmount > 0 ORDER BY s.storageNumber, p.productName;
```

☐ Профілювання [Порядкове редагування] [Редагувати] [Тлумачити SQL] [Створити PHP код] [Оновити]

☐ Показати все | Число рядків: 25 | Фільтрувати рядки: Шукати в таблиці

Екстра параметри

productName	storageNumber	storageAddress	remainderAmount
iPhone 13	ST-001	вул. Хрещатик, 10, Київ	15
Samsung Galaxy Tab S11"	ST-001	вул. Хрещатик, 10, Київ	8
Garmin Fenix 8 47mm AMOLED	ST-002	вул. Січових Стрільців, 5, Львів	5
Marshall Major V	ST-002	вул. Січових Стрільців, 5, Львів	20
GIGABYTE G6 KF 2024 16"	ST-003	вул. Дерибасівська, 12, Одеса	12

Рисунок 35 – відтворення оператора №3

4.6 Репозиторій на GitHub

Посилання на репозиторій на GitHub:

<https://github.com/SandraBSV/IS-44-appWEB-BaryshevaAnastasiia-FIOT-2025>

4.7 Висновки до розділу

У ході виконання практичної роботи було спроектовано та реалізовано реляційну базу даних відповідно до ER-діаграми веб-застосунку. Під час проєктування були визначені основні сутності предметної області, встановлені логічні зв'язки між ними та проведено нормалізацію даних для усунення надлишковості й підвищення цілісності збереженої інформації.

У структурі бази даних реалізовано модель користувачів, ролей і прав доступу, що забезпечує розмежування повноважень на рівні схеми. Для всіх таблиць визначено первинні ключі, а також створено зовнішні ключі для зв'язку між сутностями. Додатково застосовано обмеження типів UNIQUE та CHECK, а також передбачено каскадні правила ON DELETE/UPDATE, що дозволяють підтримувати цілісність даних при зміні або видаленні пов'язаних записів.

Для оптимізації продуктивності були створені індекси на полях, які часто беруть участь у фільтрації та приєднаннях. Також підготовлено seed-дані, що демонструють працездатність зв'язків між таблицями, включаючи користувачів, ролі та доменні записи. Виконано тестові SELECT-запити з використанням JOIN, фільтрації та агрегування, результати яких підтвердили коректність реалізованої структури.

Таким чином, поставлені завдання практичної роботи виконано в повному обсязі: сформовано фізичну схему реляційної БД, реалізовано модель доступу на

основі ролей, визначено всі необхідні обмеження та забезпечено початкове наповнення для подальшої інтеграції з веб-застосунком.

4.8 Контрольні запитання

1. Яка мета створення реляційної бази даних для веб-застосунку?

Метою створення реляційної бази даних є структуроване збереження інформації веб-застосунку, забезпечення цілісності даних, їх зв'язності, можливості швидкого доступу, а також розмежування прав користувачів.

2. Що таке ER-діаграма і яку інформацію вона відображає?

ER-діаграма — це графічна модель, яка відображає сутності предметної області, їх атрибути та логічні зв'язки між ними.

3. Як відбувається перехід від ER-моделі до реляційної схеми бази даних?

Перехід здійснюється шляхом перетворення кожної сутності в таблицю, атрибутів — у стовпці, а зв'язків — у зовнішні ключі та проміжні таблиці для зв'язків M:N.

4. Які сутності та зв'язки є основними для вашого веб-застосунку?

Основні сутності: користувачі, ролі, товари, склади, залишки, накладні, рядки накладних. Основні зв'язки: користувач ↔ накладні, накладна ↔ рядки, товар ↔ залишки, користувач ↔ роль.

5. Які типи зв'язків використовуються між таблицями (1:1, 1:N, M:N) і як вони реалізуються в SQL?

У схемі застосовуються:

- 1:N — реалізується за допомогою FOREIGN KEY (Customer → Invoice)
- M:N — реалізується проміжною таблицею (UserRoles)
- 1:1 — окремими таблицями з унікальними зовнішніми ключами.

6. Які ключові поля ви визначили у своїй базі даних та яку роль вони виконують?

PRIMARY KEY — забезпечують унікальність записів.

FOREIGN KEY — підтримують логічні зв'язки між таблицями.

У проміжних таблицях застосовані складені ключі для уникнення дублікатів.

7. Які види обмежень цілісності застосовані у вашій БД (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK)?

- PRIMARY KEY — унікальність рядків;
- FOREIGN KEY — логічна зв'язаність даних;
- UNIQUE — заборона дублювання критичних полів (Email);
- CHECK — перевірка допустимих значень (кількість ≥ 0).

8. Для чого використовується каскадне оновлення чи видалення (ON DELETE/UPDATE CASCADE) і де його доцільно застосовувати?

Каскадні дії забезпечують автоматичне оновлення або видалення пов'язаних даних, що запобігає «висячим» записам. Доцільно застосовувати у зв'язках 1:N та M:N, коли дочірні записи не мають сенсу без батьківських.

9. Які таблиці забезпечують зберігання інформації про користувачів та ролі?

Таблиці Customer, Role, UserRole.

10. Як реалізується зв'язок між користувачами та ролями у вашій схемі?

Через проміжну таблицю UserRole, яка зв'язує Customer_Id та Role_Id.

11. Яке призначення таблиці UserRoles та чому вона має складений ключ?

UserRole визначає, яку роль має кожен користувач. Складений ключ запобігає дублюванню комбінації (користувач, роль).

12. Які правила безпеки слід враховувати при зберіганні паролів користувачів у базі даних?

- паролі зберігати у вигляді хеша;
- використовувати соль;
- заборонено зберігати у відкритому вигляді;
- застосовувати сучасні алгоритми (bcrypt, Argon2).

13. Як у вашій БД забезпечується розмежування прав доступу між звичайним користувачем та адміністратором?

Це реалізується через ролі, таблицю UserRole, та логіку перевірки доступу у застосунку відповідно до ролі користувача.

14. Які службові сутності (наприклад, AuditLogs, Sessions) можуть бути додані та з якою метою?

- AuditLogs — фіксація змін даних;
- Sessions/RefreshTokens — аутентифікація та керування сесіями.

Вони підвищують безпеку та контроль стану системи.

15. У чому полягає нормалізація бази даних і яку нормальну форму має ваша схема?

Нормалізація — це процес усунення надлишкових даних та аномалій оновлення.

Схема відповідає 3НФ, оскільки всі неключові атрибути залежать лише від первинного ключа.

16. Які переваги дає створення індексів і для яких полів вони доцільні?

Індекси прискорюють пошук, фільтрацію та приєднання таблиць. Доцільні для:

- зовнішніх ключів,
- полів, що часто зустрічаються в WHERE,
- унікальних полів (Email).

17. Яким чином ви перевіряли коректність зв'язків між таблицями (приклади SELECT-запитів)?

За допомогою JOIN-запитів із вибіркою пов'язаних даних, фільтрами та агрегатами, що дозволяє підтвердити правильність роботи FOREIGN KEY.

18. Які початкові дані (seed-дані) ви додали до бази, і яку мету вони виконують?

Додані тестові записи користувачів, ролей, товарів, складів, накладних.

Вони демонструють працездатність зв'язків та забезпечують тестування.

19. Як організовано структуру SQL-файлів у вашому репозиторії (schema, seed, docs)?

- schema.sql — створення таблиць, ключів, обмежень;
- seed.sql — початкове наповнення даними;
- docs.sql або README — пояснення структури, опис сутностей.

Це спрощує підтримку та читабельність проєкту.

5 РОЗГОРТАННЯ БЕКЕНД-СЕРВЕРА (ПРАКТИЧНА РОБОТА 5)

Мета роботи:

Мета роботи набути навичок створення та налаштування бекенд-сервера, який надає REST API для підключення фронтенду до реляційної бази даних.

5.1 Опис структури серверної частини

Серверну логіку проєкту створено на основі Node.js із застосуванням фреймворку Express, який дозволяє реалізувати REST-орієнтований веб-сервер, а також бібліотеки mysql2 для роботи з MySQL-базою даних.

Під час розробки та запуску серверної частини використовується локальне середовище Open Server Panel, що забезпечує роботу MySQL, керування параметрами середовища та конфігурацію локального хостингу.

Архітектура серверу має модульну структуру та включає кілька основних частин: ініціалізацію застосунку, налаштування проміжних обробників (middleware), встановлення з'єднання з базою даних, визначення REST-маршрутів і запуск серверного процесу.

5.1.1 Основні компоненти серверної частини:

1. Сервер (Express app)

Основним елементом є серверний модуль, що приймає вхідні запити на визначеному порту (3000) та керує взаємодією всіх складових серверної системи. Його запуск здійснюється за допомогою методу `app.listen()`, який активує сервер і надає можливість обробляти HTTP-запити.

2. Middleware

Проміжні обробники виконують підготовку запитів перед тим, як вони потрапляють до маршрутів. Зокрема, `express.json()` відповідає за автоматичний розбір JSON-даних у тілі запиту, що дає змогу серверу коректно працювати з інформацією, надісланою клієнтом. Також використовується `cors()`, який налаштовує правила Cross-Origin Resource Sharing та дозволяє фронтенду, розміщеному на іншому домені або порту, взаємодіяти з сервером.

Код реалізації констант, middleware та запуску сервера:

```

const mysql = require('mysql2');
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());
const PORT = 3000;
app.use(express.static('public'));
app.listen(PORT, () => console.log(`Сервер запущено на порту ${PORT}`));

```

3. Підключення до бази даних (Database Connection)

Сервер встановлює підключення до бази даних MySQL через метод `mysql.createConnection()`. Під час цього процесу перевіряється, чи не виникли помилки, і якщо з'єднання не вдалося створити, робота сервера примусово завершується (`process.exit(1)`), щоб запобігти некоректному функціонуванню застосунку. Після успішного підключення сервер отримує можливість виконувати SQL-запити, необхідні для взаємодії з таблицею `Product`.

4. Маршрути (Routes)

Сервер реалізує маршрути для обробки CRUD-операцій і запитів пошуку:

- `GET /search` — пошук продуктів за назвою або описом.
- `GET /product/id/:id` — отримання конкретного виробу за унікальним ідентифікатором.
- `GET /products/:type` — отримання списку виробів певного типу.
- `POST /products/:type` — додавання нового виробу в базу даних.
- `PUT /products/:id` — оновлення існуючого виробу.
- `DELETE /products/:id` — видалення виробу з бази даних.

Кожен маршрут виконує SQL-запити через підключення до бази даних, обробляє результати та повертає клієнту відповідь у форматі JSON.

5. Обробка помилок (Error Handling)

Сервер реалізує обробку помилок на різних етапах роботи, зокрема контролює коректність підключення до бази даних, відстежує можливі помилки під час виконання SQL-запитів і перевіряє наявність потрібних даних перед формуванням відповіді. У разі виникнення проблем сервер повертає відповідні HTTP-коди, такі як 500 для внутрішньої помилки або 404, якщо необхідні дані не знайдено, а також надсилає клієнту інформативне JSON-повідомлення.

5.1.2 Принцип взаємодії компонентів серверної частини:

1. Клієнт (фронтенд) надсилає HTTP-запит (GET, POST, PUT, DELETE) на сервер.
2. Middleware обробляє запит, перевіряє дані та дозволяє або блокує запит за правилами CORS.
3. Сервер передає запит відповідному маршруту (Route), який визначає логіку обробки.
4. Маршрут виконує SQL-запит до бази даних та отримує результат.
5. Результат обробляється:
 - у разі успіху повертається JSON-об'єкт з даними;
 - у разі помилки повертається відповідний статус і повідомлення про помилку.
6. Клієнт отримує відповідь і відображає дані або повідомлення про помилку.

5.2 Підключення до реляційної бази даних

У проєкті для зберігання та обробки даних використовується **реляційна база даних MySQL**, а для взаємодії з нею на сервері застосовується модуль **mysql2** для Node.js. Це дозволяє виконувати SQL-запити безпосередньо з серверного коду, отримувати результати та обробляти їх у форматі JavaScript-об'єктів.

5.2.1 Код, де реалізоване підключення до реляційної бази:

1. `mysql.createConnection()` створює об'єкт підключення до бази даних, де вказуються:

- `host` — адреса сервера бази даних;
- `port` — порт для підключення (3306 за замовчуванням у MySQL);
- `user` та `password` — облікові дані для авторизації;
- `database` — назва бази даних для роботи.

2. connection.connect() встановлює реальне з'єднання з базою даних і перевіряє, чи немає помилок. У разі невдалого підключення сервер припиняє роботу (process.exit(1)), щоб уникнути некоректної роботи програми.

Код, де реалізоване підключення реляційної бази даних:

```
const connection = mysql.createConnection({
  host: '127.0.1.15',
  port: 3306,
  user: 'root',
  password: '',
  database: 'ByteShop'
});
connection.connect(err => {
  if (err) {
    console.error("Помилка підключення до бази даних:", err);
    process.exit(1);
  } else {
    console.log("База даних підключена успішно");
  }
});
```

5.2.2 Приклад роботи з даними:

Приклад №1: Пошук виробів (SELECT + LIKE)

Для здійснення пошуку товарів на сервері створено маршрут /search, який отримує з клієнта параметр query. На його основі формується SQL-запит із використанням оператора LIKE, що дає змогу знаходити товари за частковими збігами в назві чи описі. Значення параметрів передаються у масиві, що підвищує безпеку та запобігає SQL-ін'єкціям. Після виконання запиту сервер опрацьовує отримані дані й повертає їх клієнтові у форматі JSON.

Код:

```
app.get('/search', (req, res) => {
  const query = req.query.query || "";
  const sql = `
    SELECT * FROM Product
```

WHERE Product_Name LIKE ? OR Product_Description LIKE ?

```
`;  
const likeQuery = `%${query}%`;   
connection.query(sql, [likeQuery, likeQuery], (err, results) => {  
  if (err) {  
    console.error('Помилка SQL:', err);  
    return res.status(500).json({ error: 'Помилка сервера' });  
  }  
  res.json(results);  
});  
});
```

Приклад №2: Отримання виробів за ID (SELECT + WHERE)

Для отримання інформації про певний товар за його унікальним ідентифікатором використовується маршрут /product/id/:id. Сервер бере значення id з URL та виконує SQL-запит із використанням умови WHERE, щоб знайти відповідний запис у базі даних. Якщо товар не виявлено, сервер повертає клієнту повідомлення про помилку з кодом 404. У випадку успішного пошуку потрібні дані передаються у відповідь у форматі JSON-об'єкта.

Код:

```
app.get('/product/id/:id', (req, res) => {  
  const id = req.params.id;  
  const sql = "SELECT * FROM Product WHERE Product_Id=?";  
  connection.query(sql, [id], (err, results) => {  
    if (err) return res.status(500).json({ error: err });  
    if (results.length === 0) return res.status(404).json({ error: "Not found" });  
    res.json(results[0]);  
  });  
});
```

Такий підхід дозволяє забезпечити прямий контроль над SQL-запитами, виконувати параметризовані запити для безпеки та швидко інтегрувати нові маршрути для пошуку та отримання даних. Використання JSON-формату відповіді спрощує подальшу обробку даних на фронтенді та дозволяє легко реалізувати інтерактивні елементи інтерфейсу.

5.3 Інтеграція фронтенду з серверною частиною

Фронтенд проєкту реалізовано на чистому JavaScript у поєднанні з HTML і CSS, що дозволяє безпосередньо взаємодіяти з серверними маршрутами через API. Інтеграція здійснюється за допомогою fetch-запитів, які отримують дані у форматі JSON і відображають їх на сторінках користувацького інтерфейсу.

5.3.1 Пошук виробів на головній сторінці

На головній сторінці реалізовано механізм пошуку товарів за назвою або описом. У файлі main.js визначено функцію performSearch(), яка зчитує значення з поля введення, формує запит до маршруту /search на сервері та виконує його через fetch. Отриману відповідь функція передає до displaySearchResults(), яка динамічно створює DOM-елементи для показу знайдених товарів.

Такий підхід дає змогу користувачу вводити запит у пошукове поле й миттєво отримувати оновлені результати з сервера. Виведення реалізовано у вигляді карток із назвою продукту, його типом, ціною та описом, що робить інтерфейс більш зручним і наочним.

Код, де реалізований пошук:

```
function performSearch() {  
  const query = searchInput.value.trim();  
  if (!query) return;  
  fetch(`${API_BASE_URL}/search?query=${encodeURIComponent(query)}`)  
    .then(res => {  
      if (!res.ok) throw new Error(`HTTP error! Status: ${res.status}`);  
      return res.json();  
    })  
    .then(data => displaySearchResults(data))  
    .catch(err => {  
      console.error('Помилка пошуку:', err);  
      searchResultsContainer.innerHTML = '<p>Помилка при пошуку</p>';  
    });  
}
```

Код, де реалізоване відображення пошуку:

```
function displaySearchResults(results) {
  searchResultsContainer.innerHTML = "";
  if (results.length === 0) {
    searchResultsContainer.innerHTML = '<p>Нічого не знайдено</p>';
    return;
  }
  results.forEach(item => {
    const resultItem = document.createElement('div');
    resultItem.classList.add('search-result-item');
    resultItem.innerHTML = `
      <h3>${item.Product_Name}</h3>
      <h4>Тип: ${item.Product_Type}</h4>
      <p>Ціна: ${item.Product_Price} грн</p>
      <p>${item.Product_Description}</p>`;
    searchResultsContainer.appendChild(resultItem);
  });
}
```

Приклад виконання пошуку, наприклад, знайдемо браслет:



Рисунок 5.1 — результат пошуку на головній сторінці за словом «зарядка»

5.3.2 Завантаження виробів за типом

Для сторінок категорій (pageType.js) використовується маршрут /products/:type, який повертає всі вироби певного типу. Після завантаження сторінки фронтенд виконує fetch із зазначенням типу продукту, отримує масив об'єктів і викликає функцію renderProducts(), яка динамічно формує HTML-картки для кожного товару.

Кожна картка містить зображення, назву, ціну та посилання на сторінку конкретного виробу. Також реалізовано інтерактивні фільтри за ціною, кольором та наявністю, які працюють повністю на стороні клієнта, але оперують даними, отриманими з сервера.

Код, де реалізоване завантаження виробів за типом:

```
// --- Завантаження товару ---
try {
  const res = await
fetch(`http://localhost:3000/products/${encodeURIComponent(PRODUCT_TYPE)}`);
  if (!res.ok) throw new Error(`Не вдалося завантажити ${PRODUCT_TYPE}`);
  products = await res.json();
  if (products.length === 0) {
    container.innerHTML = `<p>${PRODUCT_TYPE} не знайдено</p>`;
    return;
  }
  renderProducts(products);
} catch (err) {
  console.error(`Помилка завантаження ${PRODUCT_TYPE}:`, err);
  container.innerHTML = `<p>Сталася помилка при завантаженні
${PRODUCT_TYPE}</p>`;
}
```

Приклад завантаження виробів за типом, наприклад сторінка браслетів:



Рисунок 5.2 – результат завантаження виробів за типом на сторінці смартфонів

5.3.3. Завантаження даних окремого товару

На сторінці окремого виробу (pageProduct.js) здійснюється виклик API `/product/id/:id`, де `id` отримується з URL. Фронтенд запитує дані конкретного продукту і відображає їх у детальному вигляді, включаючи карусель фотографій:

Зображення продукту завантажуються динамічно з папки на сервері, перевіряючи наявність кожного фото через асинхронні запити `fetch`. Користувач може перемикає фото за допомогою кнопок “Prev” і “Next”, що робить інтерактивне відображення даних зручним і наочним.:

Код, де реалізоване завантаження виробів за ID:

```
// Отримуємо дані з сервера
let product;
try {
  const res = await fetch(`http://localhost:3000/product/id/${productId}`);
  product = await res.json();
} catch (e) {
  console.error("Помилка завантаження продукту", e);
  return;
}
```

Приклад завантаження виробів за ID, наприкладі, виробу комплекту Джинс:

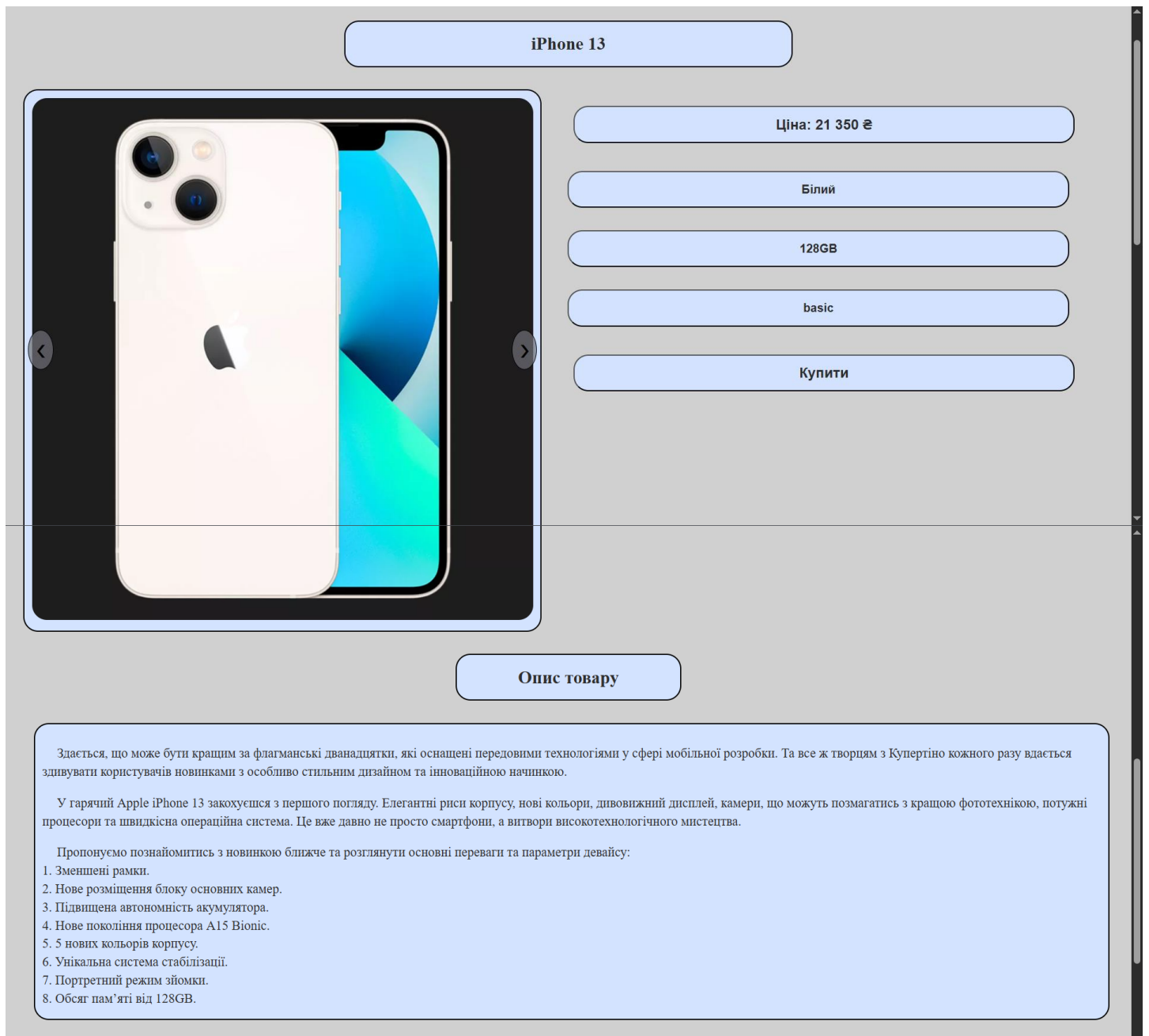


Рисунок 5.3 – результат завантаження товару смартфон за ID

5.4 CRUD-операції у серверній частині

CRUD — це набір базових операцій для роботи з даними у базі:

- C — Create (створення)
- R — Read (читання)
- U — Update (оновлення)
- D — Delete (видалення)

На сервері вони реалізуються через HTTP-операції:

- POST → Create

- GET → Read
- PUT → Update
- DELETE → Delete

5.4.1 Операція GET/READ

Операція GET використовується для читання даних, а також для отримання даних

Це запит, що повертає список товарів за категорією. Сервер отримує тип (braslets, jgut, тощо) через параметр :type, формує SQL-запит та повертає масив продуктів у форматі JSON.

Код, де реалізований GET для адмін-панелі:

// GET: отримати товар

```
app.get('/products/:type', (req, res) => {
  const type = req.params.type;
  const sql = "SELECT * FROM Product WHERE Product_Type = ?";
  connection.query(sql, [type], (err, results) => {
    if (err) {
      console.error('SQL помилка:', err);
      return res.status(500).json({ error: 'Помилка сервера' });
    }
    res.json(results);
  });
});
```

Приклад, де показано як працює GET:

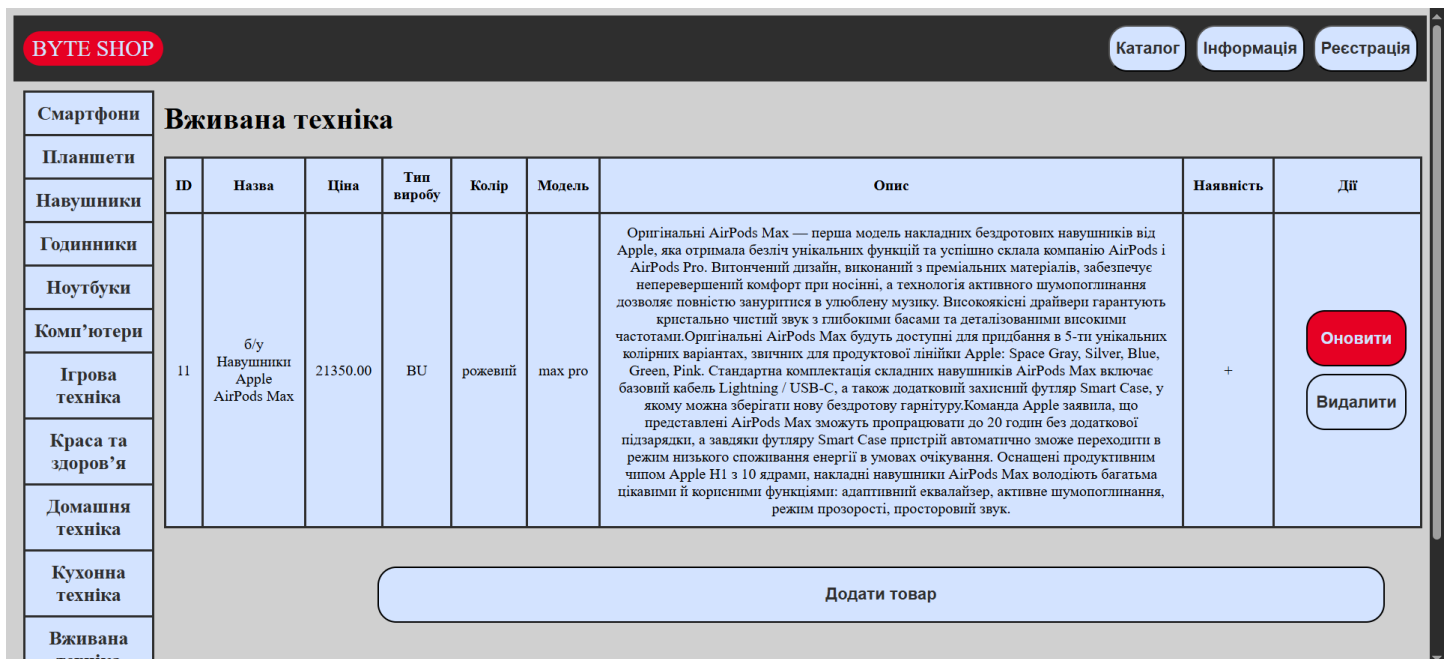


Рисунок 5.4 – результат операції GET

5.4.1 Операція POST/CREATE

Операція **POST** використовується для додавання нового виробу у базу даних. Сервер отримує дані з тіла запиту, формує SQL-команду INSERT і створює новий запис.

Код, де реалізований POST для адмін-панелі:

// POST: додати новий виріб

```
app.post('/products/:type', (req, res) => {
  const type = req.params.type;
  const { Product_Name, Product_Price, Product_Color, Product_Model,
Product_Description, Product_Availability } = req.body;
  const sql = `INSERT INTO Product
(Product_Name, Product_Price, Product_Type, Product_Color, Product_Model,
Product_Description, Product_Availability)
VALUES (?, ?, ?, ?, ?, ?, ?)`;
  connection.query(sql,
[Product_Name, Product_Price, type, Product_Color, Product_Model,
Product_Description, Product_Availability],
(err, result) => {
    if (err) return res.status(500).json({ error: 'Помилка сервера' });
```

```

res.status(201).json({ Product_Id: result.insertId, ...req.body });
}
);
});

```

Приклад, де показано як працює POST:

Додати новий продукт: BU

Назва:

Ціна:

Колір:

Модель:

Опис:

Наявність:

Рисунок 5.5 – Заповнення форми додавання товару для виконання операції POST

Додати товар

ID	Назва	Ціна	Тип виробу	Колір	Модель	Опис	Наявність	Дії
11	б/у Наушники Apple AirPods Max	21350.00	BU	рожевий	max pro	Оригінальні AirPods Max — перша модель накладних бездротових навушників від Apple, яка отримала безліч унікальних функцій та успішно склала компанію AirPods і AirPods Pro. Витончений дизайн, виконаний з преміальних матеріалів, забезпечує неперевершений комфорт при носінні, а технологія активного шумопоглинання дозволяє повністю зануритися в улюблену музику. Високоякісні драйвери гарантують кристально чистий звук з глибокими басами та деталізованими високими частотами. Оригінальні AirPods Max будуть доступні для придбання в 5-ти унікальних кольорних варіантах, звичних для продуктової лінійки Apple: Space Gray, Silver, Blue, Green, Pink. Стандартна комплектація складних навушників AirPods Max включає базовий кабель Lightning / USB-C, а також додатковий захисний футляр Smart Case, у якому можна зберігати нову бездротову гарнітуру. Команда Apple заявила, що представлені AirPods Max зможуть пропрацювати до 20 годин без додаткової підзарядки, а завдяки футляру Smart Case пристрій автоматично зможе переходити в режим низького споживання енергії в умовах очікування. Оснащені продуктивним чипом Apple H1 з 10 ядрами, накладні навушники AirPods Max володіють багатьма цікавими й корисними функціями: адаптивний еквайзер, активне шумопоглинання, режим прозорості, просторовий звук.	+	Оновити Видалити
13	б/у	250.00	BU					Оновити Видалити

Рисунок 5.6 – результат операції POST

5.4.3 Операція PUT/UPDATE

Операція PUT призначена для редагування даних існуючого виробу.

Сервер:

1. отримує ID продукту з параметра URL,
2. отримує нові значення з тіла запиту,
3. оновлює відповідний запис у базі.

Код, де реалізований POST для адмін-панелі:

// PUT: оновити товар

```
app.put('/products/:id', (req, res) => {
  const id = req.params.id;
  const { Product_Name, Product_Price, Product_Type, Product_Color, Product_Model,
Product_Description, Product_Availability } = req.body;
  const sql = `
    UPDATE Product SET
      Product_Name=?, Product_Price=?, Product_Type=?, Product_Color=?,
Product_Model=?, Product_Description=?, Product_Availability=?
    WHERE Product_Id=?`;
  connection.query(sql,
    [Product_Name, Product_Price, Product_Type, Product_Color, Product_Model,
Product_Description, Product_Availability, id],
    (err, result) => {
      if (err) return res.status(500).json({ error: err });
      if (result.affectedRows === 0) return res.status(404).json({ error: 'Product not found'
});
      res.json({ message: 'Product updated' });
    }
  );
});
```

Приклад, де показано як працює PUT:

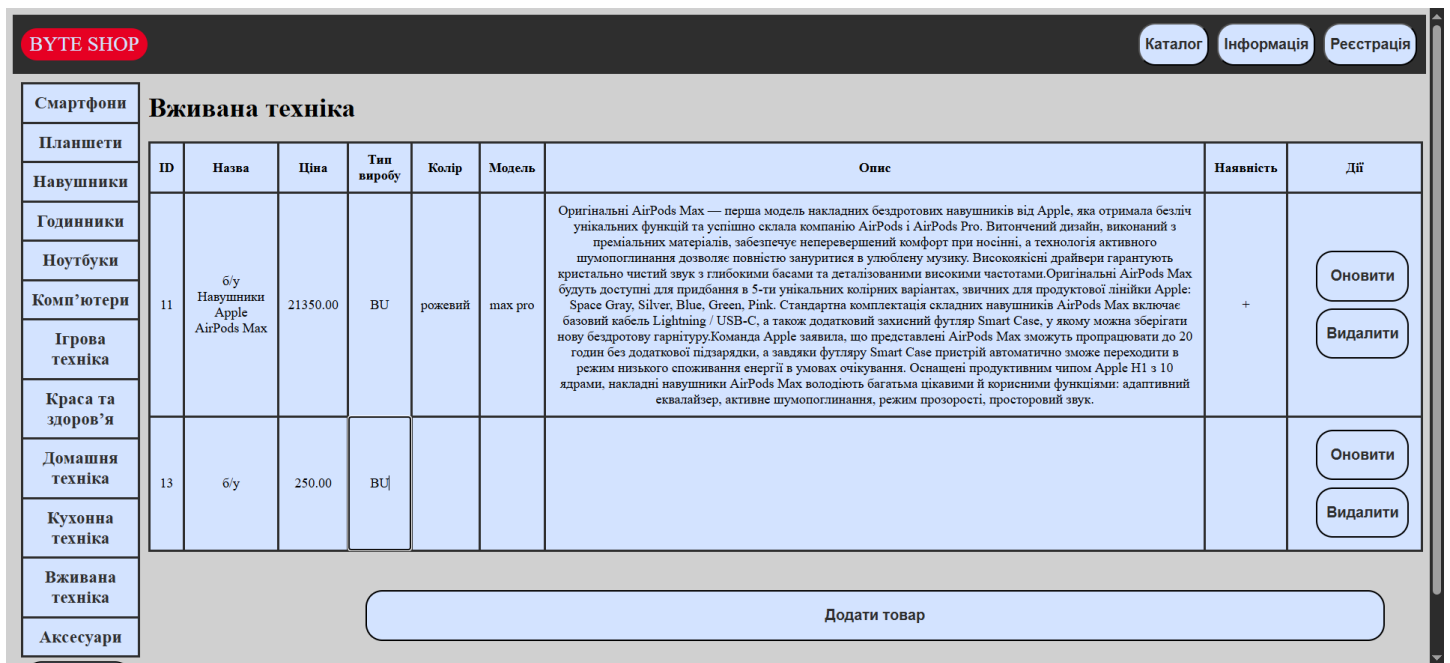


Рисунок 5.7 – додавання/оновлення певної характеристики у таблиці для операції PUT

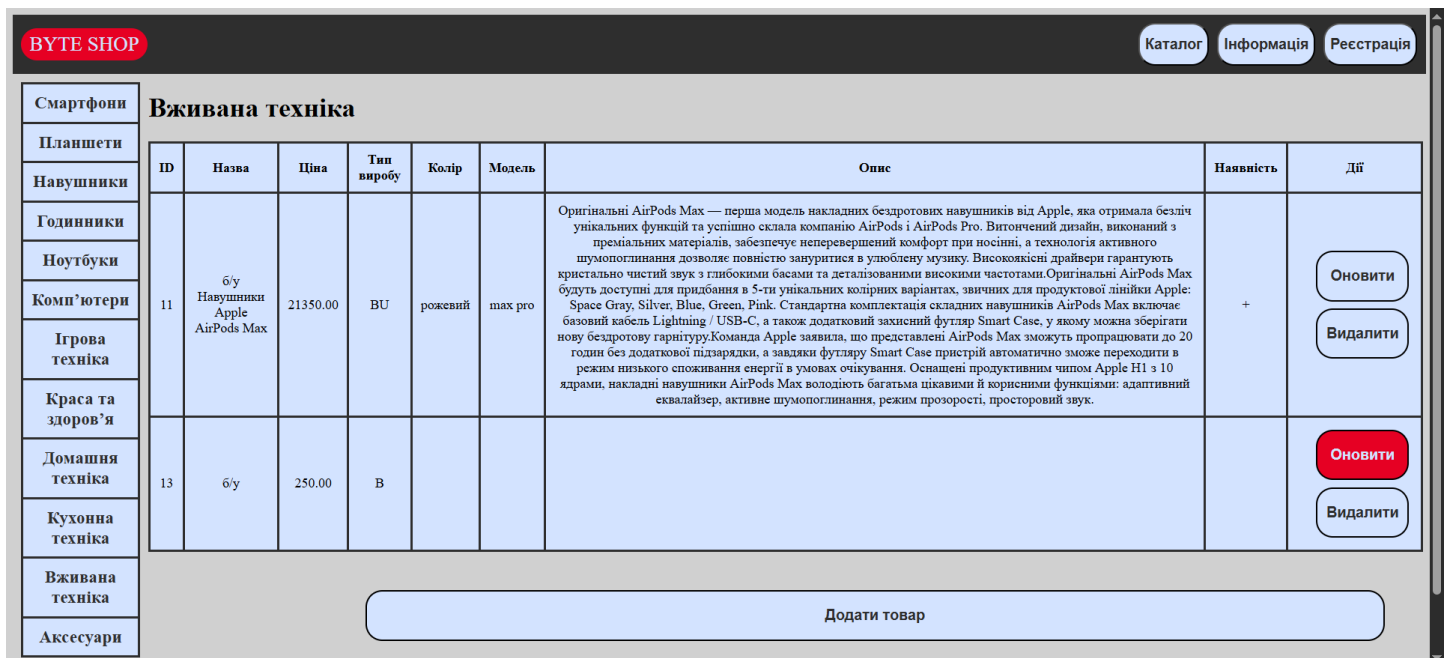


Рисунок 5.8 – результат операції PUT

5.4.4 Операція DELETE/DELETE

Операція DELETE використовується тоді, коли потрібно видалити товар з бази. Сервер отримує ID та виконує SQL-команду DELETE.

Код, де реалізований DELETE для адмін-панелі:

// DELETE: видалити виріб

```
app.delete('/products/:id', (req, res) => {
  const id = req.params.id;
  const sql = "DELETE FROM Product WHERE Product_Id=?";
  connection.query(sql, [id], (err, result) => {
```

```

if (err) {
    console.error('SQL error:', err);
    return res.status(500).json({ error: 'Server error' });
}

if (result.affectedRows === 0) return res.status(404).json({ error: 'Product not found' });
res.json({ message: 'Product deleted' });
});});

```

Приклад, де показано як працює DELETE:

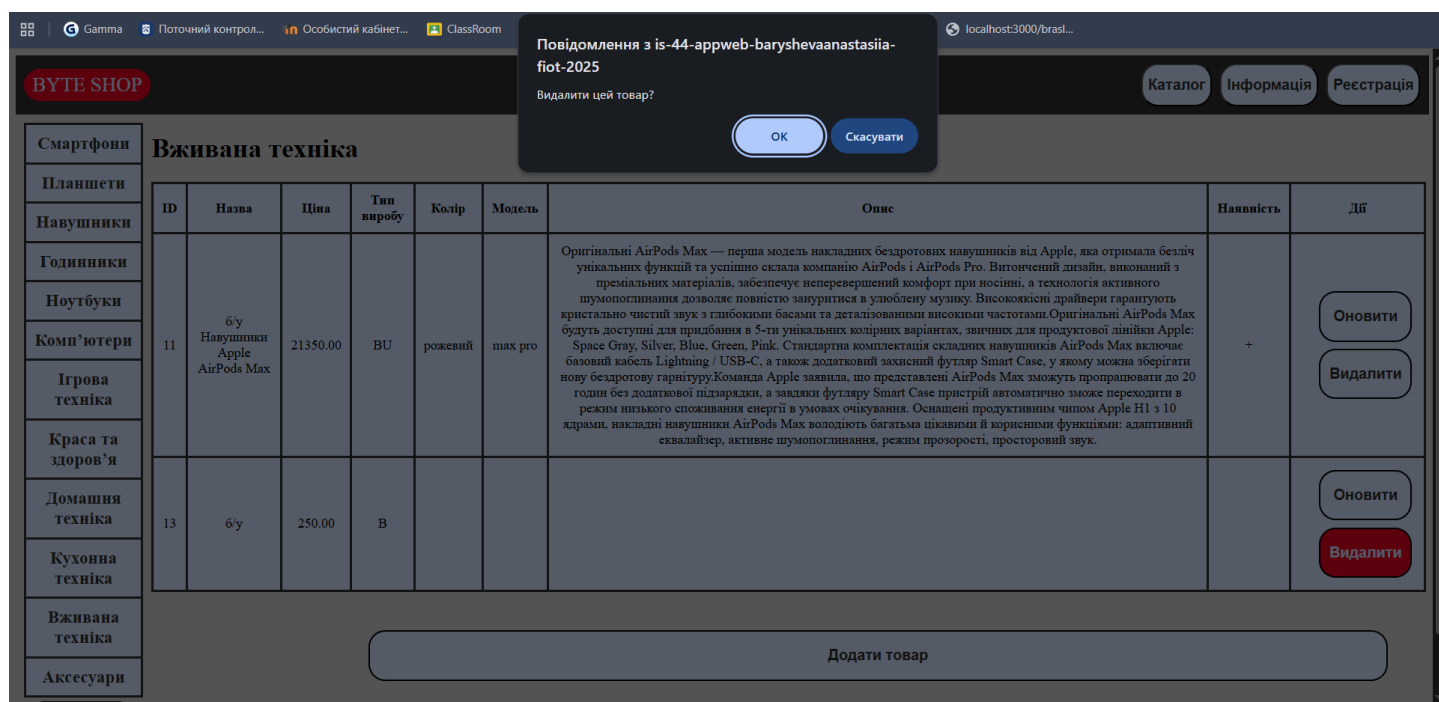


Рисунок 5.9 – натискання на кнопку «Видалити»

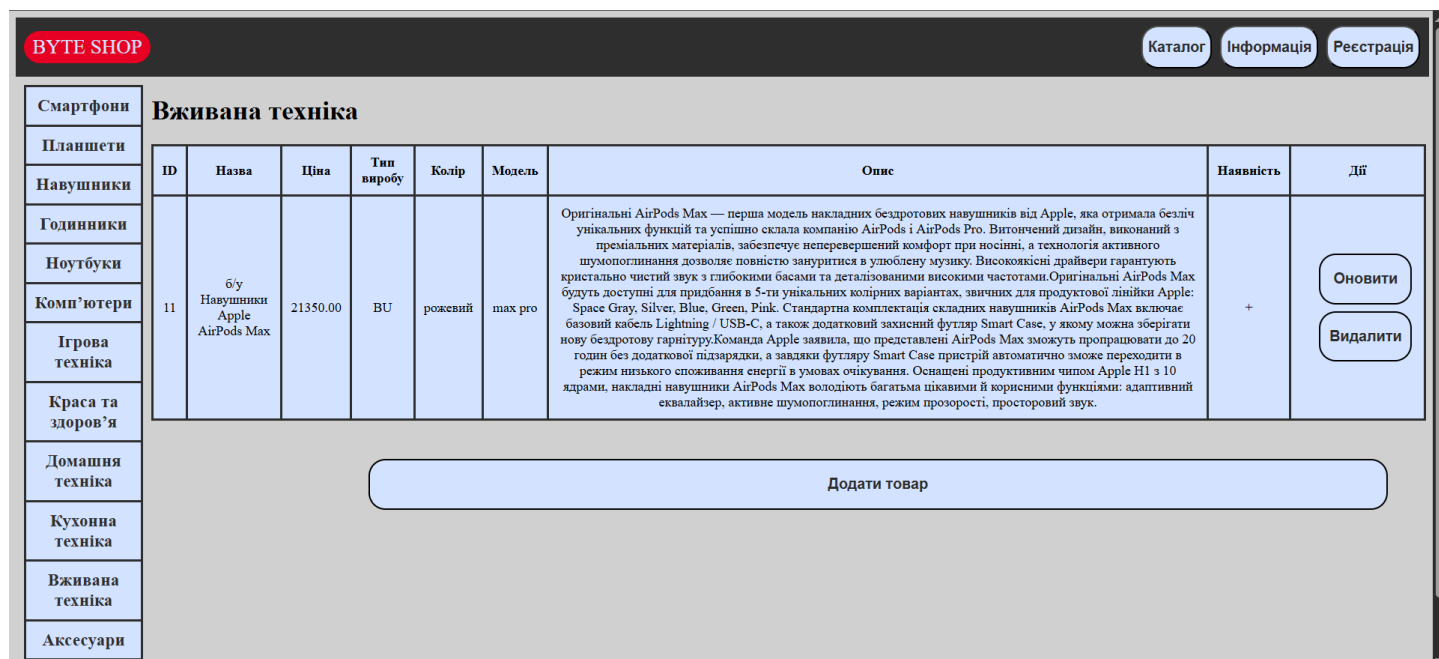


Рисунок 5.10 – результат виконання операції DELETE

5.5 Репозиторій на GitHub

Посилання на репозиторій на GitHub:

<https://github.com/SandraBSV/IS-44-appWEB-BaryshevaAnastasiia-FIOT-2025>

Коміт, який було зроблено:

```
feat(template): add all backend
```

1. add conect to data base
2. add CRUD-operations
3. add pages-js: main, product, type, logout, register, login and adminType
4. add all images for products
5. realization REST API functionals
6. refreshes some styles and add some styles detailes

5.6 Висновки до розділу

У процесі виконання практичної роботи було успішно досягнуто головної мети — отримання практичних умінь зі створення та налаштування серверної частини веб-застосунку на базі Node.js та Express, підключення реляційної бази даних MySQL та побудови REST API для взаємодії між клієнтською та серверною частинами.

Під час роботи було реалізовано низку важливих завдань.

По-перше, створено і налаштовано серверне середовище: розгорнуто сервер на Node.js та Express, організовано обробку запитів і відповідей у форматі JSON, підключено необхідні middleware для роботи з CORS та обробки JSON-даних. Це забезпечило коректну та стабільну комунікацію з фронтом.

Другим етапом стало підключення бази даних MySQL. Було створено конфігурацію з параметрами доступу та налагоджено виконання SQL-запитів до таблиць бази даних. Також реалізовано обробку можливих помилок під час встановлення з'єднання, що сприяло стабільності роботи застосунку.

Третім кроком стала розробка REST API для виконання CRUD-операцій над товарами. Було реалізовано отримання списку продукції та окремого товару за його ідентифікатором (GET), додавання нових записів (POST), оновлення існуючих (PUT) і видалення (DELETE). Для кожного типу запитів впроваджено обробку помилок і повернення відповідних HTTP-статусів.

Також було виконано інтеграцію з клієнтською частиною. На фронтенді налаштовано виклики API за допомогою fetch-запитів. Реалізовано пошук товарів,

виведення списку продукції, відображення детальної інформації про окремі вироби та застосування фільтрів за категорією, ціною, кольором і наявністю. Такий підхід дозволив оновлювати інформацію на сторінці динамічно, без її перезавантаження.

Окремим етапом проводилось тестування API. Перевірено роботу запитів GET, POST, PUT і DELETE, що підтвердило правильність реалізації CRUD-функціоналу, коректність передачі даних і стійкість серверної частини до некоректних запитів.

У результаті студент здобув низку важливих компетенцій: уміння налаштовувати серверне середовище, працювати з реляційними базами даних і SQL-запитами, проектувати REST API, інтегрувати фронтенд із бекендом через HTTP-запити, а також проводити тестування та налагодження серверної логіки.

Таким чином, мета практичної роботи була повністю досягнута, а отримані навички формують міцний фундамент для подальшої розробки повноцінних серверних частин веб-додатків на основі Node.js, Express та MySQL.

5.7 Контрольні запитання

1. Яке призначення бекенд-сервера у веб-застосунку?

Бекенд-сервер у веб-застосунку виконує роль центрального компонента для обробки бізнес-логіки, взаємодії з базою даних і надання даних фронтенду через HTTP-запити. Він приймає запити від клієнтської частини, обробляє їх відповідно до логіки застосунку та повертає відповіді у форматі, зручному для відображення на веб-сторінках.

2. Які основні компоненти складають структуру серверної частини?

Структура серверної частини зазвичай включає серверний движок, що обробляє запити (наприклад, Node.js з Express), систему маршрутизації запитів, модулі для роботи з базою даних, механізми обробки помилок і логування, а також middleware для обробки даних, авторизації, аутентифікації та взаємодії з клієнтом.

3. Які бібліотеки та інструменти використовуються для створення сервера на Node.js?

Для створення сервера на Node.js найчастіше застосовують фреймворк Express для організації маршрутизації та обробки запитів, бібліотеку mysql2 для підключення до

MySQL, cors для налаштування політик доступу з різних джерел, а також systemd для автоматичного перезапуску сервера під час розробки.

4. Як здійснюється підключення реляційної бази даних до сервера?

Підключення реляційної бази даних здійснюється через драйвер або бібліотеку, що підтримує відповідну СУБД. Наприклад, для MySQL використовується бібліотека mysql2, де задаються параметри підключення — хост, порт, ім'я користувача, пароль та назва бази. Сервер встановлює з'єднання і може виконувати SQL-запити для отримання та зміни даних.

5. Що таке REST API та які основні HTTP-методи в ньому використовуються?

REST API — це архітектурний стиль для організації взаємодії клієнта та сервера через HTTP-запити. Основні HTTP-методи, що застосовуються, це GET для отримання даних, POST для створення нових ресурсів, PUT або PATCH для оновлення існуючих ресурсів і DELETE для видалення ресурсів.

6. Які коди статусів HTTP повертаються під час виконання CRUD-операцій?

Під час виконання CRUD-операцій сервер повертає стандартні коди статусів: 200 OK — успішне отримання або оновлення даних, 201 Created — успішне створення нового ресурсу, 204 No Content — успішне видалення без тіла відповіді, 400 Bad Request — помилка клієнтського запиту, 404 Not Found — ресурс не знайдено, 500 Internal Server Error — помилка на сервері.

7. Як реалізується обробка помилок у серверному застосунку?

Обробка помилок реалізується через перевірку результатів виконання SQL-запитів або логіки сервера, використання конструкцій try/catch у асинхронних операціях та повернення користувачу зрозумілих повідомлень з відповідним HTTP-статусом. Це дозволяє уникнути аварійного завершення роботи сервера та інформувати фронтенд про проблеми.

8. Яким чином сервер взаємодіє з фронтендом?

Сервер взаємодіє з фронтендом через HTTP-запити та відповіді. Фронтенд надсилає запити з клієнтської частини (наприклад, через fetch у JavaScript) до серверного API, сервер обробляє ці запити, виконує необхідні операції над даними і повертає відповідь у форматі JSON, яку фронтенд використовує для оновлення інтерфейсу.

9. Що таке CORS і для чого його необхідно налаштовувати?

CORS (Cross-Origin Resource Sharing) — це механізм, що дозволяє або забороняє веб-

браузерам виконувати запити до серверів з інших доменів. Його необхідно налаштовувати для забезпечення безпечної взаємодії фронтенду та бекенду, щоб браузер дозволяв отримувати дані із зовнішніх або локальних серверів під час розробки та експлуатації.

10. Як перевіряється працездатність API та які інструменти для цього застосовуються?

Працездатність API перевіряється шляхом виконання тестових запитів до серверних маршрутів та аналізу отриманих відповідей. Для цього використовують інструменти Postman, Insomnia або вбудовані функції браузера та JavaScript (fetch), що дозволяють перевіряти GET, POST, PUT і DELETE запити, коди статусів та дані, що повертаються сервером.

11. Які переваги використання Node.js у порівнянні з традиційними серверними технологіями?

Node.js дозволяє створювати високопродуктивні асинхронні сервери на одній мові JavaScript, що уніфікує фронтенд і бекенд. Він забезпечує неблокуючий ввід/вивід, масштабованість, швидкість обробки великої кількості одночасних з'єднань і велику кількість готових бібліотек через npm, що робить розробку більш гнучкою та ефективною.

6 АВТОРИЗАЦІЯ ТА РОЗМЕЖУВАННЯ ПРАВ ДОСТУПУ (ПРАКТИЧНА РОБОТА 6)

Мета роботи:

Мета роботи набути практичних навичок реалізації авторизації користувачів у веб-застосунку з розмежуванням доступу за ролями.

6.1 Реалізація моделі користувачі і ролей

Під час розробки системи доступу була створена модель користувачів, яка включає три основні таблиці: Customer, Role та UserRole. Ця структура реалізує класичний підхід RBAC (Role-Based Access Control), при якому права доступу визначаються не конкретним користувачем, а його роллю, що значно полегшує масштабування системи.

Таблиця Customer містить персональні дані користувачів, такі як ім'я, прізвище, номер телефону, email та хешований пароль. Таблиця Role описує доступні ролі в системі, і в проєкті використовуються дві з них: “Адміністратор” (1) та “Користувач” (2). Таблиця UserRole встановлює відповідність між конкретним користувачем та його роллю.

Під час реєстрації через фронтенд у таблиці Customer автоматично створюється новий запис, після чого користувачу за замовчуванням присвоюється роль “Користувач”. Такий підхід дозволяє ефективно керувати доступом до різних ресурсів, перевіряти роль користувача на сервері та формувати відповідний інтерфейс на фронтенді.

Код реалізації створення користувача на сервері:

```
app.post('/register', async (req, res) => {  
  const { Customer_FirstName, Customer_LastName, Customer_Phone, Customer_Email,  
Customer_Password } = req.body;  
  if (!Customer_FirstName || !Customer_LastName || !Customer_Phone || !Customer_Email ||  
!Customer_Password) {  
    return res.status(400).json({ error: 'Недостатньо даних для реєстрації' });  
  }  
})
```

// Перевірка на існуючого користувача

```
const checkSql = `SELECT Customer_Id FROM Customer WHERE Customer_Email = ?
OR Customer_Phone = ?`;

connection.query(checkSql, [Customer_Email, Customer_Phone], async (err, rows) => {
  if (err) return res.status(500).json({ error: 'Помилка сервера' });
  if (rows.length > 0) return res.status(409).json({ error: 'Користувач з таким email або
телефоном вже існує' });

  const hashedPassword = await bcrypt.hash(Customer_Password, 10);
  const insertSql = `INSERT INTO Customer (Customer_FirstName,
Customer_LastName, Customer_Phone, Customer_Email, Customer_Password)
VALUES (?, ?, ?, ?, ?)`;

  connection.query(insertSql, [
    Customer_FirstName,
    Customer_LastName,
    Customer_Phone,
    Customer_Email,
    hashedPassword
  ], (err, result) => {
    if (err) return res.status(500).json({ error: 'Помилка вставки' });
    const customerId = result.insertId;
    // Додаємо роль User
    const roleSql = `INSERT INTO UserRole (Customer_Id, Role_Id) VALUES (?, 2)`;
    connection.query(roleSql, [customerId], (err2) => {
      if (err2) {
        connection.query('DELETE FROM Customer WHERE Customer_Id = ?',
[customerId]);

        return res.status(500).json({ error: 'Помилка при призначенні ролі' });
      }
      return res.status(201).json({ success: true, customerId, role: 2 });
    });
  });
});
});
});
```

Приклад реєстрації користувача:

BYTE SHOP

Реєстрація

Прізвище:

Ім'я:

Номер телефону:

Електронна адреса:

Пароль:

[Вже маєте кабінет?](#)

Зареєструватись

Рисунок 6.1 – заповнення форми для реєстрації на сторінці pageRegister.html

BYTE SHOP

Каталог Інформація Cabinet

Кошик

Особиста інформація

Архів замовлень

Вийти

Good to see you, dear User! :)

Рисунок 6.2 – після натискання на кнопку «Зареєструватись», відкривається сторінка кабінет для користувача

23 ім'я	прізвище	012 345 67 89	adres@gmail.com	\$2b\$10\$QGQ3HL0yx78No.HdMx4Wn.tEtavZIQ7dlWdOqk0ihLW...
---------	----------	---------------	-----------------	--

Рисунок 6.3 – як наслідок у таблиці Customer у базі даних, бачимо, є новий користувач

6.2 Розробка механізму автентифікації

Автентифікація здійснюється через REST API, який приймає запит на вхід і перевіряє email та пароль користувача. На сервері пароль не перевіряється у відкритому вигляді, а порівнюється з хешем за допомогою бібліотеки bcrypt.

Після успішної автентифікації сервер повертає структуровану відповідь, що містить ідентифікатор користувача, його роль та статус операції. Ці дані зберігаються у localStorage на фронтенді, що дозволяє підтримувати стан авторизації під час навігації між сторінками.

Такий підхід забезпечує базовий рівень безпеки та дає можливість фронтенду адаптувати інтерфейс залежно від ролі користувача.

Код автентифікації:

```
app.post('/login', (req, res) => {
  const { identifier, password } = req.body;
  if (!identifier || !password) return res.status(400).json({ error: 'Введіть логін і пароль' });
  const sql = `
    SELECT c.Customer_Id, c.Customer_Password, ur.Role_Id
    FROM Customer c
    LEFT JOIN UserRole ur ON c.Customer_Id = ur.Customer_Id
    WHERE c.Customer_Email = ? OR c.Customer_Phone = ?`;
  connection.query(sql, [identifier, identifier], async (err, rows) => {
    if (err) return res.status(500).json({ error: 'Помилка сервера' });
    if (rows.length === 0) return res.status(401).json({ error: 'Користувача не знайдено' });
    const dbPassword = rows[0].Customer_Password;
    // --- ПОРІВНЯННЯ ХЕШУ ---
    const passwordMatch = await bcrypt.compare(password, dbPassword);
    if (!passwordMatch) {
      return res.status(401).json({ error: 'Невірний пароль' });
    }
    const customerId = rows[0].Customer_Id;
    const roles = rows.map(r => r.Role_Id).filter(r => r !== null);
    let role = 2;
    if (roles.includes(1)) role = 1;
    return res.json({ success: true, customerId, role });
  });
});
```

Приклад реєстрації адміністратора:

BYTE SHOP Регістрація

Електронна пошта:

Пароль:

[Зареєструватися](#)

Увійти

Рисунок 6.4 - заповнення форми для авторизації на сторінці pageLogin.html

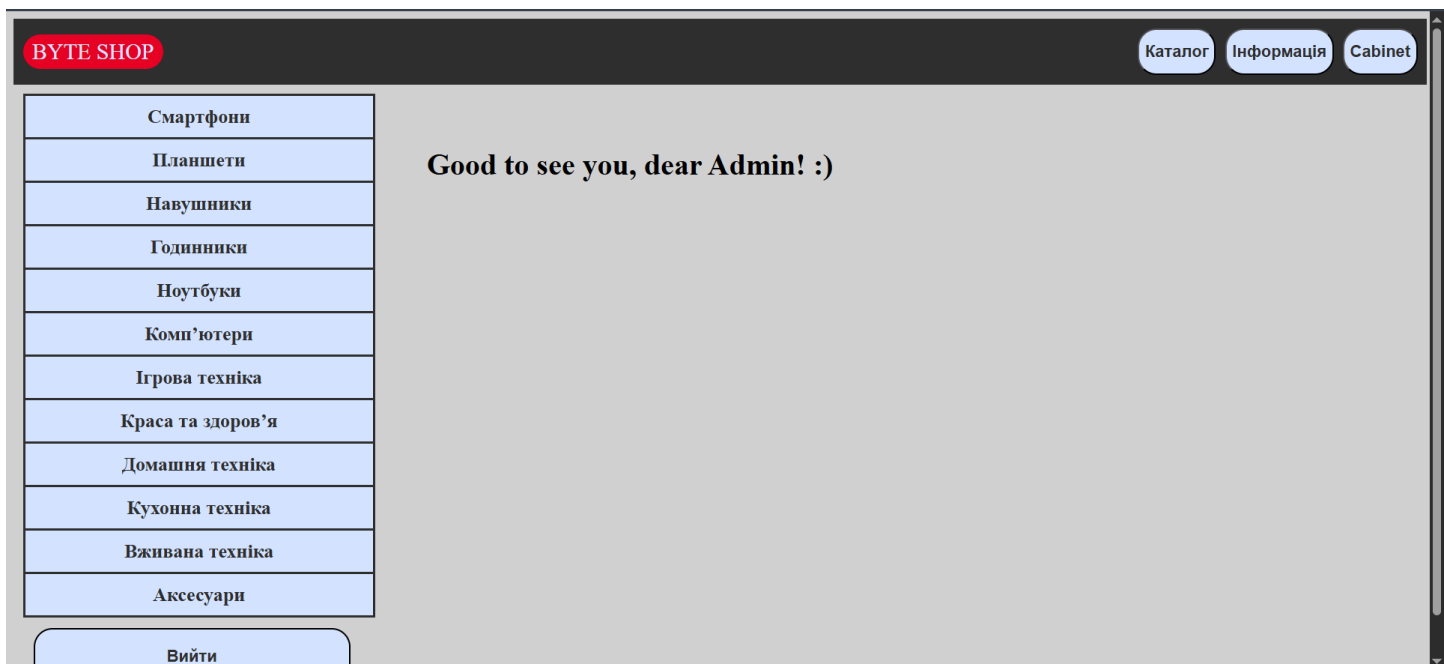


Рисунок 6.5 – після натискання на кнопку «Увійти», відкривається сторінка адмін панелі для адміністратора

6.3 Розмежування прав доступу

Права доступу реалізовані шляхом перевірки ролі користувача. Логіка контролю доступу застосовується як на сервері через Role_Id, так і на фронтенді. Після успішної авторизації клієнт отримує роль, яка визначає, на яку сторінку буде перенаправлено користувача: адміністратор переходить на сторінку керування (pageAdmin.html), а звичайний користувач — у персональний кабінет (pageCabinet.html).

Фронтенд здійснює перевірку ролі при завантаженні кожної сторінки, що дозволяє реалізувати базовий рівень RBAC та формувати інтерфейс відповідно до прав користувача.

6.4 Безпека API та обробка помилок

Усі запити обробляються всередині блоків try/catch, і сервер повертає стандартизовані помилки у форматі JSON. Паролі ніколи не зберігаються у відкритому вигляді, а тільки у хешованому вигляді за допомогою bcrypt.hash().

Реалізовано перевірку вхідних даних, запобігання дублюванню email та використання безпечних SQL-запитів із параметрами. Крім того, система забезпечує зручну взаємодію з користувачем: у разі помилки на фронтенді відображається інформативне повідомлення, що допомагає зрозуміти причину проблеми.

Код обробки помилок у файлі авторизації:

```
try {
  const res = await fetch(`${API_BASE_URL}/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ identifier, password })
  });
  const data = await res.json();
  if (!res.ok) {
    alert(data.error || 'Помилка при вході');
    return;
  }
  // Очікуємо від сервера { success: true, customerId: X, role: Y }
  localStorage.setItem('userId', data.customerId);
  localStorage.setItem('userRole', String(data.role));
  // Переадресація за роллю: 1 -> адмін панель, 2 -> кабінет
  if (Number(data.role) === 1) {
    window.location.href = '../admin/index.html';
  } else {
    window.location.href = '../pages/pageCabinet.html';
  }
} catch (err) {
  console.error(err);
}
```

```
alert('Не вдалося зв'язатися з сервером.');
```

Код обробки помилок у файлі реєстрації:

```
try {
  const res = await fetch(`${API_BASE_URL}/register`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      Customer_FirstName: firstName,
      Customer_LastName: lastName,
      Customer_Phone: phone,
      Customer_Email: email,
      Customer_Password: password
    })
  });
  const data = await res.json();
  if (!res.ok || !data.success) {
    alert(data.error || "Сталася помилка при реєстрації.");
    return;
  }
  // Зберігаємо логін
  localStorage.setItem('userId', data.customerId);
  localStorage.setItem('userRole', String(data.role)); // 2 = User
  // Перехід в особистий кабінет
  window.location.href = '../pages/pageCabinet.html';
} catch (err) {
  console.error(err);
  alert("Не вдалося підключитися до сервера.");
}
```

6.5 Інтеграція з фронтом

Було створено три JavaScript-файли для реалізації функціоналу авторизації, реєстрації та виходу користувача. У register.js відбувається відправка POST-запиту на

/register, у login.js — POST-запиту на /login, а logout.js відповідає за очищення localStorage та перенаправлення на головну сторінку.

Дані форм отримуються через DOM і відправляються на сервер у форматі JSON. Після успішної авторизації у локальному сховищі зберігаються userId та userRole, що дозволяє фронтенду динамічно змінювати інтерфейс, відображення кнопок та маршрути переходів відповідно до ролі користувача.

Код реалізації у файлі register.js:

```
const lastName = form.querySelector('input[name="firstname"]').value.trim();
const firstName = form.querySelector('input[name="lastname"]').value.trim();
const phone = form.querySelector('input[name="phone"]').value.trim();
const email = form.querySelector('input[name="email"]').value.trim();
const password = form.querySelector('input[name="password"]').value;
const res = await fetch(`${API_BASE_URL}/register`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    Customer_FirstName: firstName,
    Customer_LastName: lastName,
    Customer_Phone: phone,
    Customer_Email: email,
    Customer_Password: password
  })
});
```

Код реалізації у файлі logout.js:

```
// Видаляємо дані авторизації
localStorage.removeItem("userId");
localStorage.removeItem("userRole");
// Переадресація на головну
window.location.href = "../index.html";
```

6.6 Тестування авторизації та прав доступу

Було проведено повне тестування всіх можливих сценаріїв використання системи. Перевірялися такі випадки: успішна реєстрація нового користувача, успішний вхід у систему, спроба входу з неправильним паролем, логін з неіснуючим email, доступ до сторінок, що вимагають авторизації, доступ до сторінок, доступних лише адміністраторам, сценарій виходу з акаунта та коректність відображення інтерфейсу залежно від ролі користувача.

Результати тестування підтвердили стабільну та надійну роботу системи.

Код реалізації перевірки ролі:

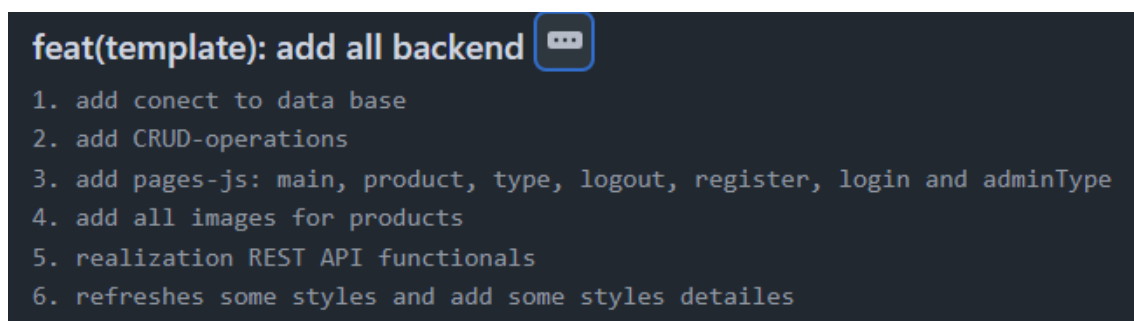
```
// Переадресація за роллю: 1 -> адмін панель, 2 -> кабінет
if (Number(data.role) === 1) {
  window.location.href = '../admin/index.html';
} else {
  window.location.href = '../pages/pageCabinet.html';
}
```

6.7 Репозиторій на GitHub

Посилання на репозиторій на GitHub:

<https://github.com/SandraBSV/IS-44-appWEB-BaryshevaAnastasiia-FIOT-2025>

Коміт, який було зроблено:



6.8 Висновки до розділу

У рамках виконання практичної роботи було повністю реалізовано механізм авторизації користувачів у веб-застосунку з підтримкою ролей і контролем доступу. Було створено модель користувачів і ролей, що дозволяє розділяти права між різними категоріями користувачів, зокрема звичайними користувачами та адміністраторами.

Це забезпечило гнучку систему доступу, де кожна роль має визначений набір дозволених дій.

Розроблено та впроваджено механізм автентифікації, який включає хешування паролів, перевірку введених даних та контроль наявності користувачів у системі. Особлива увага приділена безпеці: паролі перевіряються через bcrypt, а можливі помилки обробляються на всіх етапах авторизації. Це підвищує стійкість системи до SQL-помилки, некоректного вводу та спроб доступу з неправильними обліковими даними.

Для розмежування прав доступу реалізовано middleware, що перевіряє ролі користувачів і обмежує доступ до окремих API-маршрутів та захищених ресурсів. Такий підхід гарантує, що критичні частини застосунку, зокрема адміністративні функції, залишаються недоступними для неавторизованих або недостатньо привілейованих користувачів.

Система також підтримує коректну обробку помилок API: сервер повертає відповідні HTTP-коди та чіткі повідомлення, що спрощує діагностику й підвищує стабільність роботи. Для інтеграції з фронтендом були створені файли login.js, register.js та logout.js, додані відповідні кнопки в HTML-інтерфейсі та реалізовано збереження токена авторизації у браузері. Це дозволяє фронтенду підтримувати актуальний стан авторизації та адаптувати інтерфейс залежно від ролі користувача.

Заключним етапом роботи стало тестування маршруту /auth/login, перевірка доступу до публічних, приватних та адміністративних API-ендпоінтів, а також оцінка правильності обробки серверних помилок. Тестування підтвердило коректну роботу всієї системи авторизації, включно з валідацією користувача, генерацією токена, перевіркою ролей та керуванням інтерфейсом на стороні клієнта.

Отже, у ході виконання практичної роботи було успішно реалізовано повноцінну систему авторизації, яка відповідає сучасним вимогам безпеки та забезпечує надійний контроль доступу до ресурсів веб-застосунку.

6.9 Контрольні питання

1. Що таке автентифікація та авторизація, і чим вони відрізняються?

Автентифікація — це процес перевірки особи користувача, під час якого система визначає, чи є введені логін і пароль дійсними. Авторизація — це наступний етап, коли система вирішує, чи має автентифікований користувач право виконувати певні дії або отримувати доступ до ресурсів. Відмінність полягає в тому, що автентифікація підтверджує особу, а авторизація визначає доступні їй можливості.

2. Які основні компоненти системи авторизації реалізовано у вашому проєкті?

У проєкті реалізовано базу користувачів із ролями, механізм реєстрації та входу з перевіркою пароля, генерацію та перевірку токена, middleware для обмеження доступу за ролями, а також маршрути, які поділяються за рівнем доступу для звичайних користувачів і адміністратора.

3. Який формат має JWT-токен і з яких частин він складається?

JWT-токен складається з трьох частин: заголовка, тіла (або payload) і підпису. Заголовок визначає алгоритм підпису, payload містить дані користувача та його роль, а підпис гарантує цілісність токена. Усі частини закодовані у форматі Base64 і розділені крапками.

4. Як у системі реалізовано перевірку ролі користувача під час виконання запиту?

Після авторизації токен, який містить роль користувача, передається у запитах до сервера. Middleware розшифровує токен, зчитує роль і визначає, чи має користувач право на доступ до конкретного маршруту. Якщо роль не відповідає вимогам маршруту, сервер повертає помилку доступу.

5. Як захищається пароль користувача під час зберігання у базі даних?

Пароль ніколи не зберігається у відкритому вигляді. Під час реєстрації він хешується за допомогою алгоритму bcrypt, який використовує випадкову "сіль" і множинні обчислювальні раунди, що робить злам пароля практично неможливим навіть у випадку витоку бази даних.

6. Які коди статусів HTTP використовуються для повідомлення про помилки авторизації?

Для помилок автентифікації використовується код 401, який означає, що користувач неавторизований. Код 403 застосовується у випадку, коли користувач авторизований, але не має достатніх прав для виконання запиту. Код 400 використовується при некоректних вхідних даних, а 409 — при конфлікті, наприклад, якщо користувач уже існує.

7. Як у фронтенді організовано збереження та використання токена доступу?

Після успішного входу отриманий токен зберігається у localStorage або sessionStorage браузера. Під час виконання запитів до захищених маршрутів він додається до заголовків HTTP у полі Authorization. На основі токена інтерфейс також змінює стан системи: показує або приховує певні сторінки та кнопки.

8. Які можливі загрози безпеці при неправильному використанні JWT і як їх уникнути?

Основні загрози включають зберігання токена у незахищених місцях, його перехоплення через XSS, використання надто довгого строку життя та слабкий секрет підпису. Щоб уникнути проблем, потрібно зберігати токен у безпечному сховищі, мінімізувати строк його дії, регулярно оновлювати секретний ключ і коректно налаштовувати заголовки безпеки у браузері.

9. Яким чином реалізовано поділ маршрутів на публічні, користувацькі та адміністративні?

Публічні маршрути доступні всім користувачам без потреби входу. Користувацькі маршрути вимагають наявності дійсного токена, що підтверджує особу.

Адміністративні маршрути додатково перевіряють роль користувача, і лише токен з роллю "адмін" може отримати доступ до таких операцій, як редагування чи видалення даних.

10. Які інструменти використовуються для тестування процесів авторизації?

Для тестування використовуються інструменти запитів, такі як Postman або вбудовані можливості браузера, що дозволяють перевіряти запити до серверу, надсилати токени, тестувати помилки авторизації та переглядати відповіді API. Вони дають можливість відтворювати реальні сценарії входу, реєстрації й доступу до захищених маршрутів.

